

MASTER THESIS

SDP RELAXATIONS FOR QUANTUM INFLATION

Julius Alexander Zeiss



UNIVERSITY OF COLOGNE
INSTITUTE FOR THEORETICAL PHYSICS

SUPERVISED BY:

Laurens Ligthart

SUPERVISOR AND 1. EXAMINER:

Prof. Dr. David Gross

2. EXAMINER:

PD. Dr. Rochus Klesse

July 23, 2022

Abstract

A fundamental endeavour in science is the study of causal relations which yield a possible explanation for observed statistics. The *causal compatibility problem* questions whether a given joint probability distribution over a set of observed variables is *compatible* with a physical process adhering to a causal hypothesis. A complicated problem when we allow unobserved variables to be causally related to other observed or unobserved variables. Most notably, Bell showed that the set of compatible distributions arising from classical processes differs from those which necessitate a quantum origin. With recent attention and effort focused on developing quantum technologies exploiting quantum effects to potentially provide superior performance over their classical counterpart, quantifying this classical to quantum gap is of current practical relevance. The recently developed so-called *inflations* are designed to relax non-linear and non-convex independence relations in a causal hypothesis to linear symmetry relations so that the compatibility question can ultimately be reformulated as a *convex optimisation* problem. To do so, the quantum inflations exploit *non-commutative polynomial optimisation theory* to construct a complete hierarchy of *semidefinite programming (SDP) relaxations*. We generalise *scalar extension* to quantum inflation to construct more constraint SDPs at each hierarchy step. Convex optimisation problems have been extensively studied, and accurate computational methods exist to solve them. We introduce an effective procedure to vastly reduce the computational complexity of this hierarchy of SDP relaxations. We obtain new results quantifying the incompatibility of the *W*- and *GHZ*-distributions where the causal hypothesis is given by the *quantum triangle scenario*. Additionally, we exploit *SDP duality theory* to provide compatibility conditions on any joint probability distribution relative to this causal hypothesis.

Acknowledgement First and foremost, I would like to express my deepest gratitude to Prof. Dr. David Gross and Laurens Ligthart. I am thankful to David for any necessary realignment in times of confusion and for his unquestionable support in times of struggle. Laurens, my mentor, I learned from you not only in areas of physics but, maybe more importantly, in life. You are indeed a wonderful teacher and a beautiful mind. I am so thankful for all the time you have invested in me.

I am thankful to Elie Wolfe for all our correspondence on causality and inflations. Concretely, for our discussions on positive sets, certificate optimisation and SDP implementations. Your advice on challenges in academia was truly appreciated. I am thankful to Alejandro Pozas-Kerstjens for comparisons of numerical results from our individual implementations. Thank you Nikolai Miklin and Bora Ulu for the insightful discussions on SDP solvers. I am also grateful to Prof. Dr. Angelika Wiegele and her working group for the collaboration to tackle large-scale SDPs arising in quantum inflation.

To all members of David's working group: Felipe, Paulina, Vahide, Mark, Fabian, Arne, Vallentin, Amir, Markus and Yaiza. You made my time in the group truly wonderful. I am so thankful for your support in times of struggle and for all the advice I so helplessly needed.

Mein Dank gilt auch Yannic und Gregor, meinen Gefährten auf der Reise durch die Physik. Auch danke ich Jan für jedes Wort der Zuversicht und jeden Schritt im Stadtwald. Antonia, Paul und Constanze, ihr seid immer für mich da und ich hoffe, dass ich euch etwas stolz machen kann. Ich danke meinen Eltern für ihre unermüdliche Unterstützung. Danke, Mum und Dad, dass ihr es mir ermöglicht habt, zu träumen, zu lernen, zu wundern und ein wenig mehr von dieser Welt zu verstehen, welche mir so häufig ganz unverständlich schien.

Meine Pia, deine tiefen Töne ebnen mir den Weg in jeder Dunkelheit. Ich danke dir für all die Mühen, die Worte der Unterstützung und für jedes Fragment von Zeit gefüllt mit reiner Glückseligkeit.

Contents

1 Introduction	1
1.1 Motivation	1
1.2 Outline and relevance	2
2 Mathematical Preliminaries	4
2.1 Probability theory	4
2.2 Graph theory	8
2.3 Convex optimisation	12
2.3.1 The positive semidefinite matrix	12
2.3.2 Convexity	13
2.3.3 Semidefinite programmes	14
2.3.4 Duality theory	16
2.3.5 Eigenvalue optimisation problems	18
2.3.6 Solving semidefinite programmes	19
2.4 Abstract algebra	20
2.4.1 C^* -algebras and the GNS construction	20
2.4.2 The free $*$ -algebra	24
2.4.3 The universal C^* -algebra	26
2.4.4 Ideals from equality constraints	28
2.4.5 Tensor products of C^* -algebras	28
2.5 Non-commutative polynomial optimization theory	31
2.5.1 SDP relaxations	32
3 The Causal Compatibility Problem	37
3.1 The classical variant	37
3.2 The quantum variant	40
3.2.1 Modelling locality in quantum causal structures	41
3.2.2 Graphical description of quantum causal structures	43
4 Inflations	50
4.1 Classical inflation	50
4.1.1 Inflated causal structures	51
4.1.2 Injectable and ai-expressible sets	51
4.2 Quantum inflation	56
4.2.1 Automorphisms of inflated quantum structures	57
4.2.2 Quantum realisations of inflated quantum structures	59
4.2.3 Symmetry invariance	60

4.2.4	State factorisation	61
4.2.5	Quantum variants of injectable and ai-expressible sets	63
4.2.6	The QCCP to inflated quantum causal models	65
4.3	Ligthart-Gachechiladze-Gross hierarchy	68
5	Scalar Extension	69
5.1	Scalar extension for quantum inflation	70
6	Application - The Quantum Triangle Scenario	77
7	Implementation	80
7.1	Core programme	80
7.1.1	Alternative partitioning of $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$	80
7.1.2	Intermediate hierarchy levels	82
7.1.3	The monomial basis to the quotient algebra	83
7.1.4	Real symmetric $\Gamma^{(k)}$	87
7.1.5	$\Gamma^{(k)}$ construction	88
7.1.6	Involution equivalence	89
7.1.7	Orbits of $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}$	90
7.1.8	Factorisation	91
7.1.9	The ai-expressible set	93
7.1.10	SDP reformulation	95
7.2	Problem-specific programme	97
7.2.1	Visibility tests	97
7.2.2	Certificates and witnesses	98
7.2.3	Exploiting moment classifications	100
7.2.4	Scalar extension	102
7.3	Numerical implementation	103
7.3.1	Solvers	103
7.3.2	MOSEK Fusion and Optimizer API	104
8	Numerical results	105
9	Conclusions	107
A	Dual formulation for an SDP in standard form	109
B	Scalar extension case (3) visualisation	110
C	Pseudo code $D_R\Gamma^{(k)}$ and $D_S(\Gamma^{(k)})$ construction	111

D PSDP and DSDP with localising matrix Λ	112
E Probability distributions	114
F MOSEK Optimiser API SDP formulation [4]	115
G Computer science vs. physics notation	116
H Source code python implementation	117
References	161
Statement of Autorship	168

1 Introduction

We begin by outlining the motivation behind the study of *semidefinite programming (SDP) relaxations for quantum inflation* with reference to existing research relevant to this thesis. Next, we will elaborate on the structure of this thesis. We conclude this section by highlighting the relevance of this thesis in the context of other work in this field.

1.1 Motivation

A fundamental endeavour in science is the study of causal relations. Whether it be in medicine, economics¹ or physics, the search for a causal explanation for observed phenomena is central to the formulation of any predictive theory. Consider obtaining non-temporal data as a joint probability distribution over measured variables corresponding to observations in an experiment. Is it possible to infer causal relations from that data alone? The theory of *inferred causation* [62] tackles that question. At the basis of this theory lies the realisation that “statistical association (correlation) does not logically imply causation [62]”. Nevertheless, we can sometimes infer causal relations from the “structure” of the empirical data. Notably, *Reichenbach’s common-cause assumption* formulates a relation between causal and statistical concepts.

Theorem 1.1 (Reichenbach’s common-cause assumption [70]). *If two variables are dependent (correlated), then either there is a causal connection between them or there is a third variable (common cause), which causes the dependency between the other two variables.*

When we allow such common causes to be unobserved, inferring causal relations becomes more complicated. For simplicity, this work considers a more concrete class of problems. Whereas the formulation of the *causal compatibility problem (CCP)* is relatively straightforward, deriving a (general) solution is not [62].

Problem 1.1 (Causal compatibility problem (CCP)). *Is a given joint probability distribution P over observed variables in an experiment compatible with a hypothesis about the causal relations between variables, i.e. does there exist a process adhering to the proposed causal relations that yields P ?*

We are interested in the quantum variant of the CCP. There, we allow unobserved processes to be of a quantum nature and the observed statistic to arise partly from measurements on these quantum systems. Quantum theory introduces a number of

¹The 2021 Nobel memorial price in economics was awarded for work on causality.

“peculiarities” such as entanglement [39] or the no-cloning theorem [89] without an analogue in a classical setting. As a result, there exists a boundary between statistics which can arise from mere classical processes and those which necessitate a quantum origin. Most notably, Bell [9] showed the non-locality [67] of quantum theory by deducing that a local hidden variable theory as proposed by Einstein, Podolski and Rosen [26] does not explain the statistics obtained from performing space-like separated measurements on entangled quantum systems. Hence, a classical to quantum gap exists concerning the statistics compatible with a given causal hypothesis. Quantifying this gap is an essential undertaking in quantum information science. Many concepts of recent relevance and attention, such as quantum computers, quantum communication and cryptography, depend on a clear distinction between quantum and classical correlations and benefit from improvements in our ability to precisely determine this classical-quantum gap. For example, consider quantum information protocols [87, 27, 6] in quantum cryptography. Thus, the problem considered in this thesis is of current practical relevance.

While the literature on the classical CCP is vast, substantial progress on the quantum variant concerning more complex causal hypotheses than those considered by Bell has only recently been made. Various approaches exist to solving the classical CCP (QCCP) [62, 32, 18, 45] and to the quantum CCP [17, 55, 54, 69]. This thesis is concerned with a particular class of approaches to solving the causal compatibility problems with unobserved variables: so-called *inflations*. Our motivation to focus on these inflation procedures is simple: they provide a general and numerically viable framework to tackle the quantum and classical variants of the CCP, which additionally have proven to exceed the other methods in terms of accuracy.

The inflation algorithms allow us to reformulate the CCP in terms of a *convex optimisation* problem. The *classical inflation procedure* [86] yields a linear programming hierarchy. The quantum variants, so-called *quantum inflation* [87] and the *Ligthart-Gachechiladze-Gross (LGG) hierarchy* [46] exploit the *non-commutative polynomial optimisation (NPO) theory* [65] to cast the NPO problem corresponding to the quantum CCP as a hierarchy of SDP relaxations. This thesis focuses on formulating these SDP relaxations to quantify the incompatibility of statistics and causal hypotheses, delimiting the classical from the quantum realm.

1.2 Outline and relevance

The first part of this thesis is dedicated to the mathematical preliminaries on which the subsequent sections heavily depend. We introduce the relevant concepts from prob-

ability theory, graph theory, convex optimisation and abstract algebra. We conclude the section with an introduction to NPO theory.

Next, we formally introduce classical and quantum causal models. We link statistics corresponding to experimental data to causal structures depicted by graphs. Lastly, we provide a quantum variant of the CCP.

In the inflation section, we introduce the corresponding procedures proposed to tackle the CCP. The quantum inflation and the LGG hierarchy are suited to solve both the classical and quantum variants of the CCP. The former is the focus of this thesis. Since the classical inflation procedure is limited to solving the classical CCP, it mainly introduces the idea behind inflation procedures. Furthermore, the section highlights the conceptual differences we face when dealing with problems arising in the quantum realm.

We then introduce a generalisation of the *scalar extension* procedure [69] and argue its relevance in the quantum inflation framework. We believe this adaptation to be a new result.

The motivation for applying the concepts to the *quantum triangle scenario* is twofold. First, it allows us to make the theoretical description of the previous sections more tangible. Second, solving the quantum CCP for several probability distributions relative to this scenario constitutes one of the main results of this thesis. In the subsequent sections, we elaborate on this implementation and the numerical results obtained. We introduce several concepts allowing us to formulate the SDP relaxations efficiently, vastly reducing the computational complexity an SDP solver faces in determining its solution. Additionally, we deduce compatibility constraints that any compatible distribution necessarily needs to adhere to as a result of a given causal hypothesis. We believe some of these concepts and results to be new. While [87] generalises the idea behind the classical inflation to a quantum realm, it does not provide an effective procedure to construct the NPO problem and reduce the computational complexity of the resulting SDP relaxations. The thesis is concluded with a discussion of these results and an outline of future projects of interest.

2 Mathematical Preliminaries

We will introduce the necessary mathematical concepts in the hope of making this thesis self-contained. For a more detailed description, see the corresponding references.

2.1 Probability theory

Let us first introduce some basic concepts of probability theory relevant to this thesis following [37, 5, 62]. We omit any proofs in this section, as they are readily available in the literature on probability theory.

Firstly, let us introduce the notion of a *probabilistic model* or *probability space*. The non-empty set Ω consisting of all possible outcomes to a given experiment is called the *sample space*. Elements in Ω are mutually exclusive and called *elementary events*. The *event space* is captured by the σ -algebra $\mathcal{F}$. The elements $E \in \mathcal{F}$ are called *events*. That is, $\mathcal{F}$ is a set of subsets of Ω , such that

$$\begin{aligned} \Omega &\in \mathcal{F} \\ \text{if } E \in \mathcal{F} &\Rightarrow (\Omega \setminus E) =: E^c \in \mathcal{F} \\ \text{if } E_i \in \mathcal{F} \text{ with } i \in \mathbb{N}_+ \dots &\Rightarrow \bigcup_i E_i \in \mathcal{F} \text{ and } \bigcap_i E_i \in \mathcal{F}. \end{aligned} \quad (2.1)$$

$(\Omega, \mathcal{F})$ constitutes a *measurable space*. The *probability measure* encodes degrees of beliefs about events. A function $P : \mathcal{F} \rightarrow [0, 1]$ is a probability measure, if it fulfills the *Kolmogorov axioms*

$$\begin{aligned} \forall E \in \mathcal{F} \quad \exists P(E) &\geq 0, \\ P(\Omega) &= 1, \\ P\left(\bigcup_i A_i\right) &= \sum_i P(A_i), \quad \text{if } \bigcap_i A_i = \emptyset. \end{aligned} \quad (2.2)$$

The probability space is then given by $(\Omega, \mathcal{F}, P)$. It is complete, if

$$E' \subset E \in \mathcal{F}, P(E) = 0 \Rightarrow E' \in \mathcal{F}. \quad (2.3)$$

Remark. In this thesis, we assume all probabilistic spaces to be complete.

The causal models under consideration are limited to a finite number of discrete variables allowing for some simplifications. Let Ω be a countable set of elementary

events and $\mathcal{F} = 2^\Omega$ be its powerset. A *probability mass function*

$$\begin{aligned} p : \Omega &\rightarrow [0, 1], \\ \sum_{\omega \in \Omega} p(\omega) &= 1 \end{aligned} \tag{2.4}$$

assigns probabilities to elementary events. The probability measure on $\mathcal{F}$ is then $P(E) = p(E) \forall E \in \mathcal{F}$.

Remark. In discrete probability theory and countable Ω , it suffices to specify (Ω, P) or even just (Ω, p) .

We will limit this introduction to discrete probability theory. We are interested in *random experiments*, which can be repeated at will and where outcomes cannot be predicted. We need to shift from probability spaces to *induced probability spaces* to model those. Let us construct such a space. Let $(\Omega, \mathcal{F}, P)$ characterize a given probabilistic model. A random variable X is a measurable function $X : \Omega \rightarrow \mathbb{R}$. It preserves the measurability of sets, i.e. $\forall A \in \mathcal{R}$ with $\mathcal{R}$ a σ -algebra to $\mathbb{R}$ (*state space, Borel algebra*)

$$X^{-1}(A) = \{\omega | X(\omega) \in A\}, \tag{2.5}$$

Remark. In [\[62\]](#) random variables more generally map to a set of objects referred to as *values*. Random variables will be denoted by capital letters and corresponding values by lowercase letters.

For a given measurable space $(\mathbb{R}, \mathcal{R})$ (*Borel space*) and random variable X an *induced probability measure* $\mathbb{P}$ is given by

$$\mathbb{P}(E) = P(X^{-1}(E)). \tag{2.6}$$

Theorem 2.1 (Induced probability space). *$(\mathbb{R}, \mathcal{R}, \mathbb{P})$ is a probability space.*

Remark. From now on, we will not explicitly distinguish between the probability space and its induced space, therefore not differentiate between $\mathbb{P}$ and P . We will often abbreviate $P(X = x)$ by $P(x)$.

Lastly, let us address functions of random variables as these frequently appear in the analysis of causal models.

Theorem 2.2 (Functions of random variables). *Consider a probabilistic space and its induced space as described above. Let g be a real, measurable function of a random variable X , then $g(X)$ is also a random variable.*

Since we are generally considering not only one but many variables, we need to extend the above concepts to sets of random variables. Let $(\Omega, \mathcal{F}, P)$ characterise a given probabilistic model. A *random vector* $\mathbf{X}$ is a measurable function $\mathbf{X} : \Omega \rightarrow \mathbb{R}^n$. The random vector $\mathbf{X} = (X_1, \dots, X_n)^T$ “collects” n random variables X_i . Analogously, values are represented by vector $\mathbf{x}$. We then define the *joint distribution function* as

$$p_{\mathbf{X}}(\mathbf{x}) = P(\mathbf{X} = \mathbf{x}) = p_{X_1, \dots, X_n}(x_1, \dots, x_n) = P(X_1 = x_1 \wedge \dots \wedge X_n = x_n). \quad (2.7)$$

Remark. We will refer to a data set consisting of all outcomes in an experiment and their corresponding probability of occurrence as the joint probability distribution. We will generally drop the prefix “joint”. Also, any set of variables can be compounded to represent a new variable.

Joint probability functions allow us to define *marginal probabilities*. For jointly distributed random variables X, Y the *marginal probability function* of X is given by

$$p_X(x) = \sum_j p_{X,Y}(x, y_j). \quad (2.8)$$

A seminal concept in probability theory is the notion of independence between random variables. Let $\{X_i\}_{i=1}^n$ be a set of random variables and $\{A_i\}_{i=1}^n$ be a set of measurable sets. The random variables are independent, if and only if

$$P\left(\bigcap_{i=1}^n \{X_i \in A_i\}\right) = \prod_{i=1}^n P(X_i \in A_i). \quad (2.9)$$

Let us name two important theorems related to independence.

Theorem 2.3. *Consider the jointly distributed and independent random variables X, Y . Then,*

$$p_{X,Y}(x, y) = p_X(x) \cdot p_Y(y) = P(X = x) \cdot P(Y = y). \quad (2.10)$$

Theorem 2.4 (Independence functions of random variables). *Consider the set $\{X_i\}_{i=1}^n$ of independent random variables. Let $\{h_i\}_{i=1}^n$ be measurable functions, such that $h_i(X_i)$ are random variables. Then $\{h_i\}_{i=1}^n$ are also independent.*

Independence ties in closely to the notion of *conditional probability*. For jointly distributed random variables X, Y the conditional probability that $Y = y$ given $X = x$ is

$$p_{Y|X=x}(y) = P(Y = y|X = x) = \frac{p_{X,Y}(x, y)}{p_X(x)}. \quad (2.11)$$

A fundamental concept in causal models is based on the realisation that independence can also be conditioned on obtaining additional information. Consider the random variables X, Y, Z and their joint probability function P . Then X is conditionally independent of Y given Z , if

$$P(x|y, z) = P(x|z) \quad \text{whenever} \quad P(y, z) > 0. \quad (2.12)$$

Remark. We will adopt the terminology in [62] and refer to the above as “ Z screens off X from Y ” denoted $(X \perp\!\!\!\perp Y|Z)_P$. Unconditional independence will be denoted as $(X \perp\!\!\!\perp Y|\emptyset)_P$.

Conditional independences adhere to the *graphoid axioms* as introduced in [63]. Let us mention the *chain rule identity* in probability theory. For a random vector $\mathbf{X} = (X_1, \dots, X_n)^T$ and $\mathbf{x} = (x_1, \dots, x_n)^T$ the joint distribution function can be decomposed into n conditional probabilities

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | x_1, \dots, x_{i-1}). \quad (2.13)$$

For this thesis, we are not interested in specifying an interpretation of probabilities.

2.2 Graph theory

Graphs act as a mediator between probability theory and causality. They offer an economical display of probability distributions and represent causal relations in a causal hypothesis. We will introduce some general concepts and terminology in graph theory relevant to this work. This brief introduction is based on [23, 10].

A graph $\mathcal{G}$ is a tuple $(\mathcal{W}, \mathcal{F})$ of sets such that $\mathcal{F} \subseteq [\mathcal{W}]^2$. We refer to $\mathcal{W}$ as the set of *vertices* (*nodes*) and to $\mathcal{F}$ as the set of *edges* (*links*). Given a graph $\mathcal{H}$, the set of vertices is given by $V(\mathcal{H})$ and the set of edges by $E(\mathcal{H})$. The *order* of $\mathcal{H}$ is $|V(\mathcal{H})|$.

Remark. The condition $\mathcal{F} \subseteq [\mathcal{W}]^2$ captures the notion that an edge connects two vertices, i.e. the elements in $\mathcal{F}$ are sets of cardinality two. For $v_1, v_2 \in V(\mathcal{G})$ we write an edge as $v_1 v_2 := \{v_1, v_2\}$.

In pictorial representations of graphs (*graph drawings*), vertices are represented by circles with some index in the Euclidean plane. Edges connecting two vertices are drawn as non-intersecting lines. We restrict to these kinds of *plane* graphs and omit any further topological discussion of graphs.

Subsequently, $\mathcal{G}$ will represent an arbitrary graph. Let $v \in V(\mathcal{G})$ and $e \in E(\mathcal{G})$. Then v is called *incident* with e , if $v \in e$. Now let $v_1, v_2 \in V(\mathcal{G})$ be incident with $l \in E(\mathcal{G})$, then l *joins* the *ends* v_1, v_2 or v_1 is *adjacent* with v_2 , i.e. $l = v_1 v_2$. For any $v \in V(\mathcal{G})$ let $E(v)$ denote the set of edges to that vertex in $\mathcal{G}$. Two distinct edges $e_1, e_2 \in E(\mathcal{G})$ are adjacent, if $e_1 \cap e_2 \neq \emptyset$. Then, there exists a vertex $v \in V(\mathcal{G})$, such that v is incident with both e_1, e_2 .

Let us now introduce the notion of *independence* in graphs. Consider $v_1, v_2 \in V(\mathcal{G})$ and $e_1, e_2 \in E(\mathcal{G})$. Then v_1, v_2 are called *independent*, if there are not adjacent, i.e. if there does not exist an $e \in E(\mathcal{G})$ such that $v_1, v_2 \in e$. The edges e_1, e_2 are called *independent*, if they are not adjacent, i.e. if $e_1 \cap e_2 = \emptyset$. A subset $\mathcal{S} \subset V(\mathcal{G})$ is called *independent* or *stable*, if for any $s_1 \in \mathcal{S}$ and all $s_2 \in \mathcal{S} \setminus \{s_1\}$ the condition $\{s_1, s_2\} \notin E(\mathcal{G})$ holds.

Now let us focus on relations between graphs. For graphs $\mathcal{G}, \mathcal{G}'$ a map

$$\psi : V(\mathcal{G}) \rightarrow V(\mathcal{G}') \tag{2.14}$$

between the sets of vertices is a *graph homomorphism* from $\mathcal{G}$ to $\mathcal{G}'$, if the adjacencies of vertices are preserved under this mapping. Concretely, for any $e = v_1 v_2 \in E(\mathcal{G})$, ψ is

a graph homomorphism between $\mathcal{G}$ and $\mathcal{G}'$, if $\psi(v_1)\psi(v_2) \in E(\mathcal{G}')$. For a bijective graph homomorphism ψ let $\psi^{-1} : V(\mathcal{G}') \rightarrow V(\mathcal{G})$ denote its inverse map. Then $\forall v' \in V(\mathcal{G}')$ the set $\psi^{-1}(v')$ is stable. If ψ^{-1} is also a graph homomorphism, ψ is a *graph isomorphism* and $\mathcal{G}, \mathcal{G}'$ are *isomorphic*, i.e. $\mathcal{G} \simeq \mathcal{G}'$. We refer to a graph isomorphism $\psi : \mathcal{G} \rightarrow \mathcal{G}$ as a *graph automorphism* [10]. It is a permutation on $V(\mathcal{G})$ such that $\forall u, v \in V(\mathcal{G})$

$$\psi(u)\psi(v) \in E(\mathcal{G}) \Leftrightarrow uv \in E(\mathcal{G}). \quad (2.15)$$

Together with the composition of automorphisms, the set of automorphisms on $\mathcal{G}$ defines the *automorphism group* $\text{Aut}_{\mathcal{G}}$. For any $v \in V(\mathcal{G})$ we refer to the set

$$\text{Aut}_{\mathcal{G}} \cdot v = \{\psi(v) : \psi \in \text{Aut}_{\mathcal{G}}\} \quad (2.16)$$

as the *orbit* to v . Hence, under the action of $\text{Aut}_{\mathcal{G}}$ the set $V(\mathcal{G})$ is partitioned into orbits [25].

Remark. A homomorphism preserves the operations defined on the algebraic structure in its domain to its range. Here, this structure is given by the adjacency notion of vertices in a graph. This is an essential notion when encoding causal dependence in adjacency.

Consider the graphs $\mathcal{G}, \mathcal{G}'$. If $V(\mathcal{G}') \subseteq V(\mathcal{G})$ and $E(\mathcal{G}') \subseteq E(\mathcal{G})$, then $\mathcal{G}'$ is a *subgraph* of $\mathcal{G}$, i.e. $\mathcal{G}' \subseteq \mathcal{G}$. If additionally $\mathcal{G}' \neq \mathcal{G}$ holds, $\mathcal{G}'$ is a *proper subgraph* of $\mathcal{G}$. For $v \in V(\mathcal{G})$, $N_{\mathcal{G}} \in V(\mathcal{G})$ are the *neighbours* of v . Given a subset $\mathcal{S} \subseteq V(\mathcal{G})$, $N_{\mathcal{G}}(\mathcal{S}) \subseteq V(\mathcal{G}) \setminus \mathcal{S}$ are the *neighbours of $\mathcal{S}$* . For a subset $\{v_i\}_{i=0}^k \subseteq V(\mathcal{G})$ with $k \in \mathbb{N}$ a subgraph $\mathcal{P}$ is a *path*, if

$$V(\mathcal{P}) = \{v_i\}_{i=0}^k, \quad E(\mathcal{P}) = \{v_i v_{i+1}\}_{i=0}^{k-1}. \quad (2.17)$$

The path $\mathcal{P}$ of length k *links* the *ends* v_0 and v_k by traversing the *inner* $\{v_i\}_{i=1}^{k-1}$. We denote such a path as $\mathcal{P}^k = v_0 v_1 \dots v_k$. For $0 \leq i \leq j \leq k$ *subpaths* are denoted as $v_i \mathcal{P}^k v_j = v_i v_{i+1} \dots v_j$. We want to exclude a specific type of path. Let $\mathcal{P}^{k-1} = v_1 \dots v_{k-1}$ with $k \geq 3$ be a path in $\mathcal{G}$. A *cycle* $\mathcal{C}^k = \mathcal{P} \cup \{v_{k-1} v_0\}$ of *length* k is a subgraph of $\mathcal{G}$. A graph without any cycles is an *acyclic* graph. Paths also allow us to define the notion of connectivity in graphs. Let $\mathcal{G}$ be non-empty and $v_1, v_2 \in V(\mathcal{G})$ be any two vertices. $\mathcal{G}$ is a *connected* graph, if there exists a path $\mathcal{P}^k = u_0 \dots u_k$ in $\mathcal{G}$ such that $v_1 = u_0, v_2 = u_k$, i.e. $\mathcal{P}^k$ links v_1, v_2 . We refer to a maximal connected subgraph of $\mathcal{G}$ as a *component* of $\mathcal{G}$.

Finally, let us add a sense of direction to edges in graphs. Equip a graph $\mathcal{G}$ with

two mappings

$$\text{init} : E(\mathcal{G}) \rightarrow V(\mathcal{G}), \quad \text{ter} : E(\mathcal{G}) \rightarrow V(\mathcal{G}), \quad (2.18)$$

such that $\forall e \in E(\mathcal{G})$ is *directed* from $\text{init}(e)$ to $\text{ter}(e)$. $\mathcal{G}$ is thus lifted to a *directed graph*. Let $v_1, v_2 \in V(\mathcal{G})$ with *multiple edges* e_1, e_2 between them. If $\exists e \in E(\mathcal{G})$ such that $\text{init}(e) = \text{ter}(e)$, then e is a *loop*.

Remark. We will denote a directed edge from a vertex v_i to a vertex v_j by an arrow, $v_i \rightarrow v_j$. Directed edges are also referred to as *arcs*.

This thesis will not consider any graphs containing loops or multiple edges. We will refer to *directed acyclic graphs (DAG)* as DAGs. The concepts introduced for undirected graphs readily extend to DAGs. *Directed path* are paths as defined above, which additionally adhere to the direction of edges, i.e. they are restricted to traverse through a graph only in the direction of directed edges. Importantly, for graph homomorphisms between DAGs, we require the direction of edges to be preserved under the mapping.

It remains to introduce some terminology (*kinship* in graphs) commonly used when describing DAGs. For any $v \in V(\mathcal{G})$ the set

$$\text{Pa}_{\mathcal{G}}(v) = \{u \in V(\mathcal{G}) : uv \in E(\mathcal{G}), \text{ter}(uv) = v, \text{init}(uv) = u\} \quad (2.19)$$

are the *parents* of v . For any h, w in the set of parents, we call h a *spouse* of w (and vice-versa). The set

$$\text{Ch}_{\mathcal{G}}(v) = \{u \in V(\mathcal{G}) : vu \in E(\mathcal{G}), \text{ter}(vu) = u, \text{init}(vu) = v\} \quad (2.20)$$

are the *children* of v . For $\mathcal{V} \subseteq V(\mathcal{G})$

$$\text{Pa}_{\mathcal{G}}(\mathcal{V}) = \bigcup_{v \in \mathcal{V}} \text{Pa}_{\mathcal{G}}(v), \quad \text{Ch}_{\mathcal{G}}(\mathcal{V}) = \bigcup_{v \in \mathcal{V}} \text{Ch}_{\mathcal{G}}(v) \quad (2.21)$$

holds. Consider all directed paths $\mathcal{A} = \{\mathcal{P}_i\}_i$ terminating in v . Then, the *ancestors* of v are the elements of the set

$$\text{An}_{\mathcal{G}}(v) = \{u \in V(\mathcal{P}_i) : \mathcal{P}_i \in \mathcal{A}\}. \quad (2.22)$$

Now consider all directed paths $\mathcal{D} = \{\mathcal{P}_i\}_i$ originating from v . Likewise, the *descen-*

dants are the elements of the set

$$\text{Des}_{\mathcal{G}}(v) = \{u \in V(\mathcal{P}_i) : \mathcal{P}_i \in \mathcal{D}\}. \quad (2.23)$$

We will denote the *non-descendants* of v as $\text{Nd}_{\mathcal{G}}(v)$. A vertex without parents is a *root*, while a vertex without children is a *sink*.

Remark. For any vertex v , the set of parents is a subset of the ancestors, whereas the set of children is a subset of the descendants.

Lastly, a vital notion when considering DAGs in causal models is *d-separation* (directional separation) from [62]. Consider a DAG $\mathcal{G}$, a path $\mathcal{P}$ in $\mathcal{G}$ with $V(\mathcal{P}) = \{v_i\}_i$ and vertices $V(\mathcal{G}) \supseteq \mathcal{Z} = \{z_i\}_i$. $\mathcal{P}$ is d-separated by $\mathcal{Z}$, if and only if there exist any of the following subpaths in $\mathcal{P}$

1. $\mathcal{P}_{\text{chain}} = v_i z_m v_j$ such that $v_i \rightarrow z_m \rightarrow v_j$
2. $\mathcal{P}_{\text{fork}} = v_i z_m v_j$ such that $v_i \leftarrow z_m \rightarrow v_j$
3. $\mathcal{P}_{\text{inverted fork}} = v_i v_m v_j$ such that $v_i \rightarrow v_m \leftarrow v_j$ with $v_m \notin \mathcal{Z}$ and $\forall d \in \text{Des}_{\mathcal{G}}(v_m) \Rightarrow d \notin \mathcal{Z}$.

$\mathcal{C} \subset V(\mathcal{G})$ d-separates $\mathcal{A} \subset V(\mathcal{G})$ from $\mathcal{B} \subset V(\mathcal{G})$ denoted $(\mathcal{A} \perp\!\!\!\perp \mathcal{B} | \mathcal{Z})_{\mathcal{G}}$, if and only if $\forall a \in \mathcal{A}, \forall b \in \mathcal{B} \exists c \in \mathcal{C}$ such that the path $a\mathcal{P}b$ is d-separated by c .

2.3 Convex optimisation

The QCCP will be recast as a convex optimisation problem in the subsequent sections. We assume familiarity with the fundamental concepts of optimisation theory and will merely focus on introducing its convex variant, specifically semidefinite programmes, following [14] [79] [84].

2.3.1 The positive semidefinite matrix

At the very core of this thesis are *positive semidefinite (psd) matrices*. Several equivalent characteristics of a psd matrix come into use at different instances in this thesis.

Theorem 2.5 (Characterization of positive semidefinite matrices [84] p.99). *Let $\mathcal{S}^n = \{X \in \mathbb{R}^{n \times n} : X^T = X\}$ denote the set of symmetric $n \times n$ matrices. Then for $X \in \mathcal{S}^n$, the following are equivalent*

- $X \succeq 0$
- $y^T X y \geq 0$ for $y \in \mathbb{R}^n$
- $\lambda(X) \geq 0$ for eigenvalue function λ
- All principal minors of X are non-negative
- $\exists L \in \mathbb{R}^{n \times n} : X = LL^T$ (Cholesky Decomposition)

We will denote the set of psd matrices as $\mathcal{S}_+^n \subset \mathcal{S}^n$. Additionally, the following lemmas will be of importance.

Lemma 2.6 ([84] p. 99). *Any principal submatrix of a psd matrix is also psd.*

Lemma 2.7 ([84] p. 99). *If $A, B \in \mathcal{S}_+^n$, then $\langle A, B \rangle \geq 0$ and $\langle A, B \rangle = 0$ if and only if $AB = 0$.*

The $\langle \cdot, \cdot \rangle$ denotes a Frobenius inner product defined on the space of these matrices.

Remark. Since we are working with real matrices, we do not need to worry about the order in $\langle \cdot, \cdot \rangle$.

We restrict to real and symmetric matrices rather than, more generally, to hermitian matrices being psd. From any hermitian psd matrix, we can construct a real and symmetric psd matrix [35]. Let X be a hermitian matrix, i.e. $\text{Re}(X)^T = \text{Re}(X)$ and $\text{Im}(X) = -\text{Im}(X)^T$. For $z = u + iv$ with u, v real vectors

$$\begin{aligned} z^* X z &= (u - iv)^T (\text{Re}(X) + i\text{Im}(X))(u + iv) \\ &= \begin{bmatrix} u^T & v^T \end{bmatrix} \begin{bmatrix} \text{Re}(X) & -\text{Im}(X) \\ \text{Im}(X) & \text{Re}(X) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix}. \end{aligned} \tag{2.24}$$

Hence,

$$X \succeq 0 \Leftrightarrow Y := \begin{bmatrix} \text{Re}(X) & -\text{Im}(X) \\ \text{Im}(X) & \text{Re}(X) \end{bmatrix} \succeq 0. \quad (2.25)$$

2.3.2 Convexity

For an *affine* set $\mathcal{A} \subseteq \mathbb{R}^n$ choose any $\{x_i\}_{i=1}^k \subseteq \mathcal{A}$ and $\{\alpha_i\}_{i=1}^k \subseteq \mathbb{R}$ with $\sum_{i=1}^k \alpha_i = 1$ and $k \in \mathbb{N}_+$, then the *affine combination* $\sum_{i=1}^k \alpha_i x_i$ lies in $\mathcal{A}$. The *affine hull* $\mathbf{aff} \mathcal{B}$ of a set $\mathcal{B} \subseteq \mathbb{R}^n$ is the smallest affine set containing $\mathcal{B}$. Let $B(x, r) = \{y : \|y - x\| \leq r\}$ with any norm $\|\cdot\|$ defined on that space. The *relative interior* of $\mathcal{B}$ is

$$\mathbf{relint} \mathcal{B} = \{x \in \mathcal{B} : B(x, r) \cap \mathbf{aff} \mathcal{B} \subseteq \mathcal{B} \text{ with } r > 0\}. \quad (2.26)$$

For a *convex* set $\mathcal{C} \subseteq \mathbb{R}^n$ again choose any $\{x_i\}_{i=1}^k \subseteq \mathcal{C}$ and $\{\alpha_i\}_{i=1}^k \subseteq \mathbb{R}$ with $\sum_{i=1}^k \alpha_i = 1$, $\alpha_i \geq 0$, $i = 1, \dots, k$ then the *convex combination* $\sum_{i=1}^k \alpha_i x_i$ lies in $\mathcal{C}$. Analogously to the affine hull, we can define the *convex hull* to some set $\mathcal{B} \subseteq \mathbb{R}^n$ to be the smallest convex set to contain $\mathcal{B}$. The intersection of two convex sets is again convex. The image of a convex set under an affine function is again affine.

An important notion when dealing with convex optimisation problems are *hyperplanes* and *halfspaces* corresponding to constraints. Hyperplanes are governed by linear equalities, whereas halfspaces result from linear inequalities. Let $\mathbf{x}, \boldsymbol{\alpha}$ be vectors in the vector space $\mathbb{R}^n$ with $\boldsymbol{\alpha} \neq \mathbf{0}$ and $b \in \mathbb{R}$. A hyperplane is then the affine set of the solutions $\mathbf{x}$ to the expression $\boldsymbol{\alpha}^T \mathbf{x} = b$. Any hyperplane separates the underlying space $\mathbb{R}^n$ into two closed halfspaces. These are convex, but not affine sets of solutions $\mathbf{x}$ to the expressions $\boldsymbol{\alpha}^T \mathbf{x} \leq b$ and $\boldsymbol{\alpha}^T \mathbf{x} \geq b$. Combining finitely many such equalities and inequalities, i.e. intersecting their corresponding hyperplanes and halfspaces, gives rise to a *polytope*

$$\mathcal{P} = \{\mathbf{x} : \boldsymbol{\alpha}_j^T \mathbf{x} \leq c_j, j = 1, \dots, m, \boldsymbol{\beta}_j^T \mathbf{x} = b_j, j = 1, \dots, k\} \quad (2.27)$$

for some non-zero $\boldsymbol{\alpha}_j, \boldsymbol{\beta}_j \in \mathbb{R}^n$ and $b_j, c_j \in \mathbb{R}$.

A *cone* is a set $\mathcal{A} \subseteq \mathbb{R}^n$, such that $\forall x \in \mathcal{A}$ and $\alpha \in \mathbb{R}_{\geq 0}$ implies $\alpha x \in \mathcal{A}$. A set $\mathcal{C}$ is a *convex cone*, if and only if for any $\{x_i\}_{i=1}^k \subseteq \mathcal{C}$, $\{\alpha_i\}_{i=1}^k \subseteq \mathbb{R}_{\geq 0}$ with $\sum_{i=1}^k \alpha_i \geq 0$ and $k \in \mathbb{N}_+$ the *conic combination* $\sum_{i=1}^k \alpha_i x_i$ lies in $\mathcal{C}$. Again, we can define the *conic hull* of a set $\mathcal{B}$ to be the set of all conic combinations of elements from $\mathcal{B}$ and hence, the smallest convex cone containing $\mathcal{B}$. The set of psd matrices $\mathcal{S}_+^n$ also defines a convex

cone, the *positive semidefinite cone*. Even more so, it is a *proper cone* allowing for the definition of a *generalised inequality*. We have introduced $\succeq$ without much elaboration. Consider a convex cone $\mathcal{K} \in \mathbb{R}^n$. It is a proper cone, if it is closed, has a non-empty interior (*solid*) and that for any $x \in \mathcal{K}$ with $-x \in \mathcal{K}$ implies $x = 0$ (*pointed*). We then say that $x \succeq_{\mathcal{K}} y$ if and only if $x - y \in \mathcal{K}$. From now on, we will abbreviate a psd matrix $X \succeq_{\mathcal{S}_+^n} 0 \Leftrightarrow X \in \mathcal{S}_+^n$ as $X \succeq 0$. Thus, we obtain a partial order on the set of psd matrices, the *Loewner partial order*. For a cone $\mathcal{K}$, its *dual cone* is given by the closed and convex cone

$$\mathcal{K}^* = \{y : x^T y \geq 0\} \quad (2.28)$$

The $\mathcal{S}_+^n$ is *self-dual*, i.e. $(\mathcal{S}_+^n)^* = \mathcal{S}_+^n$. For a proof, see [14, p.52]. The dual of a proper cone is also proper and defines a *dual generalised inequality*. Since the $\mathcal{S}_+^n$ is self-dual, we do not need to differentiate between $\succeq_{(\mathcal{S}_+^n)^*}$ and $\succeq_{\mathcal{S}_+^n}$.

We are interested in *convex functions* over convex sets. A convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with **dom** f a convex set satisfies the *Jensen inequality*

$$f(\alpha \mathbf{x} + \beta \mathbf{y}) \leq \alpha f(\mathbf{x}) + \beta f(\mathbf{y}) \quad (2.29)$$

for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and all $\alpha, \beta \in \mathbb{R}$ with $\alpha + \beta = 1$, $\alpha, \beta \geq 0$. According to Eq. (2.29), we define a $\mathcal{K}$ -*convex* function, with $\mathcal{K}$ a proper cone, by replacing the inequality with the generalised inequality $\succeq_{\mathcal{K}}$. A function $g : \mathbb{R}^n \rightarrow \mathbb{R}$ is *concave*, if and only if $-g$ is convex. For *affine* functions, these inequalities become equalities, and they are thus both convex and concave.

2.3.3 Semidefinite programmes

We are interested in a specific variant of convex optimisation problems with generalised inequality constraints, namely semidefinite programmes (SDP). In an SDP, we optimise a linear objective function over the positive semidefinite cone intersected by an affine space generated by affine constraint functions. Hence, we minimise a linear objective function over a convex set. Consider an SDP in *standard form*

$$\begin{aligned} \min_{X \in \mathcal{D}} \quad & \langle C, X \rangle \\ \text{s.t.} \quad & \langle A_i, X \rangle = b_i, \quad i = 1, \dots, m \\ & X \succeq 0 \end{aligned} \quad (2.30)$$

where $C, A_i \in \mathcal{S}^n$. As required for a convex optimisation problem with generalised inequalities, the objective function is $\mathcal{S}_+^n$ -convex, the equality constraints are affine, and the general inequality constraint is $\mathcal{S}_+^n$ -convex (in particular affine). Let $\mathcal{D}$ be the domain of the above problem, i.e. the intersection of the domains of the objective function and the domains of the constraint function. By the above $\mathcal{D}$ is convex. The *feasible set* is the set of all X in $\mathcal{D}$ which satisfy the constraints. It is also convex since intersecting $\mathcal{D}$ with convex hyperplanes (or even convex halfspaces) and the convex $\mathcal{S}_+^n$ cone results in a convex set. The *optimal value* of the SDP is given by

$$p^* = \inf_{X \in \mathcal{D}} \{ \langle C, X \rangle : \langle A_i, X \rangle = b_i, i = 1, \dots, m, X \succeq 0 \} . \quad (2.31)$$

If the above SDP is *infeasible*, then $p^* = \inf \emptyset = \infty$. An optimal point X^* is an element of the feasible set such that $\langle C, X^* \rangle = p^*$ and the *optimal set* is the collection of all such points. The linear objective function necessitates that the optimal point lies on the boundary of the feasible set. Very importantly, in convex optimisation problems, any locally optimal point X^* is globally optimal (see [14] p. 138 for a more detailed discussion). That is, X^* is optimal over the entire feasible set.

Equivalently, in the SDP above, we can maximise the concave objective function $-\langle C, X \rangle$. We consider two convex optimisation problems $\mathbf{P}, \mathbf{P}'$ *equivalent*, if from a solution p^* to $\mathbf{P}$ a solution p'^* to $\mathbf{P}'$ is obtainable and vice-versa.

A notion frequently encountered when dealing with numerical solvers of convex optimisation problems is *slack variables*. These allow us to replace inequality constraints with an equality constraint and a (generalised) non-negative constraint. The constraint $f(x) \preceq_{\mathcal{K}} 0$ can be replaced by $f(x) + s = 0$ and $s \succeq_{\mathcal{K}} 0$, where s is the slack variable. Since the introduced equality is affine and the inequality is $\mathcal{K}$ -convex, the convexity of the problem is conserved, and the problems are equivalent.

More natural to the SDP formulations arising in this thesis is the *inequality form* SDP [79]

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{D}} \quad & \langle \mathbf{c}, \mathbf{x} \rangle \\ \text{s.t.} \quad & F(\mathbf{x}) = \sum_{i=1}^m x_i F_i + G \succeq 0 \end{aligned} \quad (2.32)$$

where $\mathbf{x} = (x_1, \dots, x_m)$, $\mathbf{c} \in \mathbb{R}^m$ and $G, F_i \in \mathcal{S}^n$. The constraint is referred to as a *linear matrix inequality (LMI)*. Again, we optimise a linear objective function, but now over the intersection of the image of the constraint function F with the $\mathcal{S}_+^n$ cone.

Remark. In the literature, the LMI is more commonly portrayed as $\sum_{i=1}^n x_i A_i + B \preceq_{\mathcal{S}_+^n} 0$. The two formulations are equivalent. The constraint function is affine, hence assigning $F_i = -A_i \forall i$ and $G = -B$ flips the inequality sign without changing the convexity of the problem.

We refer to a *feasibility problem* as one where the objective function is trivial, and a feasible solution certifies the consistency among the imposed constraints, i.e.

$$\begin{aligned} & \text{find} && \mathbf{x} \\ & \text{s.t.} && F(\mathbf{x}) = \sum_{i=1}^m x_i F_i + G \succeq 0, \end{aligned} \tag{2.33}$$

where $\mathbf{x} \in \mathbb{R}^m$ and $G, F_i \in \mathcal{S}^n$.

2.3.4 Duality theory

We will now move on to a fundamental concept in semidefinite programming. Informally, in *Lagrangian duality theory* we alter the objective function by adding to it each constraint weighted by a *Lagrangian multiplier* or *dual variable*. We then construct an optimisation problem, the *Lagrange dual problem*, in terms of these dual variables from which we can deduce information about the feasibility, optimal set and optimal value of the original problem.

Let us start by constructing the Lagrange dual problem to the SDP in inequality form in Eq. (2.32) as it is of greater importance to this thesis. Let us denote the Lagrangian multiplier by Z and require that Z be an element of the dual cone corresponding to the generalised inequality. Since $\mathcal{S}_+^n$ is self-dual, we require $Z \in \mathcal{S}_+^n$. We refer to the resulting affine function as the *Lagrangian*

$$\begin{aligned} \mathcal{L}(\mathbf{x}, Z) &= \langle \mathbf{c}, \mathbf{x} \rangle - \langle F(\mathbf{x}), Z \rangle \\ &= \langle \mathbf{c}, \mathbf{x} \rangle - \langle G, Z \rangle - \sum_{i=1}^m x_i \langle F_i, Z \rangle \\ &= -\langle G, Z \rangle + \sum_{i=1}^m x_i (c_i - \langle F_i, Z \rangle) . \end{aligned} \tag{2.34}$$

Remark. When dealing with an LMI with $\preceq 0$, the weighted constraint needs to be added rather than subtracted from the objective function.

We define the *Lagrangian dual function* as the concave function

$$\begin{aligned} g(Z) &= \inf_{\mathbf{x} \in \mathcal{D}} \mathcal{L}(\mathbf{x}, Z) \\ &= \begin{cases} -\langle G, Z \rangle & \text{for } c_i - \langle F_i, Z \rangle = 0, i = 1, \dots, m \\ -\infty & \text{otherwise.} \end{cases} \end{aligned} \quad (2.35)$$

Each coefficient to x_i in the sum must be equal to zero to obtain a non-trivial infimum. Now let p^* be an optimal value to the **(PSDP)**. Then $g(Z) \leq p^*$, since any $\tilde{\mathbf{x}}$ in the feasible set to the primal SDP **(PSDP)** adheres to the generalised inequality constraint. Then with Lemma 2.7 it follows

$$-\langle F(\tilde{\mathbf{x}}), Z \rangle \leq 0 \quad (2.36)$$

so that $\mathcal{L}(\tilde{\mathbf{x}}, Z) \leq \langle \mathbf{c}, \tilde{\mathbf{x}} \rangle$. Hence,

$$g(Z) \leq \inf_{\tilde{\mathbf{x}} \in \mathcal{D}} \mathcal{L}(\tilde{\mathbf{x}}, Z) \leq \langle \mathbf{c}, \tilde{\mathbf{x}} \rangle. \quad (2.37)$$

The resulting Lagrange dual problem is

$$\max_{Z \in \mathcal{S}_+^n} g(Z), \quad (2.38)$$

i.e. the task of finding the maximal lower bound to p^* . Transforming the domain restriction on Z in $g(Z)$ into explicit constraints results in an equivalent problem to Eq. (2.38) the dual SDP **(DSDP)**

$$\begin{aligned} \max_{Z \in \mathcal{S}^n} \quad & -\langle G, Z \rangle \\ \text{s.t.} \quad & \langle F_i, Z \rangle = c_i, \quad i = 1, \dots, m \\ & Z \succeq 0, \end{aligned} \quad (2.39)$$

which is an SDP in standard form. It is *dual feasible*, if $\exists \tilde{Z} \in \mathcal{S}^n$ adhering to the constraints. Obtaining such a dual feasible $\tilde{Z}$ certifies $g(\tilde{Z}) \leq p^*$. Any dual feasible point $\tilde{Z}$ provides a lower bound for the optimal value of the primal problem. Accordingly, any feasible primal point $\tilde{\mathbf{x}}$ is an upper bound for the dual optimal value. With Lemma 2.7 the inequality for the *duality gap*

$$\begin{aligned} \langle \mathbf{c}, \tilde{\mathbf{x}} \rangle - (-\langle G, \tilde{Z} \rangle) &= \sum_{i=1}^m x_i \langle F_i, \tilde{Z} \rangle + \langle G, \tilde{Z} \rangle \\ &= \langle F(\tilde{\mathbf{x}}), \tilde{Z} \rangle \geq 0 \end{aligned} \quad (2.40)$$

always holds. $Z^\star$ is a *dual optimal point* with *dual optimal value* $-\langle G, Z^\star \rangle$. Similarly, to the SDP in standard form in [Eq. \(2.30\)](#), we obtain the **(DSDP)** in inequality form

$$\begin{aligned} \max_{\mathbf{y} \in \mathbb{R}^m} \quad & \langle \mathbf{b}, \mathbf{y} \rangle \\ \text{s.t.} \quad & C - \sum_{i=1}^m y_i A_i \succeq 0. \end{aligned} \tag{2.41}$$

with dual variable $\mathbf{y} \in \mathbb{R}^m$, dual optimal $\mathbf{y}^\star$ and dual optimal value $\langle \mathbf{b}, \mathbf{y}^\star \rangle$. For a more detailed derivation of [Eq. \(2.41\)](#) see [Appendix A](#).

Let us denote the dual optimal value by $d^\star$ and the *optimal duality gap* by $p^\star - d^\star$. As by the above, *weak duality* $d^\star \leq p^\star$ always holds. *Strong duality* $d^\star = p^\star$ does not hold in general for SDPs. The generalised *Slater condition* provides a criterion for strong duality to hold. Consider the SDP in inequality form with domain $\mathcal{D}$, if $\exists \mathbf{x} \in \text{relint } \mathcal{D}$ satisfying $F(\mathbf{x}) \succ 0$, then $d^\star = p^\star$.

Remark. Constructing the dual problems should not be confused with transformations between standard and inequality forms. The two are vastly different concepts. Especially when strong duality does not hold, the dual problem will not be equivalent, whereas transforming between formulations will always do so.

2.3.5 Eigenvalue optimisation problems

Recall that in the definition of a psd matrix, all eigenvalues were non-negative. For a given matrix Z , let us denote the smallest eigenvalue by $\lambda_{\min}(Z)$. Then,

$$Z \in \mathcal{S}_+^n \Leftrightarrow \lambda_{\min}(Z) \geq 0. \tag{2.42}$$

We are interested in exploiting this fact when facing a feasibility problem $\mathbf{P}_f$ as in [Eq. \(2.33\)](#). Equivalently, we can consider the *eigenvalue problem (EVP)*

$$\begin{aligned} \max_{\mathbf{x} \in \mathbb{R}^m, \lambda \in \mathbb{R}} \quad & \lambda \\ \text{s.t.} \quad & F(\mathbf{x}) - \lambda I \succeq 0 \end{aligned} \tag{2.43}$$

where I represents the identity in the space of $n \times n$ matrices. Assume that the problem is feasible with feasible point $(\tilde{\mathbf{x}}, \tilde{\lambda})$. An optimal value $p^\star = \lambda^\star$ with optimal point $(\mathbf{x}^\star, \lambda^\star)$ is attained. Then any $(\tilde{\mathbf{x}}, \tilde{\lambda})$ adheres to the inequality

$$\lambda_{\min}(F(\tilde{\mathbf{x}}) - \tilde{\lambda}I) = \lambda_{\min}(F(\tilde{\mathbf{x}})) - \tilde{\lambda} \geq 0. \tag{2.44}$$

Equality holds for any optimal point $(\mathbf{x}^*, \lambda^*)$. Then,

$$\begin{aligned}\lambda^* \geq 0 &\Rightarrow (F(\mathbf{x}^*)) \geq 0 \quad \mathbf{P}_f \text{ feasible} \\ \lambda^* < 0 &\Rightarrow (F(\mathbf{x}^*)) < 0 \quad \mathbf{P}_f \text{ infeasible.}\end{aligned}\tag{2.45}$$

2.3.6 Solving semidefinite programmes

As convex minimisation problems, SDPs are solvable in polynomial time [84]. There exist a variety of algorithms such as the *interior-point methods* [14] and *bundle methods* [84] to do so. In this thesis, we will not concern ourselves with these methods and will refer to the literature.

Remark. The literature on solving SDPs is vast and outside the scope of this thesis. Unfortunately, this limits our ability to formulate problems specifically to a solver's needs or even score different solvers on their eligibility for the problems we want to solve.

2.4 Abstract algebra

We will briefly introduce some relevant notions from abstract algebra following [25, 44, 11, 76]. We omit most proofs as they are widely available in the corresponding literature.

The problems of interest arising in the context of quantum causal analysis are formulated as *non-commutative polynomial optimisation (NPO) problems*. This section will introduce the algebraic structures to formulate NPO problems and the connection to operators acting on Hilbert spaces.

2.4.1 C^* -algebras and the GNS construction

Let us start with the latter by introducing the fundamental theory behind C^* -algebras and their realisation as operators acting on a Hilbert space via the *Gelfand–Naimark–Segal (GNS) construction*.

Consider a vector space $(\mathcal{U}, +, \cdot_{\mathbb{C}})$. Equip this space with a norm $\|\cdot\|$ to obtain a complete normed vector space, i.e. a complex *Banach space*. Define on $\mathcal{U}$ an associative bilinear operation $\circ : \mathcal{U} \rightarrow \mathcal{U}$ lifting it to a complex algebra, such that $\forall a, b \in \mathcal{U}$

$$\|a \circ b\| \leq \|a\| \cdot \|b\|. \quad (2.46)$$

We thus defined a *Banach algebra* and denote it by $\mathcal{U}$.

Remark. We will not explicitly denote $\circ$.

Let us denote the unit element of a $\mathcal{U}$ by $1_{\mathcal{U}}$. We are interested in adding even more structure. A *Banach $*$ -algebra* is $\mathcal{U}$ equipped with $*$: $\mathcal{U} \rightarrow \mathcal{U}$, such that $\mathcal{U}$ is a $*$ -algebra. Concretely, the mapping $*$ is $\mathbb{C}$ -antilinear and an *anti-involution* operation, i.e. an involutive antiautomorphism, satisfying $\forall a, b \in \mathcal{U}$ and $\lambda \in \mathbb{C}$

$$\begin{aligned} (a^*)^* &= a \\ (a + b)^* &= a^* + b^* \\ (\lambda a)^* &= \bar{\lambda} a^* \\ (ab)^* &= b^* a^*. \end{aligned} \quad (2.47)$$

Example 2.1. The space of bounded operators $\mathcal{B}(\mathcal{H})$ on a Hilbert space $\mathcal{H}$ constitutes such a $*$ -algebra. In this case, the involution is defined by the operator adjoint, and the norm is given by the operator norm.

If the norm in [Eq. \(2.46\)](#) additionally satisfies

$$\|a^*a\| = \|a\|^2 \quad (2.48)$$

we refer to it as a C^* -norm on $\mathcal{U}$. If $\mathcal{U}$ is complete with respect to this norm, it is a C^* -algebra. Furthermore, if $\mathcal{U}$ contains $1_{\mathcal{U}}$ it is called *unital*. Since we can *unitise* a C^* -algebra without a unit element, we assume unital C^* -algebra in this thesis.

Example 2.2. $\mathcal{B}(\mathcal{H})$ to some Hilbert space $\mathcal{H}$ constitutes such a C^* -algebra.

Closed subspaces of a C^* -algebra are called *operator spaces*. If such a subspace is self-adjoint and contains the identity, it is called an *operator system*. We refer to $a \in \mathcal{U}$ as *positive*, if $\exists b \in \mathcal{U}$ such that $a = b^*b$. The set of all such positive elements in $\mathcal{U}$ forms a convex cone.

A *state* on a C^* -algebra $\mathcal{U}$ is a normalised positive map $\varrho : \mathcal{U} \rightarrow \mathbb{C}$, i.e. $\forall b \in \mathcal{U}$

$$\begin{aligned} \text{(Positivity)} \quad & \varrho(b^*b) \geq 0 \\ \text{(Normalised)} \quad & \varrho(1_{\mathcal{U}}) = 1. \end{aligned} \quad (2.49)$$

The *state space* $S(\mathcal{U})$ to $\mathcal{U}$ is a compact convex space under the *weak-* topology* containing all states on $\mathcal{U}$. *Pure states* are the extreme points of $S(\mathcal{U})$ while *mixed states* lie in the interior of the corresponding convex set. In other words, $S(\mathcal{U})$ is the closed convex hull of the set of pure states [\[11\]](#), p. II.6.2.9]. If we are given an operator system $\mathcal{X} \in \mathcal{U}$ and a state ϕ on $\mathcal{X}$, then by the *Hahn-Banach theorem* [\[53\]](#) we obtain a state on all of $\mathcal{U}$ by the extension of ϕ .

Remark. We are working with normed spaces and thus can define a metric and ultimately a topology. The choice of a norm on a space is generally not unique, and thus the resulting topology is neither. For a given topological (vector) space $\mathcal{M}$ we can define a topology on the corresponding topological dual space $\mathcal{M}^*$ via the topology on $\mathcal{M}$. We refer to the “weakest” or “coarsest” topology induced on $\mathcal{M}^*$ from the norm topology on $\mathcal{M}$ by the *weak-* topology* [\[74\]](#). Informally, the coarseness of a topology refers to the number of open sets in the topological space. Alternatively, the weak-* topology is the topology such that for every bounded linear functional $\omega \in \mathcal{M}^*$, ω is continuous on all of $\mathcal{M}$. This weak-* topology allows for the application of the *Banach-Alaoglu theorem* [\[16\]](#) to prove *weak-* compactness* of the *polar*, i.e. the norm unit ball in $\mathcal{M}^*$. Consider a C^* -algebra $\mathcal{A}$ and its dual space $\mathcal{A}^*$. The Banach-Alaoglu theorem certifies the weak-* compactness of the polar in $\mathcal{A}^*$. Since we require states to

be normalised positive functionals and $\mathcal{A}$ to be unital, we can identify the hyperplane

$$\{\varrho \in \mathcal{A}^* : \varrho(1_{\mathcal{A}}) = 1\}, \quad (2.50)$$

which is closed in the weak-* topology. State space $S(\mathcal{A}) \subseteq \mathcal{A}^*$ is then the intersection of the weak-* compact polar to $C^*(\Sigma|\mathcal{R})^*$ and the above hyperplane. Thus, it is weak-* closed. Since it is a subset of a weak-* compact space, it is itself weak-* compact.

On the topic of metric spaces, we define the *closure* $\bar{\mathcal{N}}$ of a subset $\mathcal{N}$ to metric space $\mathcal{M}$ as

$$\bar{\mathcal{N}} = \mathcal{N} \cup \{\text{Limit points to } x \in \mathcal{N}\}. \quad (2.51)$$

Let us focus on the relation between C^* -algebras and algebras of operators on Hilbert spaces. A **-representation* π of a C^* -algebra $\mathcal{U}$ on a Hilbert space $\mathcal{H}$ is a **-homomorphism* $\pi : \mathcal{U} \rightarrow \mathcal{B}(\mathcal{H})$, such that $\forall a, b \in \mathcal{U}$ and $\forall \alpha, \beta \in \mathbb{C}$

$$\begin{aligned} \pi(ab) &= \pi(a)\pi(b) \\ \pi(\alpha a + \beta b) &= \alpha\pi(a) + \beta\pi(b) \\ \pi(a^*) &= \pi(a)^*. \end{aligned} \quad (2.52)$$

and π additionally preserves the unit, i.e. it is *non-degenerate*. Representations on C^* -algebras are continuous and bounded. The above representation π is *cyclic*, if $\exists \Omega \in \mathcal{H}$, such that the norm closure of

$$\pi(\mathcal{U})\Omega = \{\pi(a)\Omega : a \in \mathcal{U}\} \quad (2.53)$$

yields $\mathcal{H}$, i.e. it is *dense* in $\mathcal{H}$. We consider two representations π_1, π_2 of $\mathcal{U}$ on $\mathcal{H}_1, \mathcal{H}_2$ *unitarily equivalent*, if there exists a unitary isomorphism

$$U : \mathcal{H}_1 \rightarrow \mathcal{H}_2 \quad (2.54)$$

such that $U\pi_1(a) = \pi_2(a)U \ \forall a \in \mathcal{U}$.

To every unit vector $\zeta \in \mathcal{H}$ a *vector state* $\phi_{\zeta} \in S(\mathcal{U})$ to π is given by

$$\begin{aligned} \phi_{\zeta} : \mathcal{U} &\rightarrow \mathbb{C} \\ \phi_{\zeta} : a &\mapsto \langle \zeta, \pi(a)\zeta \rangle_{\mathcal{H}} \quad . \end{aligned} \quad (2.55)$$

Conversely, the *Gelfand–Naimark–Segal (GNS) construction* guides us in constructing canonically such a cyclic representation on a Hilbert space from a given state in

the state space to a C^* -algebra. Let us start with the construction of a Hilbert space $\mathcal{H}_\varrho$ to a state $\varrho \in S(\mathcal{U})$. This state defines a semidefinite sesquilinear form on $\mathcal{U}$ via

$$\langle a, b \rangle_0 := \varrho(a^*b) \quad (2.56)$$

$\forall a, b \in \mathcal{U}$. This constitutes a pre-inner product since the null space

$$N_\varrho = \{a \in \mathcal{U} : \varrho(a^*a) = 0\} \quad (2.57)$$

is non-trivial. N_ϱ is a closed left-ideal of $\mathcal{U}$. Let us define a projection operation P from $\mathcal{U}$ to the quotient space $\mathcal{U}/N_\varrho$, such that $\forall a \in \mathcal{U}$

$$P(a) =: a_\varrho \in \mathcal{U}/N_\varrho \quad (2.58)$$

is the image of a in the quotient space. Then

$$\langle a_\varrho, b_\varrho \rangle_\varrho := \langle a, b \rangle_0 \quad (2.59)$$

is a well-defined inner-product on the quotient space rendering it a pre-Hilbert space. We thus obtain the *GNS Hilbert space* $\mathcal{H}_\varrho$ by completing (*Cauchy completion*) $\mathcal{U}/N_\varrho$ with respect to $\langle \cdot, \cdot \rangle_\varrho$.

Remark. Some authors refer to the entire process of constructing the quotient followed by the completion as the completion.

Let $\pi_\varrho(\mathcal{U})$ be the action of $\mathcal{U}$ on $\mathcal{U}/N_\varrho$ such that for each $a \in \mathcal{U}$ and $b_\varrho \in \mathcal{U}/N_\varrho$

$$\pi_\varrho(a)b_\varrho = P(ab) = (ab)_\varrho. \quad (2.60)$$

Hence, $\pi_\varrho(a)$ acts on elements in $\mathcal{U}/N_\varrho$ by left multiplication. Since $\pi_\varrho(a)$ is a bounded operator, it can be continuously extended to a unique bounded operator on the completion of $\mathcal{U}/N_\varrho$, i.e. $\mathcal{H}_\varrho$. It is a cyclic representation of $\mathcal{U}$ in $\mathcal{B}(\mathcal{H}_\varrho)$ corresponding to ϱ and is called the *GNS representation*. Representations obtained via the GNS construction from pure states are irreducible.

Finally, a *GNS cyclic vector* $\Omega_\varrho \in \mathcal{H}_\varrho$ for our unital C^* -algebra $\mathcal{U}$ is the image of $\mathbb{1}_\mathcal{U}$ in $\mathcal{H}_\varrho$ under the projection P . Thus, $\forall a, b \in \mathcal{U}$

$$\varrho(a^*b) = \langle \pi_\varrho(a)\Omega_\varrho, \pi_\varrho(b)\Omega_\varrho \rangle_\varrho \quad (2.61)$$

and $\{\pi_\varrho(a)\Omega_\varrho : a \in \mathcal{U}\}$ is norm dense in $\mathcal{H}_\varrho$.

Remark. The requirement for $\{\pi_\varrho(a)\Omega_\varrho : a \in \mathcal{U}\}$ to be dense in $\mathcal{H}_\varrho$ captures the notion,

that our cyclic GNS representation π_ϱ and vector Ω_ϱ allow us to construct all of $\mathcal{H}_\varrho$ by ranging through $\mathcal{U}$.

Remark. In physics, we often use the Dirac notation

$$\varrho(a) = \langle \Omega_\varrho | \pi_\varrho(a) | \Omega_\varrho \rangle \quad (2.62)$$

and abbreviate it with $\langle a \rangle_{\Omega_\varrho}$. Often, we decide to work with the density operator formalism as it encapsulates pure and mixed states. Analogously, we can assign to each algebraic state $\varrho \in S(\mathcal{U})$ a density operator $\omega_\varrho \in \mathcal{B}(\mathcal{H})$ via

$$\varrho(a) = \langle a \rangle_{\omega_\varrho} \quad (2.63)$$

$\forall a \in \mathcal{U}$. Assuming that a trace operation exists and given a representation σ mapping the density operator ω_ϱ to a density matrix $\sigma(\omega_\varrho)$ and the operator $A \in \mathcal{B}(\mathcal{H})$ to a matrix $\sigma(A)$ we evaluate

$$\langle a \rangle_{\omega_\varrho} = \text{Tr} [\sigma(\omega_\varrho) \sigma(a)] . \quad (2.64)$$

We do not explicitly differentiate between operators and their matrix representation. The GNS construction with mixed states yields not irreducible representations but direct sums of independent representations.

In summary, given a cyclic representation π of a C^* -algebra $\mathcal{U}$ on a Hilbert space $\mathcal{H}$ such that $\Omega \in \mathcal{H}$ is a cyclic unit vector to π and $\varrho(a) = \langle \Omega, \pi(a)\Omega \rangle \forall a \in \mathcal{U}$. Then, the GNS construction provides us with $(\mathcal{H}_\varrho, \pi_\varrho, \Omega_\varrho)$, such that π, π_ϱ are unitarily equivalent. The *Gelfand-Neimark theorem* [11, p. II.6.4.10] states that any C^* -algebra is isometrically isomorphic to a subspace of $\mathcal{B}(\mathcal{H})$ for some Hilbert space $\mathcal{H}$. This seminal result provides the argument to abstractly consider C^* -algebras instead of a realisation as $\mathcal{B}(\mathcal{H})$ for some Hilbert space $\mathcal{H}$.

2.4.2 The free $*$ -algebra

Now that we have introduced a connection between algebras and operators acting on Hilbert spaces let us introduce the algebra capturing non-commutative polynomials.

Let $\mathcal{X} = \{x_1, \dots, x_n\}$ be the set of n indeterminates. Each x_i will be referred to as a *letter*. Additionally, define an *involution* operation $*$ on letters, such that $(x^*)^* = x \forall x \in \mathcal{X}$. The set $\Sigma = \{x_1, \dots, x_n, x_1^*, \dots, x_n^*\}$ is an *alphabet*. Finite sequences of letters from Σ result in a *word* or *monomial*. The unique sequence of zero elements is referred to as the *empty word* and is denoted by 1. Let Σ^+ be the *Kleene plus* of Σ [21], i.e. the

set of all words formed from letters from the alphabet Σ . Expanding this set to include the empty word results in the *Kleene star* $\Sigma^* = \Sigma^+ \cup \{1\}$ of Σ [21]. Define an associative binary operation $\cdot : \sigma \times \sigma \rightarrow \sigma$ with $\sigma \in \{\Sigma^+, \Sigma^*\}$ as the concatenation of words from Σ^+ or Σ^* . One obtains the *free semigroup* $(\Sigma^+, \cdot)$ and the *free monoid* $(\Sigma^*, \cdot)$ with the empty word as the identity element. We then refer to Σ as the set of *generators* of those algebraic structures. Let us focus on the free monoid $(\Sigma^*, \cdot)$ and define a map $|\cdot| : \Sigma^* \rightarrow \mathbb{N}^+$ assigning to each monomial a *length* (degree)². A *free *-monoid* [24] $(\Sigma^*, \cdot, *)$ is then $(\Sigma^*, \cdot)$ equipped with an *anti-involution* $*$: $(\Sigma^*, \cdot) \rightarrow (\Sigma^*, \cdot)$.

We are interested in non-commutative polynomial expressions in the monomials from $(\Sigma^*, \cdot, *)$ with coefficients from a field $\mathbb{K} \in \{\mathbb{R}, \mathbb{C}\}$. Consider the vector space $(\Sigma^*, +, \cdot_{\mathbb{K}})$. Lifting this vector space structure to a *free $\mathbb{K}$ -algebra* requires the definition of a $\mathbb{K}$ -bilinear operation (algebra multiplication) on this space. This multiplication is given by the concatenation of words. The corresponding unit element is again the empty word. If we then additionally define on this algebra a anti-involution $*$, we obtain the *free *-algebra* $\mathbb{K}\langle\Sigma\rangle$.

Remark. As opposed to a polynomial ring over Σ and $\mathbb{K}$, the order of letters in words is relevant. For example, in $\mathbb{K}\langle\Sigma\rangle$ the elements x_1x_2 and x_2x_1 are distinct. Hence, $\mathbb{K}\langle\Sigma\rangle$ constitutes the algebraic structure to capture non-commutative polynomials.

In summary, any element $p \in \mathbb{K}\langle\Sigma\rangle$ can be written as a finite polynomial

$$p = \sum_w p_w w \quad (2.65)$$

with words $w \in (\Sigma^*, \cdot, *)$ and $p_w \in \mathbb{K}$. To any such p ,

$$p^* = \sum_w (p_w)^* w^* \quad (2.66)$$

constitutes the image under the anti-involution defined on the algebra. The anti-involution acts on elements from $\mathbb{K}$ as a complex conjugation while on words as an anti-involution. A hermitian polynomial $p \in \mathbb{K}\langle\Sigma\rangle$ satisfies $p^* = p$. The degree of a polynomial $p \in \mathbb{K}\langle\Sigma\rangle$ is given by the maximal degree of the monomials in its linear combination. We adapt the notation introduced in [65] and denote by $\mathbb{K}\langle\Sigma\rangle_d$ the free *-algebra restricted to polynomials of degree $\leq d$.

²Each letter corresponds to a word of length one.

2.4.3 The universal C^* -algebra

We have introduced the algebraic structure corresponding to non-commutative polynomials. This structure alone is only of limited interest to us. We want to formulate optimisation problems in terms of non-commutative polynomials. Optimisation problems with non-commutative polynomials generally consist of polynomial objective functions subject to several polynomial constraint functions. These constraint functions are generally formulated as inequalities over the elements of the corresponding free structure³. Since we have no natural notion of inequalities in a free algebraic structure⁴ we need to provide extra structure. We refer to the finite set of algebraic expressions in $\mathbb{K}\langle\Sigma\rangle$ corresponding to our constraint inequalities as *relations* $\mathcal{R}$.

Remark. While $\mathcal{R}$ generates an infinite dimensional two-sided ideal in the free algebraic structure, we collect in $\mathcal{R}$ only the generators of these ideals, i.e. $\mathcal{R}$ is the smallest set expressing the constraint we want to enforce.

Example 2.3. Consider elements of Σ which we constrain to commute. Let $x, y \in \Sigma$ be such that $xy - yx = 0$. Then $xy - yx$ and $-xy + yx$ are elements of $\mathcal{R}$.

The mathematical description of quantum phenomena requires more machinery, such as state and operator spaces. Concretely, from a free $*$ -algebra with generators Σ and relations $\mathcal{R}$, we want to construct a C^* -algebra. We want to ensure that the elements of the resulting C^* -algebra adhere to these constraints and that we constructed the “largest” such C^* -algebra. While our intent is simple, the implementation following [46, II.B.2] will initially seem peculiar.

A representation π of the tuple $(\Sigma|\mathcal{R})$ maps elements from $\mathbb{K}\langle\Sigma\rangle$ to bounded operators in $\mathcal{B}(\mathcal{H})$ for some Hilbert space $\mathcal{H}$, such that $\pi(q)$ is a positive (semidefinite) operator $\forall q \in \mathcal{R}$. Concretely, for every $q \in \mathcal{R}$ we have $\langle\Omega, \pi(q)\Omega\rangle \geq 0 \ \forall \Omega \in \mathcal{H}$ and write

$$\pi(q) \succeq 0. \tag{2.67}$$

For a hermitian $x \in \mathbb{K}\langle\Sigma\rangle$ the operator $\pi(x)$ is hermitian. Then, $\langle\Omega, \pi(x)\Omega\rangle \in \mathbb{R} \ \forall \Omega \in \mathcal{H}$. In other words, representations of $(\Sigma|\mathcal{R})$ are bounded operators adhering to the relations. Thus, any representation on $(\Sigma|\mathcal{R})$ defines a representation on $\mathbb{K}\langle\Sigma\rangle$.

Remark. We have exploited representations of algebra elements on $\mathcal{B}(\mathcal{H})$ to formulate the inequality conditions on $\mathcal{R}$ as a positive operator requirement. We only allow for

³We can formulate $x = 0$ as two inequalities $x \geq 0$ and $x \leq 0$. In our unital free $*$ -algebra, we can always formulate inequalities relative to 0.

⁴We do not yet have defined a norm on the structure.

representations yielding positive operators to $\mathcal{R}$. We will see that this ensures elements of the resulting C^* -algebra adhere to the constraints we initially wished to enforce.

When constructing C^* -algebras from generators and relations, we must address one more problem. Consider generators Σ giving rise to a free $*$ -algebra $\mathbb{K}\langle\Sigma\rangle$ and relations $\mathcal{R}$. If we wish to construct a C^* -algebra from $\mathbb{K}\langle\Sigma\rangle$ and $\mathcal{R}$, we need to formulate a requirement on the relations to uphold the connection between C^* -algebras and $\mathcal{B}(\mathcal{H})$ for some Hilbert space $\mathcal{H}$. Concretely, the relations necessarily need to impose bounds on the elements of $\mathbb{K}\langle\Sigma\rangle$ so that representations on $\mathcal{B}(\mathcal{H})$ truly yield bounded operators. Since elements of $\mathbb{K}\langle\Sigma\rangle$ are finite sums over monomials in the generators from Σ , it suffices to include in $\mathcal{R}$ bounds on each generator.

Remark. In [65, p. 6] a requirement formulated on $\mathbb{K}\langle\Sigma\rangle$ and $\mathcal{R}$, namely the *Archimedean property*, ensures that the relations impose a bound on elements in $\mathbb{K}\langle\Sigma\rangle$ and thus, representations of $\mathbb{K}\langle\Sigma\rangle$ on $\mathcal{B}(\mathcal{H})$ yield bounded operators.

Remark. By requiring the representations of generators to correspond to projective operators, we implicitly bound the elements in $\mathbb{K}\langle\Sigma\rangle$.

Remark. If we are only given a set of generators Σ yielding a free $*$ -algebra $\mathbb{K}\langle\Sigma\rangle$, we cannot construct a corresponding C^* -algebra since no bound on algebra elements exists.

Assume that the relations are such that π is a representation of $(\Sigma|\mathcal{R})$ on $\mathcal{B}(\mathcal{H})$. Then, π allows us to define a C^* -seminorm on $\mathbb{K}\langle\Sigma\rangle$. For all $x \in \mathbb{K}\langle\Sigma\rangle$, there exists

$$\|x\|_{\mathbb{K}\langle\Sigma\rangle} = \sup\{\|\pi(x)\|_{\mathcal{B}(\mathcal{H})} : \pi \text{ representation of } (\Sigma|\mathcal{R})\}, \quad (2.68)$$

since the supremum is finite. Correspondingly, we identify the ideal $I \subseteq \mathbb{K}\langle\Sigma\rangle$ via

$$\mathcal{I} = \{x \in \mathbb{K}\langle\Sigma\rangle : \|x\|_{\mathbb{K}\langle\Sigma\rangle} = 0\}. \quad (2.69)$$

Remark. The elements of this ideal result from the relations and for $\mathcal{R} = \emptyset \Rightarrow \mathcal{I} = \emptyset$.

As before, complete the quotient of $\mathbb{K}\langle\Sigma\rangle/\mathcal{I}$ to obtain a *universal C^* -algebra* $C^*(\Sigma|\mathcal{R})$ [11 p.158]. The completion ensures that for any two elements $x, y \in \mathbb{K}\langle\Sigma\rangle$ yields $x = y$ in $C^*(\Sigma|\mathcal{R})$, if and only if $\|x - y\|_{\mathbb{K}\langle\Sigma\rangle} = 0$. It can be made into an unital C^* -algebra by explicitly extending Σ by 1 and including in $\mathcal{R}$ the necessary relations, such that 1 constitutes a unit of the C^* -algebra. By [46, Lemma 3], $\forall q \in \mathcal{R}$, q is positive in $C^*(\Sigma|\mathcal{R})$.

Remark. By [46, Lemma 3], we have constructed the “largest” C^* -algebra fulfilling the requirement that any $q \in \mathcal{R}$ is positive.

2.4.4 Ideals from equality constraints

Consider the equality $x = y$ for $x, y \in \mathbb{K} \langle \Sigma \rangle$ corresponding to $x - y, y - x \in \mathcal{R}$. For any representation π of $(\Sigma | \mathcal{R})$ we have

$$\pi(x - y) \succeq 0 \quad , \quad \pi(y - x) \succeq 0 \quad \Rightarrow \pi(x) = \pi(y) \quad (2.70)$$

and the norm $\|\cdot\|_{\mathbb{K} \langle \Sigma \rangle}$ completion of $\mathbb{K} \langle \Sigma \rangle / \mathcal{I}$ with ideal $\mathcal{I}$ from Eq. (2.69) yields $x = y$ in $C^*(\Sigma | \mathcal{R})$. Any equality e generates a two-sided ideal

$$\mathcal{E} = \{zew : z, w \in \mathbb{K} \langle \Sigma \rangle\} , \quad (2.71)$$

such that $\forall x \in \mathcal{E} : \|x\|_{\mathbb{K} \langle \Sigma \rangle} = 0$.

2.4.5 Tensor products of C^* -algebras

We are interested in the mathematical description of composite quantum systems. We face a challenge when attempting to construct a C^* -algebra describing the composite system from C^* -subalgebras corresponding to the different subsystems. Following the arguments in [46], sec. II B 4] and [11], sec. II.9] defining a C^* -norm on the $*$ -algebra corresponding to the *algebraic tensor product* $\otimes_{alg}$ of two C^* -algebras $\mathcal{A}, \mathcal{B}$ is not unique.

Remark. Elements of $\mathcal{A} \otimes_{alg} \mathcal{B}$ are denoted as $a \otimes b$ for all $a \in \mathcal{A}, b \in \mathcal{B}$. Multiplication and involution on $\mathcal{A} \otimes_{alg} \mathcal{B}$ are defined componentwise with respect to “ $\otimes$ ”.

Let us first address how to define representations of the elements from $\mathcal{A} \otimes_{alg} \mathcal{B}$. Consider the faithful representations $\pi_{\mathcal{A}} : \mathcal{A} \rightarrow \mathcal{H}_{\mathcal{A}}^A$ and $\pi_{\mathcal{B}} : \mathcal{B} \rightarrow \mathcal{H}_{\mathcal{B}}^B$. Then, $\pi = \pi_{\mathcal{A}} \otimes \pi_{\mathcal{B}} : \mathcal{A} \otimes_{alg} \mathcal{B} \rightarrow \mathcal{B}(\mathcal{H}_{\mathcal{A}} \otimes \mathcal{H}_{\mathcal{B}})$ defines a faithful representation inducing the *minimal C^* -norm (spatial norm)* on $\mathcal{A} \otimes_{alg} \mathcal{B}$ via

$$\|a \otimes b\|_{\min} = \sup\{\|(\pi_{\mathcal{A}} \otimes \pi_{\mathcal{B}})(a \otimes b)\|_{\mathcal{B}(\mathcal{H}_{\mathcal{A}} \otimes \mathcal{H}_{\mathcal{B}})} : \pi_{\mathcal{A}}, \pi_{\mathcal{B}} \text{ a rep. of } \mathcal{A}, \mathcal{B}\} \quad (2.72)$$

$\forall a \in \mathcal{A}$ and $\forall b \in \mathcal{B}$. Completing $\mathcal{A} \otimes_{alg} \mathcal{B}$ with respect to $\|\cdot\|_{\min}$ yields the *minimal tensor product* of $\mathcal{A}, \mathcal{B}$ denoted $\mathcal{A} \otimes_{\min} \mathcal{B}$. In other words, $\mathcal{A} \otimes_{\min} \mathcal{B}$ corresponds to the C^* -algebra where the tensor product of representations gives a representation of elements to the subalgebras $\mathcal{A}$ and $\mathcal{B}$. Equivalently, the corresponding Hilbert spaces are in a tensor product relation.

Remark. In elementary quantum theory, we lift local operators defined on local subsystems to representations as global operators defined on the composite system by simply padding them with the corresponding identities of the other subsystems. The Hilbert

space to the composite system is assumed to be in a minimal tensor product over the Hilbert spaces to the individual subsystems. This corresponds to representations of C^* -algebras in a minimal tensor product relation.

Consider two commuting C^* -subalgebras $\mathcal{A}, \mathcal{B}$ of a C^* -algebra $\mathcal{U}$. By [29, Theorem 1, p.] $\mathcal{A}$ and $\mathcal{B}$ are C^* -independent if and only if to every state $\varrho_{\mathcal{A}} \in S(\mathcal{A})$ and $\varrho_{\mathcal{B}} \in S(\mathcal{B})$ there exists a state $\varrho \in S(\mathcal{U})$ such that $\forall a \in \mathcal{A}$ and $\forall b \in \mathcal{B}$

$$\varrho(ab) = \varrho_{\mathcal{A}}(a)\varrho_{\mathcal{B}}(b) = \varrho(a)\varrho(b). \quad (2.73)$$

Equivalently, we require that any state on $\mathcal{U}$ yields states on the subalgebra when restricted to those algebras, i.e.

$$\varrho|_{\mathcal{A}} = \varrho_{\mathcal{A}} \quad \varrho|_{\mathcal{B}} = \varrho_{\mathcal{B}}. \quad (2.74)$$

This notion is the C^* -algebraic equivalent to a *product state* [39] in elementary quantum theory. We adapt the terminology and refer to such a ϱ as a *product state*. Consider two commuting subalgebras $\mathcal{A}, \mathcal{B}$ generating a C^* -algebra $\mathcal{U}$. Let $\varrho \in S(\mathcal{U})$ be a product state over $\mathcal{A}, \mathcal{B}$. The GNS construction [46, p.12] with ϱ yields the GNS representation π_{ϱ} such that

$$\begin{aligned} \pi_{\varrho}(ab) &= \pi_{\mathcal{A}}(a) \otimes \pi_{\mathcal{B}}(b), \quad \forall a \in \mathcal{A}, \forall b \in \mathcal{B} \\ \mathcal{H}_{\varrho} &= \mathcal{H}_{\varrho}^{\mathcal{A}} \otimes \mathcal{H}_{\varrho}^{\mathcal{B}} \end{aligned} \quad (2.75)$$

where $\pi_{\mathcal{A}} : \mathcal{A} \rightarrow \mathcal{H}_{\varrho}^{\mathcal{A}}$ and $\pi_{\mathcal{B}} : \mathcal{B} \rightarrow \mathcal{H}_{\varrho}^{\mathcal{B}}$. Thus, we can assume without loss of generality that $\mathcal{U} = \mathcal{A} \otimes_{\min} \mathcal{B}$.

Now by [11, p. II.9.2.1] any non-degenerate representation of $\mathcal{A} \otimes_{alg} \mathcal{B}$ on $\mathcal{B}(\mathcal{H})$ allows us to identify non-uniquely $\pi_{\mathcal{A}} : \mathcal{A} \rightarrow \mathcal{B}(\mathcal{H})$ and $\pi_{\mathcal{B}} : \mathcal{B} \rightarrow \mathcal{B}(\mathcal{H})$ such that

$$\pi(a \otimes b) = \pi_{\mathcal{A}}(a)\pi_{\mathcal{B}}(b) = \pi_{\mathcal{B}}(b)\pi_{\mathcal{A}}(a). \quad (2.76)$$

This formalises the requirement that the C^* -algebra to the composite system contains $\mathcal{A}, \mathcal{B}$ as commuting subalgebras. However, we do not assume the corresponding Hilbert space to have a tensor product structure.

Remark. $\mathcal{A} \otimes_{\min} \mathcal{B}$ also contains $\mathcal{A}, \mathcal{B}$ as commuting subalgebras but additionally requires the representation space to be in a tensor product relation.

The *maximal* C^* -norm is defined as

$$\|a \otimes b\|_{\max} = \sup\{\|\pi(a \otimes b)\|_{\mathcal{B}(\mathcal{H})} : \pi \text{ a representation of } \mathcal{A} \otimes_{\text{alg}} \mathcal{B}\} \quad (2.77)$$

$\forall a \in \mathcal{A}$ and $\forall b \in \mathcal{B}$ with which the completion of $\mathcal{A} \otimes_{\text{alg}} \mathcal{B}$ yields the *maximal tensor product* $\mathcal{A} \otimes_{\max} \mathcal{B}$. From [11, sec. II.9.2.5] $\mathcal{A} \otimes_{\max} \mathcal{B}$ is the C^* -algebra generated by unital copies of $\mathcal{A}, \mathcal{B}$ such that they are commuting subalgebras.

Remark. Since all our C^* -subalgebras are unital, their minimal and maximal tensor product will be too [11, sec. II.9.2.5].

Theorem 2.8. *Consider a $*$ -algebra $\mathcal{A} = \bigotimes_{i=1}^n \mathcal{A}_i$ as an algebraic tensor product over $\mathcal{A}_i$ C^* -algebras. Let us denote the completion of $\mathcal{A}$ with respect to $\|\cdot\|_{\max}$ by $\mathcal{A}_{\|\cdot\|_{\max}}$ and with respect to any other C^* -seminorm $\|\cdot\|_{\mathcal{A}}$ defined on $\mathcal{A}$ as $\mathcal{A}_{\|\cdot\|_{\mathcal{A}}}$. Then*

$$\mathcal{A}_{\|\cdot\|_{\mathcal{A}}} \subseteq \mathcal{A}_{\|\cdot\|_{\max}}. \quad (2.78)$$

Proof. For any $x \in \mathcal{A}$ we have $\|x\|_{\mathcal{A}} \leq \|x\|_{\max}$ by definition of the maximal norm [11, p. II.9.5.2]. Then with

$$I_{\|\cdot\|_{\max}} := \{x \in \mathcal{A} : \|x\|_{\max} = 0\} \subseteq \{x \in \mathcal{A} : \|x\|_{\mathcal{A}} = 0\} =: I_{\|\cdot\|_{\mathcal{A}}} \quad (2.79)$$

follows $\mathcal{A}/I_{\|\cdot\|_{\mathcal{A}}} \subseteq \mathcal{A}/I_{\|\cdot\|_{\max}}$ so that the completion yields $\mathcal{A}_{\|\cdot\|_{\mathcal{A}}} \subseteq \mathcal{A}_{\|\cdot\|_{\max}}$. $\square$

Remark. Informally, $\|\cdot\|_{\max}$ defines a finer topology on $\mathcal{A}$ and thus allows us to differentiate the limit points of two Cauchy sequences, which are considered equal in a coarser topology.

2.5 Non-commutative polynomial optimization theory

In [65], a general framework to treat arbitrary non-commutative polynomial optimisation problems was introduced. It allows us to formulate NPO problems arising in quantum inflation as hierarchies of SDPs. These can then be numerically treated on a computer. We will next introduce all relevant concepts following [65, 46].

Consider a set of generators⁵ $\Sigma = \{x_1, \dots, x_n, x_1^*, \dots, x_n^*\}$ which defines a free $*$ -algebra $\mathbb{K} \langle \Sigma \rangle$ with $\mathbb{K} = \{\mathbb{R}, \mathbb{C}\}$. Also, consider relations $\mathcal{R} \subseteq \mathbb{K} \langle \Sigma \rangle$. We assume that the relations impose the necessary bounds on elements of $\mathbb{K} \langle \Sigma \rangle$. Let $p \in \mathbb{K} \langle \Sigma \rangle$ be a polynomial representing the objective function of interest. Let $C^*(\Sigma|\mathcal{R})$ be the corresponding universal unital C^* -algebra. Additionally, we want to allow for constraints on images of $C^*(\Sigma|\mathcal{R})$ under the action of states. Let $\mathcal{S} \subseteq \mathbb{K} \langle \Sigma \rangle$ be a countable subset such that $\varrho(y) \geq 0$ for any $s \in \mathcal{S}$ and $\varrho \in S(C^*(\Sigma|\mathcal{R}))$. We obtain an optimisation over states ϱ from the convex set $S(C^*(\Sigma|\mathcal{R}))$ [46, eq. 7]

Problem 2.1 ($C^*(\Sigma|\mathcal{R})$ NPO problem).

$$\begin{aligned} p^* &= \min_{\varrho \in S(C^*(\Sigma|\mathcal{R}))} \varrho(p) \\ \tilde{\mathbf{P}}: \quad \text{s.t.} \quad & \varrho(s) \geq 0, \quad s \in \mathcal{S}. \end{aligned} \tag{2.80}$$

We can rephrase this rather general formulation. Via the GNS construction, we can construct to any optimal $\omega \in S(C^*(\Sigma|\mathcal{R}))$ of $\tilde{\mathbf{P}}$ a GNS representation π_ω , a GNS Hilbert space $\mathcal{H}_\omega$ and a GNS cyclic vector $\Omega_\omega \in \mathcal{H}_\omega$. Let π be a representation of $C^*(\Sigma|\mathcal{R})$ in $\mathcal{B}(\mathcal{H})$ unitary equivalent to such a GNS representation and let $\Omega \in \mathcal{H}$ be a unit vector. Then,

Problem 2.2 (NPO problem).

$$\begin{aligned} p^* &= \min_{\pi, \Omega} \langle \pi(p) \rangle_\Omega \\ \mathbf{P}: \quad \text{s.t.} \quad & \pi(q) \succeq 0, \quad \forall q \in \mathcal{R} \\ & \langle \pi(s) \rangle_\Omega \geq 0, \quad \forall s \in \mathcal{S} \end{aligned} \tag{2.81}$$

is equivalent [46, Lemma 4] to problem [2.1]

Remark. Since we optimise over representations π and vectors Ω , the corresponding Hilbert space $\mathcal{H}$ is not fixed. No bound on its dimension is being enforced.

⁵In all of this thesis, we do not explicitly include an identity generator, but always assume that there is one.

Remark. We have mentioned state space is a compact space in the weak-* topology induced on $S(C^*(\Sigma|\mathcal{R}))$ from $C^*(\Sigma|\mathcal{R})$. The weak-* compactness of state space ensures that the infimum is obtained [65] Lemma 5].

Remark. We optimise over all of state space and do not restrict to its boundary. Thus, a state can be mixed or pure in $\tilde{\mathbf{P}}$.

2.5.1 SDP relaxations

Let us assume that $\tilde{\mathbf{P}}$ (or $\mathbf{P}$) is feasible, i.e. there exists $\varrho \in S((C^*(\Sigma|\mathcal{R})))$ (or (π, Ω)) with corresponding Hilbert space $\mathcal{H}$) such that the constraints are fulfilled. Rather than considering the full problem $\tilde{\mathbf{P}}$, we focus on finite-dimensional subspaces $\mathcal{V}^{(k)} \subset \mathbb{K}\langle \Sigma \rangle$, $k \in \mathbb{N}_+$ with monomial basis $\mathcal{B}_k = \{w_1^{(k)}, \dots, w_{d_k}^{(k)}\}$, i.e. $|\mathcal{B}_k| = d_k$. Recall that any element in $\mathcal{V}^{(k)}$ can be written as a finite polynomial with coefficients from $\mathbb{K}$ and words from $\mathcal{B}_k$. From $\mathbb{K}\langle \Sigma \rangle$, we construct a hierarchy of these subspaces such that for any $k \leq k'$, $\mathcal{V}^{(k)} \subset \mathcal{V}^{(k')}$ and for $\lim_{k \rightarrow \infty} \mathcal{V}^{(k)} = \mathbb{K}\langle \Sigma \rangle$. In other words, we choose a partitioning of $\mathbb{K}\langle \Sigma \rangle$. This partitioning is non-unique. From this we construct a hierarchy of necessary conditions that any feasible ϱ to $\tilde{\mathbf{P}}$ (or (π, Ω) to $\mathbf{P}$) needs to fulfill. For a specific choice of partitioning, this hierarchy of necessary conditions is referred to as the *Navascués-Pironio-Acín (NPA) hierarchy* [54] [55], [65].

We bound the value of k from below by requiring that $\forall k$

$$\mathcal{R} \subset \mathcal{V}^{(2k)} \quad \wedge \quad p \in \mathcal{V}^{(2k)}. \quad (2.82)$$

In other words, we require k to assume a value such that we can express all elements of $\mathcal{R}$ and the objective function p via basis elements from $\mathcal{B}_{2k}$.

Remark. We can always choose to consider a relaxation of the problem by only enforcing a subset of the constraints. However, we require k to be such that elements in $\mathcal{R}$ enforce norm bounds on generators in $\mathbb{K}\langle \Sigma \rangle$. For the NPO problem considered in this thesis, we will require a lower bound on k to enforce a specific subset $\mathcal{S}' \subset \mathcal{S}$ to be in $\mathcal{V}^{(2k)}$.

At any given step k in this hierarchy, we consider the dual space $(\mathcal{V}^{(2k)})^*$ with linear functional L_y

$$\begin{aligned} L_y : \quad \mathcal{V}^{(2k)} &\rightarrow \mathbb{K} \\ x &\mapsto L_y(x). \end{aligned} \quad (2.83)$$

We adapt the notation and terminology from [65] and refer to the image of $w \in \mathcal{B}_k$ under L_y as a *moment* y_w . By linearity, the image of any element from $\mathcal{V}^{(2k)}$ under L_y

can be written as a finite sum with coefficients from $\mathbb{K}$ and moments y_w with $w \in \mathcal{B}_{2k}$. From $\mathcal{B}_k$, construct a *moment matrix* $\Gamma^{(k)} \in \mathbb{K}^{d_k \times d_k}$ via

$$\Gamma_{i,j}^{(k)} = L_y \left((w_i^{(k)})^* w_j^{(k)} \right) = y_{(w_i^{(k)})^* w_j^{(k)}}. \quad (2.84)$$

Remark. $\Gamma^{(k)}$ is indexed in $\mathcal{B}_k$ but contains the moments to all monomials appearing in $\mathcal{V}^{(2k)}$. We sometimes refer to $\mathcal{B}_k$ constructing $\Gamma^{(k)}$ as the *generating monomials*.

The subspace $\mathcal{V}^{(2k)}$ is not a C^* -algebra; hence, we cannot define a C^* -algebraic state mapping its elements to the underlying field $\mathbb{K}$. We relax the requirements imposed on states $\varrho \in S(C^*(\Sigma|\mathcal{R}))$ to the universal C^* -algebra $C^*(\Sigma|\mathcal{R})$ to requirements on functionals on $\mathcal{V}^{(2k)}$. We obtain necessary conditions by requiring that L_y be a normalised positive functional⁶ on $\mathcal{V}^{(2k)}$. Concretely, for any $w, v \in \mathcal{V}^{(2k)}$ and unit $\mathbb{1} \in \mathcal{V}^{(2k)}$

$$\begin{aligned} \text{Normalisation:} \quad & y_{\mathbb{1}} = L_y(\mathbb{1}) = 1 \\ \text{Positivity:} \quad & y_{w^*w} = L_y(w^*w) \geq 0 \\ \text{Hermiticity:} \quad & y_{w^*v} = L_y(w^*v) = L_y(v^*w)^* = y_{w^*v}^*. \end{aligned} \quad (2.85)$$

Equivalently, we require $\Gamma^{(k)} \succeq 0$ and set the entry corresponding to $L_y(\mathbb{1})$ to 1. Any state $\varrho \in C^*(\Sigma|\mathcal{R})$ necessarily satisfies those conditions. Conversely, any L_y satisfying Eq. (2.85) is not guaranteed to be a state in $S(C^*(\Sigma|\mathcal{R}))$. In summary, requiring L_y to be a normalised positive functional on $\mathcal{V}^{(2k)}$ constitutes a relaxation of the state requirement on $C^*(\Sigma|\mathcal{R})$.

So far, we have not included the requirement that L_y satisfies the constraints we imposed in $\tilde{\mathbf{P}}$. For any $x, y \in \mathbb{K} \langle \Sigma \rangle$ and $q \in \mathcal{R}$

$$L_y(x^*qx) \geq 0, \quad L_y(x^*qy) = L_y(y^*qx)^* \quad (2.86)$$

since q is positive. We introduce a relaxation of the above by requiring that all $q \in \mathcal{R}$ satisfy Eq. (2.86) only to the extent possible in $\mathcal{V}^{(2k)}$. To each $q \in \mathcal{R}$ we require L_y to be positive over $x^*qx, x^*qy \in \mathcal{V}^{(2k)}$ with $x, y \in \mathcal{V}^{(2k)}$. Let us exploit the linearity of L_y and the basis $\mathcal{B}_k$. To all $q \in \mathcal{R}$, construct a *localising matrix* $\Lambda_q^{(k)} \in \mathbb{K}^{t \times t}$ via

$$\left(\Lambda_q^{(k)} \right)_{i,j} = L_y(b_i^* q b_j) = y_{(b_i)^* q b_j} \quad (2.87)$$

where $(b_i, b_j) \in \{(w, v) \in \mathcal{B}_k : w^*qv \in \mathcal{V}^{(2k)}\}$. We sandwich q with basis elements from

⁶In [65] p.7], this requirement is formulated as the y_w for $w \in \mathcal{B}_{2k}$ admitting a *moment representation*.

$\mathcal{B}_k$ such that the resulting polynomial is an element of $\mathcal{V}^{(2k)}$. Therefore,

$$t = |\{w \in \mathcal{B}_k : 2|w| + |q| \leq \max_{v \in \mathcal{B}_k} |v|\}|. \quad (2.88)$$

Remark. While $\Gamma^{(k)}$ consists of moments corresponding to monomials, $\Lambda_q^{(k)}$ contains polynomial expressions in moments as entries.

We relax the condition on L_y given in Eq. (2.86) to $\Lambda_q^{(k)} \succeq 0$. Lastly, for $\forall s \in \mathcal{S}$ we require

$$L_y(s) \geq 0. \quad (2.89)$$

We can thus formulate the SDP

Problem 2.3 (SDP Relaxations of order k to $\tilde{\mathbf{P}}$ (or $\mathbf{P}$)).

$$\begin{aligned} \mathbf{R}^{(k)} : \quad & \text{s.t.} \quad p^k = \min_{L_y \in (\mathcal{V}^{(2k)})^*} L_y(p) \\ & \Gamma^{(k)} \succeq 0 \\ & \Lambda_q^{(k)} \succeq 0 \quad \forall q \in \mathcal{R} \cap \mathcal{V}^{(2k)} \\ & L_y(s) \geq 0 \quad \forall s \in \mathcal{S} \cap \mathcal{V}^{(2k)}. \end{aligned} \quad (2.90)$$

Remark. This finite problem can be fully characterised in moments of basis elements. Thus, we can formulate the optimisation in terms of a vector $\mathbf{y} \in \mathbb{K}^{|\mathcal{B}_{2k}|}$ collecting all moments y_w for all $w \in \mathcal{B}_{2k}$.

Any feasible point (π, Ω) of $\mathbf{P}$ necessarily yields a solution $L_y \in (\mathcal{V}^{(2k)})^*$ of $\mathbf{R}^{(k)}$ via

$$L_y(x) = \langle \pi(x) \rangle_\Omega \quad (2.91)$$

for all $x \in \mathcal{V}^{(2k)}$. The converse does not hold in general. Any (π, Ω) corresponding to a solution L_y to $\mathbf{R}^{(k)}$ is not a solution to $\mathbf{P}$. Since $\mathbf{R}^{(k)}$ constitutes a relaxation of a minimisation problem, we obtain a lower bound p^k on p^* to $\mathbf{P}$. By construction, for any $k \leq k'$ and $\forall q \in \mathcal{R}$, the matrices $\Gamma^{(k)}, \Lambda_q^{(k)}$ are principal submatrices of $\Gamma^{(k')}, \Lambda_q^{(k')}$ and

$$\Gamma^{(k')}, \Lambda_q^{(k')} \succeq 0 \Rightarrow \Gamma^{(k)}, \Lambda_q^{(k)} \succeq 0. \quad (2.92)$$

Thus, any solution L_y to k' is a solution to k and $p^k \leq p^{k'}$. If the relations impose the required bounds on $\mathbb{K} \langle \Sigma \rangle$ such that the operators satisfying the constraints are bounded, then $\lim_{k \rightarrow \infty} p^k \rightarrow p^*$ [65 Theorem 1].

Proof. We provide a sketch of the proof from [65] by mentioning the pivotal arguments. The hierarchy of relaxations corresponds to a monotonically increasing sequence of optimal values p^k with a limit point $\hat{p} = \lim_{k \rightarrow \infty} p^k$ [65, Lemma 4]. By construction $\hat{p} \leq p^*$. There exists $L_{\hat{y}}$ corresponding to an infinite vector $\hat{y}$ indexed in $\mathcal{W}_\infty$ which yields $\hat{p}$ [65, Lemma 5]. Using the GNS construction, from $L_{\hat{y}}$, construct a feasible point $(\pi_{L_{\hat{y}}}, \Omega_{L_{\hat{y}}})$ with optimal value $\hat{p}$ to $\mathbf{P}$. Since $\mathbf{P}$ is a minimisation problem, $\hat{p} \geq p^*$ and thus $\hat{p} = p^*$ [65, Theorem 1 proof]. $\square$

Remark. We will encounter NPO formulations where $\mathcal{Y}$ is countable infinite. By [46, Lemma 5], the completeness of the hierarchy remains valid.

Remark. There exist *rank conditions* [65, Section 3.3] yielding the optimal value p^* and optimal solution or ϱ to $\tilde{\mathbf{P}} ((\pi^*, \Omega^*) \text{ to } \mathbf{P})$ at a finite level k of the hierarchy.

Remark. While the completeness of the hierarchy is a seminal result, often, only the first steps of this hierarchy are computationally viable. There also does not exist an efficient algorithm to exploit the rank condition [65, Section 3.3].

Consider a set of generators Σ and relations $\mathcal{R}$. Recall that any equality $e = x - y = 0$ with $x, y \in \mathbb{K} \langle \Sigma \rangle$ with corresponding relations $\{x - y, -x + y\} \subseteq \mathcal{R}$ generates a two-sided ideal

$$\mathcal{E} = \{zew : z, w \in \mathbb{K} \langle \Sigma \rangle\} \quad (2.93)$$

such that $\forall v \in \mathcal{E} : \|v\|_{\mathbb{K} \langle \Sigma \rangle} = 0$. Consider m such equalities $\{e_i\}_{i=1}^m$ each generating such a two-sided ideal $\mathcal{E}_i$ in $\mathbb{K} \langle \Sigma \rangle$. Let $\mathcal{N} \subseteq \mathbb{K} \langle \Sigma \rangle$ be such that all its elements have a vanishing C^* -seminorm

$$\mathcal{N} = \{x \in \mathbb{K} \langle \Sigma \rangle : \|x\|_{\mathbb{K} \langle \Sigma \rangle} = 0\}. \quad (2.94)$$

Then,

$$\mathcal{E} := \bigcup_{i=1}^m \mathcal{E}_i \subseteq \mathcal{N}. \quad (2.95)$$

In the process of constructing a universal C^* -algebra $C^*(\Sigma|\mathcal{R})$ from $\Sigma, \mathcal{R}$, the completion will remove the elements from $\mathcal{N}$ so that they are not elements of $C^*(\Sigma|\mathcal{R})$. In the above construction of the SDP hierarchy to $\tilde{\mathbf{P}}$ we can thus restrict to the quotient algebra $\mathbb{K} \langle \Sigma \rangle / \mathcal{E}$. Let us denote partitions of this quotient algebra by $(\mathbb{K} \langle \Sigma \rangle / \mathcal{E})^{(k)}$ for $k \in \mathbb{N}_+$. Alternatively, consider partitions $\mathcal{V}^{(k)}$ of $\mathbb{K} \langle \Sigma \rangle$ as in the construction above.

Then

$$(\mathbb{K}\langle\Sigma\rangle/\mathcal{E})^{(k)} \neq \mathcal{V}^{(k)}/(\mathcal{E} \cap \mathcal{V}^{(k)}) , \quad (2.96)$$

because of the relations. However, in the limit we have

$$\lim_{k \rightarrow \infty} (\mathbb{K}\langle\Sigma\rangle/\mathcal{E})^{(k)} = \lim_{k \rightarrow \infty} \mathcal{V}^{(k)}/(\mathcal{E} \cap \mathcal{V}^{(k)}) = \mathbb{K}\langle\Sigma\rangle/\mathcal{E} . \quad (2.97)$$

via NPO theory. Hence, formulating requirements on $L_y \in \left(\mathcal{V}^{(k)}/(\mathcal{E} \cap \mathcal{V}^{(k)})\right)^*$ constitutes a relaxation of requirements on $L_y \in \left((\mathbb{K}\langle\Sigma\rangle/\mathcal{E})^{(k)}\right)^*$.

Example 2.4. Consider a partitioning of $\mathbb{K}\langle\Sigma\rangle$ into $\mathbb{K}\langle\Sigma\rangle_k$, i.e. polynomials of at most degree k . For $y \in \Sigma$ let $y^2 - y, -y^2 + y \in \mathcal{R}$ and let $\mathcal{E}$ be the two-sided ideal generated by the equality $y^2 - y = 0$. Then, with $x \in \mathbb{K}\langle\Sigma\rangle_k$ and $|x| = k$ we have

$$x^*yx \in \mathbb{K}\langle\Sigma\rangle_{2k+1}/(\mathcal{E} \cap \mathbb{K}\langle\Sigma\rangle_{2k+1}) \quad (2.98)$$

but $x^*y^2x \notin \mathbb{K}\langle\Sigma\rangle_{2k+1}/(\mathcal{E} \cap \mathbb{K}\langle\Sigma\rangle_{2k+1})$. However, $x^*y^2x \in (\mathbb{K}\langle\Sigma\rangle/\mathcal{E})^{(2k+1)}$ with $(\mathbb{K}\langle\Sigma\rangle/\mathcal{E})^{(2k+1)}$ a partitioning of $\mathbb{K}\langle\Sigma\rangle/\mathcal{E}$.

Remark. In practice, we will have to formulate an SDP hierarchy in terms of $\mathcal{V}^{(k)}/(\mathcal{E} \cap \mathcal{V}^{(k)})$.

3 The Causal Compatibility Problem

There are two closely related problems to the CCP. Given a probability distribution determine all causal models constituting a plausible explanation for the distribution (*causal discovery problem* [56]). Conversely, provided a causal model, determine the set of compatible distributions by deriving *causal compatibility constraints* that any compatible statistic needs to adhere to (*causal characterisation problem* [56])⁷. A prominent example of the former is given by the Bell inequalities, which any statistics arising from a local classical model must satisfy [9] [83].

The causal compatibility problem lies at the intersection of those problems. Treating only the causal compatibility problem avoids the necessity to address ambiguities (e.g. *scoring* [47] [28], *minimality* [62], *fine-tuning* [88]) that may arise in the less specified problems.

The theory on causal models is vast. We will briefly introduce the classical variant following [62] [7]. Subsequently, we will introduce a quantum generalisation limited to the application in this thesis following [87] [46] [30] and conclude the section by explicitly formulating the quantum CCP.

3.1 The classical variant

Let us firstly relate probabilities to graphs without a causal interpretation. Consider a probability distribution P over the set of random variables $\mathcal{X} = \{X_1, \dots, X_n\}$. Let us limit to situations where the X_i correspond to a discrete state space of finite cardinality, i.e. they take on values from a finite alphabet. We refer to P as being *Markov* relative to a DAG $\mathcal{G}$ with $V(\mathcal{G}) = \mathcal{X}$, if P factorises as

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i | \text{Pa}_{\mathcal{G}}(X_i)). \quad (3.1)$$

$\mathcal{G}$ is then a *Bayesian network* to P . The *local (parental) Markov condition* formalises a necessary and sufficient requirement for P to be Markov relative to $\mathcal{G}$. P is Markov relative to $\mathcal{G}$ if each $X_i \in \mathcal{X}$ is conditionally independent of its non-descendants given its parents

$$(X_i \perp\!\!\!\perp \text{Nd}_{\mathcal{G}}(X_i) | \text{Pa}_{\mathcal{G}}(X_i))_P. \quad (3.2)$$

⁷There exists a variety of terminology for concepts in the field of causality.

Equivalently, the d -separations in $\mathcal{G}$ must imply the conditional independencies in P in order for P to be Markov relative to $\mathcal{G}$. This is referred to as the *global Markov condition*.

Let us now consider DAGs capturing causal relations between vertices. A *causal structure* $\mathcal{D}$ of a set of random variables $\mathcal{X}$ is a DAG $\mathcal{G}$ with $V(\mathcal{G}) = \mathcal{X}$ and each edge corresponds to a direct cause among the corresponding variables. The parents to a vertex are its direct causes. In contrast, the ancestors to a vertex can be its direct or mediated causal influence. The graphs are necessarily acyclic, as no effect can be its cause.

In the stochastic interpretation a *classical causal model* (or *causal Bayesian network*) is then such a $\mathcal{D}$ together with *causal parameters* assigning each $X \in \mathcal{X}$ a conditional probability $P(X|\text{Pa}_{\mathcal{G}}(X))$. The causal model induces a probability distribution over $\mathcal{X}$ that is *stable* [62] (independencies invariant under changes in causal parameters) and Markov relative to $\mathcal{D}$.

In physics, we are accustomed to expressing causal relations in a *functional model*. A *structural classical causal model* ([62] p.44) is then a causal structure $\mathcal{D}$ together with parameters $\Theta_{\mathcal{D}}$ assigning to each $X_i \in \mathcal{X}$ a non-deterministic function $x_i = f(pa_i, u_i)$. The pa_i represent the values of the random variables $\text{Pa}_{\mathcal{D}}(X_i)$ in a direct functional relationship with X_i . The u_i capture our assumption that our functional equations characterising the causal relations between the X_i are not complete. There exist unobserved factors U_i influencing X_i . These *random disturbances* are distributed according to a probability measure $P(u_i)$. They are not represented in $\mathcal{D}$. We require the U_i to be mutually independent of salvaging the notion of a Markovian model. That is, the $\text{Pa}_{\mathcal{D}}(X_i)$ are complete in the sense that no immediate cause to X_i is omitted. Any causes we omit influence solely X_i and are represented by the disturbances U_i . Finally, the *causal Markov condition* ([62] p.30) states, that any Markovian causal model $M = (\mathcal{D}, \Theta_{\mathcal{D}})$ induces a stable probability distribution P satisfying the local Markov condition relative to $\mathcal{D}$. Hence, the functional equations more natural to physics impose just the same constraint that any P needs to adhere to in order to be Markov relative to a DAG. To every Bayesian network $\mathcal{G}$ characterizing P , one can find a structural classical causal model inducing $\tilde{P}$ such that $P = \tilde{P}$ ([62] p.31). When testing whether P is Markov relative to a DAG $\mathcal{G}$ we need not worry about the underlying model generating the conditional probabilities dictated by $\mathcal{G}$.

In summary, answering the CCP reduces to a test of whether the probability distribution P is Markov relative to the causal structure $\mathcal{G}$ under consideration. It is sufficient to check whether all d -separations in $\mathcal{G}$ correspond to conditional independencies in P .

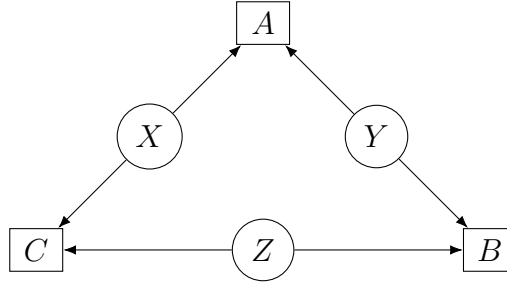


Figure 3.1: The classical triangle scenario consists of three unobserved random variables X, Y, Z and three observed random variables A, B, C . The causal structure captures the causal relations. X is a direct cause of A, B ; Y is a direct cause of A, C and Z of B, C . The X, Y, Z are mutually independent. The values that A, B, C attain are functions of the values their graph theoretical parents assume together with the random disturbances which are mutually independent between A, B, C . The CCP is whether a probability distribution over A, B, C is compatible with this description.

If we allow for unobserved (*hidden, latent* [62]) random variables to be explicitly represented in a DAG $\mathcal{G}$ to capture unobserved common causes the above is no longer sufficient. We denote these hidden variables by circular nodes in $\mathcal{G}$. Square nodes in $\mathcal{G}$ represent observed variables. Without loss of generality, by [72] the cardinalities of hidden random variables can also be chosen to be upper bounded. The probability distribution over the observed random variables arises from the full distribution via marginalisation over the probabilities of these hidden variables. Hence, we require that any compatible observed probability distribution arises from the marginalisation dictated by $\mathcal{G}$. This is called the *marginal problem*.

Problem 3.1 (The classical causal compatibility problem (CCP)). *Given a probability distribution P over the observed random variables in a classical causal model M with causal structure $\mathcal{G}$. Does there exist a probability distribution $\tilde{P}$ over all random variables in M , such that P is obtained from $\tilde{P}$ via marginalisation over the unobserved random variables and $\tilde{P}$ is Markov relative to $\mathcal{G}$?*

Example 3.1. Consider the classical triangle scenario given in Figure 3.1. $P(A, B, C)$ is compatible with the triangle scenario, if there exists $\tilde{P}(A, B, C, X, Y, Z)$ such that

$$\begin{aligned} P(A, B, C) &= \sum_{X, Y, Z} \tilde{P}(A, B, C, X, Y, Z) \\ &= \sum_{X, Y, Z} \tilde{P}(A|X, Y) \tilde{P}(B|Y, Z) \tilde{P}(C|X, Z) \tilde{P}(X) \tilde{P}(Y) \tilde{P}(Z). \end{aligned} \quad (3.3)$$

The d -separations over the subset of observed random variables in $\mathcal{G}$ still need to correspond to conditional independencies in any P that ought to be compatible with

$\mathcal{G}$ [86] [38]. As in example 3.1 there are scenarios of interest where this necessary condition on the observed variables does not impose any constraints on the set of compatible distributions. Additionally, independencies between random variables in a causal structure (such as between X, Y, Z in Figure 3.1) impose non-linear and non-convex constraints on any compatible probability distribution. These are very tricky to deal with and prevent a straightforward adaptation of the CCP to robust convex optimisation procedures.

By [72], if we indeed choose to upper bound the cardinality of the state space of the hidden random variables, then the set of compatible distributions P relative to a causal model M is a connected, closed and semi-algebraic set. As mentioned frequently in the literature [18, 56, 82, 8] determining such a set for any M is very hard. However, there exist several approaches to tackle the CCP. For example, in [18] the reformulation of independence relations leads to entropic constraints on any compatible distribution. Here, we will instead focus on the *classical inflation* procedure [86, 56] as a framework to tackle the CCP, which does not rely on a state space restriction of unobserved random variables.

3.2 The quantum variant

Consider an experiment regularly arising in quantum mechanics. We perform measurements on quantum systems obtaining a probability distribution over the random variables representing the measurement outcomes. We do not assume any knowledge about the exact measurement processes and apparatuses used, as well as how quantum states are prepared⁸. Thus, we formulate a causal hypothesis as a potential candidate explaining the observed statistics as a *device-independent quantum causal structure* (abbr. quantum causal structure) [87]. Again, we want to determine the compatibility of the obtained statistics and our assumed causal hypothesis.

By now, physicists have become well aware that the quantum-theoretical description of nature offers many peculiarities impeding straightforward adaptations of classical concepts to the quantum realm. Causal inference is no exception. As an example, consider the no-cloning theorem [89]. It prohibits us from creating an identical and independent copy of a given quantum state. Hence, it limits our ability to measure a quantum state and then use it as an input to subsequent operations. This does not pose a restriction in the classical case. Nevertheless, a wide variety of generalisations exist of the causal inference framework to quantum scenarios [7, 30, 17, 38, 20, 64, 2]⁹.

⁸While we assume without loss of generality that state preparations are pure, the overall state of a composite system need not be pure.

⁹By no means a complete list.

Substantial efforts have gone into salvaging the fundamental concept from classical causal inference for its quantum variant. More fundamentally, vivid discussions on the very notion of causality in quantum mechanics can be found in the provided literature.

Remark. Contrary to the classical variant, quantum theory is inherently probabilistic. Obtaining joint probability distributions over the observed variables corresponding to the outcomes of measurements on quantum systems is thus very natural.

We will abstain from entering into this discussion and refer to the literature. Here we take a rather economical approach. We will limit the introduction of *quantum causal models* to the concepts used in the quantum inflation technique in this thesis. This means, that we will restrict to *correlation scenarios* [30] (denoted *network forms* in [87]). These quantum causal structures comprise one layer of latent nodes with directed edges terminating in another layer of observed nodes. Latent nodes represent quantum systems, and information about them is solely obtained via measurement operations on these systems. An arbitrary quantum causal structure can be reduced to such a correlation scenario [87, 46]. From the set of correlation scenarios, we restrict to the *Bell scenario* and the *quantum triangle scenario*.

We start with addressing a technical challenge we face when providing a mathematical formulation of quantum causal structures. Subsequently, we elaborate on the mathematical interpretation of graphical components in a correlation scenario. This allows us to formalise a quantum variant of the CCP and explicitly formulate it concerning quantum causal structures of interest.

3.2.1 Modelling locality in quantum causal structures

Any quantum causal structure we consider is a composite quantum system consisting of local subsystems. Recall our freedom in choosing a tensor product to construct a C^* -algebra to the composite system from C^* -algebras to the subsystems. Let us address whether this choice has any implications on the compatibility of observed statistics given a causal hypothesis. The *Tsirelson problem* [77] tackles this question, and Tsirelson himself provided an informal formulation:

Problem 3.2 (Tsirelson’s problem [77]). *The subsystems may be treated either via (local) Hilbert spaces, – tensor factors of the given (global) Hilbert space, or via commuting (local) operator algebras. The latter approach is less restrictive; it requires that the given operators commute whenever they belong to different subsystems.*

Are these two approaches equivalent?

Let us rephrase Tsirelson's problem regarding quantum correlations as in [54, 31]. Any composite system can be grouped into a collection of bipartite systems. Hence, as in the literature, consider two spatially separated parties¹⁰ *Alice* and *Bob*. For simplicity, assume that Alice (Bob) can choose a measurement setting x (y) from a finite alphabet of cardinality k and that the outcome a (b) is one of m possible values. Let $\mathcal{A}$ ($\mathcal{B}$) be the C^* -algebra to Alice (Bob) constructed from the generators $\{A_a^x\}_{a,x}$ ($\{B_b^y\}_{b,y}$) and corresponding POVM relations. Note that we have indexed the generators in their settings and outcomes. The *tensor set of quantum correlations* $Q_\otimes$ [42] corresponds to the set of all *behaviours*

$$P = \{P(a, b|x, y)\}_{x,y \in \{1, \dots, k\}}^{a,b \in \{1, \dots, m\}} \quad (3.4)$$

arising from the C^* -algebra resulting from the norm completion of the minimal tensor product $\mathcal{A} \otimes_{\min} \mathcal{B}$. By [31] it is a convex set.

Remark. Whether $Q_\otimes$ is closed is handled differently by different authors. In short [31] argues that the difference is merely a mathematical pedantry and not of practical relevance. We will, therefore, not distinguish between $Q_\otimes$ and its closure $\overline{Q_\otimes}$.

Equivalently, the *commutative set of quantum correlations* Q_C [42] corresponds to the set of all behaviours arising from $\mathcal{A} \otimes_{\text{alg}} \mathcal{B}$ where we assume no extra structure but that $\mathcal{A}, \mathcal{B}$ be commuting subalgebras. By [31] Q_C is closed and convex.

Remark. The commutativity captures the joint (simultaneous) measurability in space-like separated regions. Hence, one has a physical motivation for the above description of locality [46]. In contrast, it is much harder to find an equivalent relation to physical reality for the Hilbert space tensor product description [31].

Remark. Relaxing orthogonality conditions (projectors) on operators to positivity conditions (POVMs) does not change $Q_\otimes, Q_C$ [42].

Since both descriptions require commutativity of the operators, $Q_\otimes \subseteq Q_C$ is trivial.

Problem 3.3 (Reformulation of Tsirelson's Problem). *Does*

$$Q_\otimes = Q_C \quad (3.5)$$

hold?

Firstly, note that Tsirelson himself was able to conclude that for finite-dimensional Hilbert spaces, $Q_\otimes = Q_C$ holds. In attempts to solve the posed problem for infinite-dimensional Hilbert spaces, [31] relates it to the *Kirchberg's QWEP conjecture*, while

¹⁰Subsystems need not be spatially separated to allow for the formulation of Tsirelson's problem. However, since causal analysis considers spatially separated subsystems, we will limit the discussion to those. Parties and subsystems are being used interchangeably.

analogously [42] shows *Connes’ embedding problem* to be “essentially equivalent to Tsirelson’s problem”. Relating a result in the field of complexity classes to work done by [31, 42] allowed [41] to conclude that for infinite-dimensional Hilbert spaces $Q_C \neq Q_\otimes$. The description of subsystems by commuting subalgebras of the algebra to the composite system captures more general behaviours. However, the concepts to obtain these results lie vastly outside the scope of this thesis.

Albeit an interesting problem with recent breakthrough results, we want to further elaborate on its relevance to this work. Not only has Tsirelson’s problem been the subject of discussions in [54], but since the publication of the LGG-hierarchy [46], there exists an algebraic analogue to the original quantum inflation procedure [87]. The former is, in theory, more general. The NPA hierarchy converges to Q_C [54]. For infinite-dimensional Hilbert spaces $Q_C \neq Q_\otimes$ ensuring convergence to $Q_\otimes$ would require adding constraints corresponding to the assumed tensor product structure. We follow [54, 65, 46]. We argue that commutativity constraints are far easier to implement within numerical implementations of the SDP formalism and thus decide on modelling locality by commutativity. Our algebraic description of quantum mechanics corresponds to the decision to describe composite systems via C^* -algebras which contain the algebras to local subsystems as commuting subalgebras.

3.2.2 Graphical description of quantum causal structures

We introduce the relevant components and their mathematical interpretation in the graphical description of quantum causal structures. We elaborate on combining them to formulate a quantum causal structure corresponding to a composite system and provide a quantum variant of the CCP. Subsequently, we make these concepts more tangible by applying them to the quantum triangle scenario and the Bell scenario.

Let a DAG $\mathcal{G}$ constitute a quantum causal structure. Then $V(\mathcal{G})$ consists of latent nodes representing quantum systems and observed nodes representing classical systems. Accordingly, $E(\mathcal{G})$ consists of edges representing classical information channels and edges as carriers of quantum information. *Classical edges* originate and terminate in classical nodes while *quantum edges* initiate from quantum nodes and terminate in classical nodes. A quantum edge is always itself a quantum system.

The correlation scenarios considered here consist of a collection of measurement procedures. The measurements correspond to POVMs and produce observable probabilities when combined with a state. Informally, the quantum variant of the CCP is then to decide whether such POVM operators exist and a state is adhering to the inde-

dependencies dictated by the quantum causal structure, which yields the joint probability distribution being tested. Concretely, we assign an algebra of observables generated by such POVM elements to each classical sink node. Enforcing POVM relations when constructing the corresponding universal C^* -algebra bounds all algebra elements [46, II.A.2]. When we specify such algebras and their related state spaces, we speak of a *quantum realisation* [87, p.3] of the underlying DAG.

Remark. Technically, an algebra of observables is a concrete realisation of an abstract C^* -algebra as an operator algebra. In other words, we consider C^* -algebras modelling bounded operators.

In the simplest case, such a measurement is only performed on one quantum system. In Figure 3.2 we specify how to depict this measurement procedure graphically. An algebra of observables $\mathcal{A}$ is generated by POVMs $\{E_{a|x}\}_{a,x}$ indexed in their setting x and outcome a . The latent node ϱ represents states in the state space $S(\mathcal{A})$. The outcome a of the state ϱ acting on a POVM element $E_{a|x}$ with setting dictated by the value x the classical random variable X assumes is assigned to the classical random variable A . Thus,

$$P(a|x) = P(x)\varrho(E_{a|x}) \quad (3.6)$$

where $P(x)$ constitutes a probability distribution over the measurement settings.

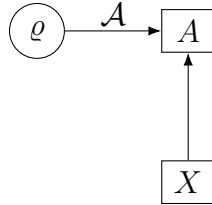


Figure 3.2: Graphical depiction of a measurement process on one quantum system.

Remark. We refer to the (sub)graph of nodes with directed edges terminating in an observed random variable as a party.

Let us now consider composite systems consisting of two quantum systems. We encounter two variants of such systems arising as components of larger structures. Notably, the way we connect graphical elements representing measurement procedures dictates the independence relations we assume in a causal hypothesis. Thus, the first component will describe two C^* -independent observable algebras, while the second structure does not. When we consider the mathematical description of a composite (global) system, we first define a universal C^* -algebra on this system. We then require

the local algebras to be commuting subalgebras and that they generate D . While we do not additionally impose a minimal tensor product relation between these subalgebras, there are cases where we can assume such a structure without loss of generality.

Remark. The local subalgebras inherit the norm from the global algebra of observables.

Consider the graph depicted in [Figure 3.3](#) to make this notion more precise. The $\mathcal{A}^L$ and $\mathcal{A}^R$ are two commuting subalgebras generating a C^* -algebra $\mathcal{A} := \mathcal{A}^L \otimes_{\max} \mathcal{A}^R$ of observables corresponding to the composite system. In other words, $\mathcal{A}$ is a universal C^* -algebra generated by POVMs $\{E_{a|x}\}_{a,x}$ so that $\mathcal{A}^L$ and $\mathcal{A}^R$ are commuting subalgebras. By the graph's structure, $\mathcal{A}^L, \mathcal{A}^R$ are C^* -independent. To any states $\varrho_L \in S(\mathcal{A}^L)$ and $\varrho_R \in S(\mathcal{A}^R)$ there exists a state $\varrho \in S(\mathcal{A})$ such that $\varrho|_{\mathcal{A}^L} =: \varrho_L$ and $\varrho|_{\mathcal{A}^R} =: \varrho_R$. By the C^* -independence, any state $\varrho \in S(\mathcal{A})$ will necessarily be a product state over $\mathcal{A}^L$ and $\mathcal{A}^R$; thus, we assume without loss of generality $\mathcal{A} = \mathcal{A}^L \otimes_{\min} \mathcal{A}^R$. The POVMs $\{E_{a|x}\}_{a,x} \subset \mathcal{A}$ together with $P(x)$ yield

$$P(a|x) = P(x)\varrho(E_{a|x}). \quad (3.7)$$

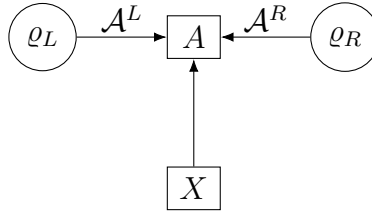


Figure 3.3: Graphical depiction of a measurement process on two independent quantum systems. We index the subalgebras in the direction they enter an observed node, i.e. from the left (L) or the right (R).

Remark. While the quantum inflation procedure introduced in [\[87\]](#) only considers the operators to the algebra $\mathcal{A}$, the LGG hierarchy [\[46\]](#) does not neglect this deeper structure and considers operators corresponding to algebra elements from $\mathcal{A}^L, \mathcal{A}^R$.

Remark. We also do not explicitly consider any operations from an algebra $\mathcal{U}$ before a POVM is applied and the result assigned to A as depicted in [Figure 3.4](#). Ultimately, this will look no different to us than a corresponding POVM operation directly applied to the composite system.

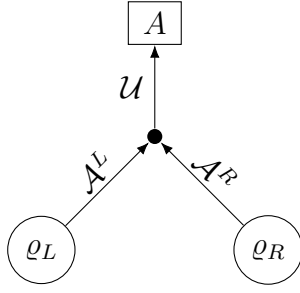


Figure 3.4: Graphical depiction of a quantum channel.

In contrast, consider the composite system describing the Bell scenario depicted in [Figure 3.5](#). A C^* -algebra $\mathcal{D} := \mathcal{A}^L \otimes_{\max} \mathcal{B}^R$ to the composite system is generated by $\mathcal{A}^R, \mathcal{B}^L$ and contains them as commuting subalgebras. We cannot assume any more structure. $\mathcal{A}^R$ and $\mathcal{B}^L$ are not C^* -independent by the graph's structure. Any $\varrho_{AB} \in S(\langle \mathcal{A}^L \cdot \mathcal{B}^R \rangle)$ is not, in general, a product state over these algebras. There exist POVMs $\{(A_R)_{a|x}\}_{a,x} \subset \mathcal{A}^R$ and $\{(B_L)_{b|y}\}_{b,y} \subset \mathcal{B}^L$ together with $P(x), P(y)$ such that

$$P(a, b|x, y) = P(x)P(y)\varrho_{AB}((A_R)_{a|x}(B_L)_{b|y}). \quad (3.8)$$

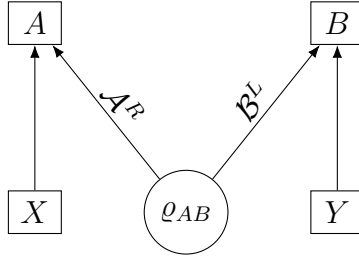


Figure 3.5: The Bell scenario.

In general, in a quantum realisation of a correlation scenario $\mathcal{G}$, we assign a C^* -algebra of observables $\mathcal{D}$ to the global system. This algebra is generated by commuting subalgebras, each corresponding to a quantum edge in $\mathcal{G}$. The graphical structure encodes the C^* -independence between certain subalgebras. Any state $\varrho \in S(\mathcal{D})$ must factorise according to these C^* -independent subalgebras.

We are now able to phrase the quantum variant of the CCP formally.

Problem 3.4 (The quantum causal compatibility problem (QCCP)). *Given a probability distribution P over the observed nodes in a device-independent quantum causal structure $\mathcal{G}$, do there exist a quantum realisation of $\mathcal{G}$ and $P(X|Pa_{\mathcal{G}}(X))$ for all classical random variables $X \in V(\mathcal{G})$ corresponding to settings, such that P and $\mathcal{G}$ are compatible?*

In analogy to the classical variant, a quantum causal model is then a quantum causal structure $\mathcal{G}$ together with a quantum realisation of $\mathcal{G}$ and conditional probabilities corresponding to parent-child relations between classical random variables.

Let us formulate the QCCP for the quantum triangle and Bell scenarios.

Example 3.2 (Quantum triangle scenario ([46] II.C)). Consider the quantum triangle scenario $\mathcal{G}$ depicted in [Figure 3.6](#). In a quantum realisation to $\mathcal{G}$, there exists a C^* -algebra of observables $\mathcal{D}$ to the composite system, such that it is generated by C^* -algebras $\mathcal{A}, \mathcal{B}, \mathcal{C}$ and that these are commuting subalgebras of $\mathcal{D}$, i.e.

$$\mathcal{D} = \mathcal{A} \otimes_{\max} \mathcal{B} \otimes_{\max} \mathcal{C}. \quad (3.9)$$

These commuting subalgebras are themselves algebras corresponding to a composite system in turn characterised by two commuting subalgebras which we can assume without loss of generality to be in a minimal tensor product relation

$$\mathcal{A} = \mathcal{A}^L \otimes_{\min} \mathcal{A}^R, \quad \mathcal{B} = \mathcal{B}^L \otimes_{\min} \mathcal{B}^R, \quad \mathcal{C} = \mathcal{C}^L \otimes_{\min} \mathcal{C}^R. \quad (3.10)$$

To $\mathcal{D}$ consider the state space $S(\mathcal{D})$. The C^* -independence of certain subalgebras dictates a factorisation property of states in that state space. Let us denote algebra elements according to their membership in one of these subalgebras $A_L \in \mathcal{A}^L, A_R \in \mathcal{A}^R, B_L \in \mathcal{B}^L, B_R \in \mathcal{B}^R, C_L \in \mathcal{C}^L$ and $C_R \in \mathcal{C}^R$. Then, $\forall \varrho \in S(\mathcal{D})$

$$\varrho(A_L A_R B_L B_R C_L C_R) = \varrho(A_L C_R) \varrho(A_R B_L) \varrho(B_R C_L). \quad (3.11)$$

We can reformulate this factorisation property by introducing subalgebra restrictions

$$\varrho|_{\langle \mathcal{A}^L, \mathcal{C}^R \rangle} =: \varrho_{AC} \quad \varrho|_{\langle \mathcal{A}^R, \mathcal{C}^L \rangle} =: \varrho_{AB} \quad \varrho|_{\langle \mathcal{B}^R, \mathcal{C}^L \rangle} =: \varrho_{BC} \quad (3.12)$$

yielding

$$\varrho = \varrho_{AB} \otimes \varrho_{AC} \otimes \varrho_{BC} \quad (3.13)$$

capturing the independence between these restrictions. Lastly, in $\mathcal{D}$, more specifically in the corresponding subalgebras, exist POVMs indexed in their corresponding outcome value a, b, c assigned to A, B, C

$$\{E_a\}_a \subset \mathcal{A}, \quad \{F_b\}_b \subset \mathcal{B}, \quad \{G_c\}_c \subset \mathcal{C} \quad (3.14)$$

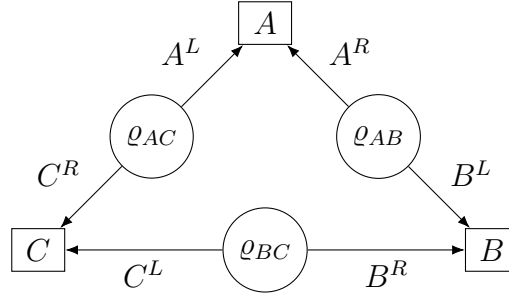


Figure 3.6: The quantum triangle scenario.

such that

$$P(a, b, c) = \varrho(E_a F_b G_c). \quad (3.15)$$

The QCCP is then concerned with deciding whether a given joint probability distribution matches $P(a, b, c)$ and thus is compatible with $\mathcal{G}$.

Remark. In practice, we provide a quantum realisation of a quantum causal structure by constructing a universal C^* -algebra to the composite system using generators, POVM (projector) and commutativity relations.

Analogously, the QCCP can be formulated for the Bell scenario in [Figure 3.5](#). There are distributions compatible with this causal model when assuming ϱ_{AB} to be quantum rather than a classical latent variable leading to a classical-quantum gap. Bell most notably realised this. However, as shown in [\[75\]](#), the QCCP is undecidable. This is in stark contrast to the classical case.

Remark. In [\[87\]](#) the QCCP to the Bell scenario is formulated as follows: do there exist Hilbert spaces $\mathcal{H}_A, \mathcal{H}_B$, a quantum state $\varrho_{AB} \in \mathcal{B}(\mathcal{H}_A \otimes \mathcal{H}_B)$, POVMs $E_{a|x} \in \mathcal{B}(\mathcal{H}_A)$ $F_{b|y} \in \mathcal{B}(\mathcal{H}_B)$ and probability distributions $P(X = x), P(Y = y)$ such that

$$P(A = a, B = b, X = x, Y = y) = P(X = x)P(Y = y) \text{Tr} \left[(E_{a|x} \otimes F_{b|y}) \varrho_{AB} \right]. \quad (3.16)$$

Note that this formulation additionally assumes the composite system's Hilbert space tensor product structure.

While many concepts from classical causal inference do not translate to the quantum variant, there exist some that do. The d -separation criterion for conditional independencies remains valid over the set of observable variables [\[38\]](#). However, [\[88\]](#) reveals severe limitations of its application to quantum causal scenarios. Causal discovery algorithms depending on such conditional independencies over observed variables cannot

distinguish Bell-inequality-violating and Bell-inequality-satisfying correlations. This is a fundamental problem when identifying probability distributions compatible with a quantum causal structure and not with its classical variant. Also, there are scenarios of interest where no such conditional independencies of observable variables exist (e.g. quantum triangle scenario).

The difficulty in solving the QCCP lies in the realisation that while state space is convex, formulating factorisation constraints capturing independencies modelled by the quantum causal structure renders the problem non-convex and non-linear [46, p. II.D]. Additionally, the universal C^* -algebra characterising the composite system can, in theory, be infinite-dimensional. These two aspects of the QCCP make it a challenging problem to consider.

There exist a variety of approaches to tackle the QCCP. The Bell scenario in Figure 3.5 was treated with the NPA hierarchy [55, 54] which was specifically designed to treat non-commutative optimisation problems with two parties arising in quantum information. Alternatively, [17] introduced a procedure based on entropies, while the *scalar extension* method [69] exploits independence of spatially separated parties. We will cover the latter in more detail in this thesis. Our main focus, however, lies on the quantum inflation procedure [87] as a framework to tackle the QCCP.

4 Inflations

Three inflation techniques tackling the CCP and QCCP have been proposed in recent years. These algorithms reformulate independencies into symmetries. They constitute a general framework applicable to any quantum causal model and were able to identify incompatible probability distributions where other procedures have failed to do so. In practice, their exponential numerical complexity limits their application.

4.1 Classical inflation

Let us outline the general idea behind the *classical inflation* procedure following [86]. We adapt the terminology introduced in this reference. In the next section, many concepts presented here directly translate to the quantum variant.

Consider a classical causal model M with $\mathcal{X} = \{X_i\}_{i=1}^n$ random variables. Let us not distinguish between the causal structure $\mathcal{D}$ and the DAG $\mathcal{G}$ representing it. Recall, M induces a probability distribution $P(\mathcal{X})$ which is Markov relative to $\mathcal{G}$. Since we allow for unobserved random variables $\mathcal{U} \subseteq \mathcal{X}$ to be represented in $\mathcal{G}$, the probability distribution $P(\mathcal{O})$ over the observed random variables $\mathcal{O} \subseteq \mathcal{X}$ is obtained via marginalisation over the probabilities to these unobserved ones. Any $P(\mathcal{O})$ compatible with M must result from the causal parameters in M via the Markov condition and marginalisation. Accordingly, given a probability distribution $P(\tilde{\mathcal{O}})$ over a subset of observed random variables $\tilde{\mathcal{O}} \subseteq \mathcal{O}$, $P(\tilde{\mathcal{O}})$ is compatible with M , if $\exists P(\mathcal{O})$ compatible with M and $P(\tilde{\mathcal{O}})$ is obtained via marginalisation from it.

For $\mathcal{S}_1, \mathcal{S}_2 \subseteq V(\mathcal{G})$, if $\text{An}_{\mathcal{G}}(\mathcal{S}_1) \cap \text{An}_{\mathcal{G}}(\mathcal{S}_2) = \emptyset$ then $\mathcal{S}_1, \mathcal{S}_2$ are *ancestrally independent (ai)*. Equivalently, $(\mathcal{S}_1 \perp\!\!\!\perp \mathcal{S}_2)_{\mathcal{G}}$. From the Markov condition together with $\mathcal{S}_1, \mathcal{S}_2$ ancestrally independent it follows that $P(\mathcal{S}_1, \mathcal{S}_2) = P(\mathcal{S}_1)P(\mathcal{S}_2)$. In the classical triangle scenario in Figure 3.1 X, Y, Z are ancestrally independent, but A, B, C are not. As mentioned, the independence relations are non-linear and non-convex and cannot be imposed in a convex optimisation framework. The inflation procedure offers a way to include relaxations of such independence constraints by connecting them to convex and linear symmetry constraints. These can then be effectively incorporated into convex optimisations.

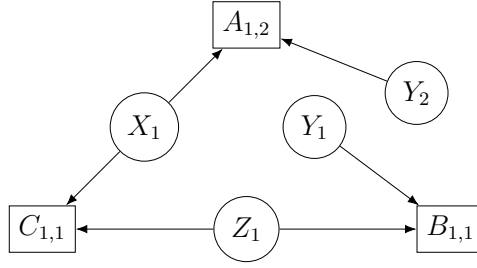


Figure 4.1: The cut inflation of the classical triangle scenario. Only the unobserved variable Y has been copied into Y_1, Y_2 such that $Y_1 \sim Y_2 \sim Y$. We labelled the observed nodes according to the unobserved nodes that influence them. Note that $\{A_{1,2}, Y_1, B_{1,1}\} \sim \{A, Y, B\}$, but $\{A_{1,2}, Y_1, Y_2, B_{1,1}\} \not\sim \{A, Y, B\}$.

4.1.1 Inflated causal structures

An *inflation* of a DAG $\mathcal{G}$ is another DAG $\mathcal{G}'$ where each $X_i \in V(\mathcal{G}')$ is the i -th copy of $X \in V(\mathcal{G})$ and the characteristics of the nodes in $\mathcal{G}$ are preserved. That is, for $X \in V(\mathcal{G})$ an unobserved (observed) node with cardinality c all copies $X_i \in V(\mathcal{G}')$ with *copy-index* $i = 1, \dots, k$ are unobserved (observed) nodes with cardinality c . Hence, in accordance with equivalence relations we say $X_j \sim X_i \sim X$ for all $i, j \in \{1, \dots, k\}$ are copy-index equivalent. Let $\mathcal{S} \subseteq V(\mathcal{G})$ and $\mathcal{S}' \subseteq V(\mathcal{G}')$, then $\mathcal{S} \sim \mathcal{S}'$ if and only if there do not exist two elements in $\mathcal{S}'$ that are copy-index equivalent to an element in $\mathcal{S}$. Additionally, $X_i Y_j \in E(\mathcal{G}')$ if and only if $XY \in E(\mathcal{G})$, i.e. that $\mathcal{G}'$ is locally isomorphic to $\mathcal{G}$. In summary, an inflation of $\mathcal{G}$ is a graph homomorphism (*fibration* [12], i.e. locally isomorphic) from $\mathcal{G}$ to $\mathcal{G}'$, such that

$$\forall X_i \in V(\mathcal{G}') : \text{Pa}_{\mathcal{G}'}(X_i) \sim \text{Pa}_{\mathcal{G}}(X). \quad (4.1)$$

These are non-unique as can be seen in the *cut* (Figure 4.1) and *web* (Figure 4.2) inflations of the classical triangle scenario. Now an inflation of a causal model M with $\mathcal{G}$ is a unique model M' such that $\mathcal{G}'$ is an inflation of $\mathcal{G}$ and

$$\forall X_i \in V(\mathcal{G}') : P(X_i | \text{Pa}_{\mathcal{G}'}(X_i)) = P(X | \text{Pa}_{\mathcal{G}}(X)). \quad (4.2)$$

4.1.2 Injectable and ai-expressible sets

We want to translate the compatibility conditions on a given $P(\mathcal{O})$ in $\mathcal{G}$ to compatibility conditions on $P(\mathcal{O}')$ in $\mathcal{G}'$. Both in $\mathcal{G}, \mathcal{G}'$ it holds that any probability distribution over a node or a set of nodes only depends on its ancestors. Consider $\mathcal{S} \subseteq V(\mathcal{G})$ and $\text{An}_{\mathcal{G}}(\mathcal{S})$. The *ancestral subgraph* of $\mathcal{S}$ in $\mathcal{G}$ is then the subgraph $\mathcal{G}_{\text{An}_{\mathcal{G}}(\mathcal{S})} \subseteq \mathcal{G}$ such that $V(\mathcal{G}_{\text{An}_{\mathcal{G}}(\mathcal{S})}) = \text{An}_{\mathcal{G}}(\mathcal{S})$ and $E(\text{An}_{\mathcal{G}}(\mathcal{S})) \subseteq \mathcal{G}_{\text{An}_{\mathcal{G}}(\mathcal{S})}$. Let $(M', \mathcal{G}')$ be an inflation of

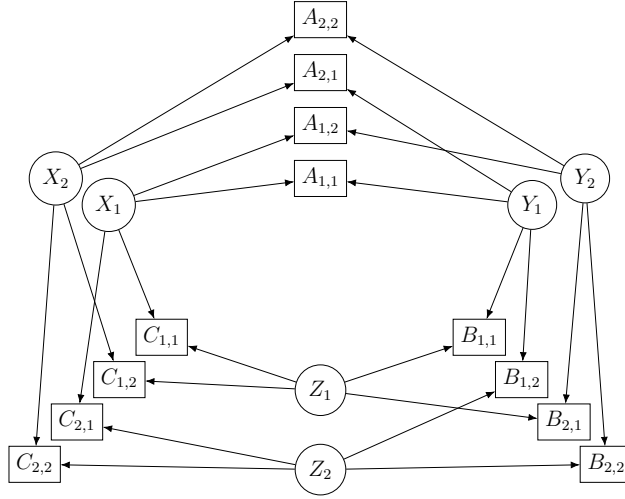


Figure 4.2: The Web inflation of the triangle scenario [86]. The unobserved variables X, Y, Z of the original classical triangle scenario in Figure 3.1 have been each copied twice $X_1, X_2, Y_1, Y_2, Z_1, Z_2$. Note that $X \sim X_1 \sim X_2$ and analogously for Y, Z . The observed nodes have been copied four times. Again $A_{1,1} \sim A_{1,2} \sim A_{2,1} \sim A_{2,2}$ and analogously for B, C . Note that $\{A_{2,2}, X_2, C_{2,2}\} \sim \{A, X, C\}$ but $\{A_{2,2}, A_{2,1}\} \not\sim \{A\}$.

$(M, \mathcal{G})$. If $\mathcal{S} \subseteq V(\mathcal{G})$ and $\mathcal{S}' \subseteq V(\mathcal{G}')$ such that $\mathcal{S} \sim \mathcal{S}'$ and

$$\mathcal{G}_{\text{An}_{\mathcal{G}}(\mathcal{S})} \sim \mathcal{G}_{\text{An}_{\mathcal{G}'}(\mathcal{S}')} . \quad (4.3)$$

then

$$P(\mathcal{S}) = P(\mathcal{S}') . \quad (4.4)$$

Since M' is an inflation of M , the probability distributions over $\mathcal{S}$ and $\mathcal{S}'$ are equal. We refer to an *injectable set* $\mathcal{V}_{\text{in}} \subseteq V(\mathcal{G}')$, if and only if $\exists \mathcal{W} \subseteq V(\mathcal{G})$, such that $\mathcal{V}_{\text{in}} \sim \mathcal{W}$ and $\mathcal{G}_{\text{An}_{\mathcal{G}'}(\mathcal{V}_{\text{in}})} \sim \mathcal{G}_{\text{An}_{\mathcal{G}}(\mathcal{W})}$. $\mathcal{W}$ is the *image* of the injectable set $\mathcal{V}_{\text{in}}$ in the original causal structure $\mathcal{G}$. In summary, to every injectable set in $\mathcal{G}'$, we assign the probability given by the probability to its image in $\mathcal{G}$. Hence, we need to determine those ancestral subgraphs of observed nodes in $\mathcal{G}'$, which are isomorphic to ancestral subgraphs in $\mathcal{G}$. As an example consider Figure 4.3.

A vital realisation and the fundamental insight behind the inflation procedure is that inflations allow us to exploit ancestrally independence in the inflated graph, whereas this might not have been possible in the original graph. A set $\mathcal{S}' \in V(\mathcal{G}')$ is an *ai-expressible set*, if and only if

$$\mathcal{S}' = \bigcup_j \mathcal{V}_{\text{in}}^j \quad (4.5)$$

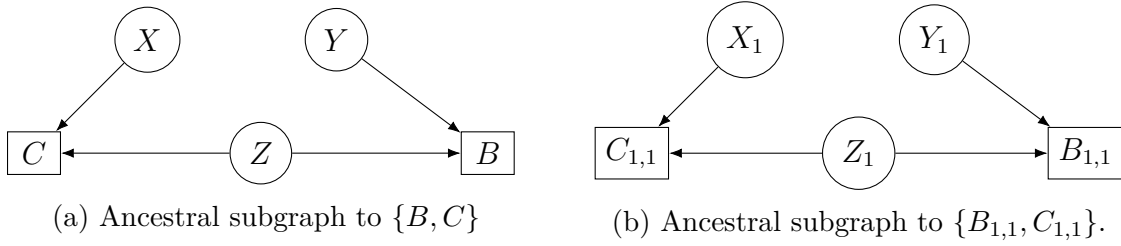


Figure 4.3: Ancestral subgraphs in the cut inflation of the classical triangle scenario. We have $\{B_{1,1}, C_{1,1}\} \sim \{B, C\}$ and $\mathcal{G}_{\text{An}_{\mathcal{G}}(\{B_{1,1}, C_{1,1}\})} \sim \mathcal{G}_{\text{An}_{\mathcal{G}'}(\{B, C\})}$ so that $\{B_{1,1}, C_{1,1}\}$ is an injectable set and $\{B, C\}$ is its image in $\mathcal{G}$. Hence, we can equate $P(B, C) = P(B_{1,1}, C_{1,1})$.

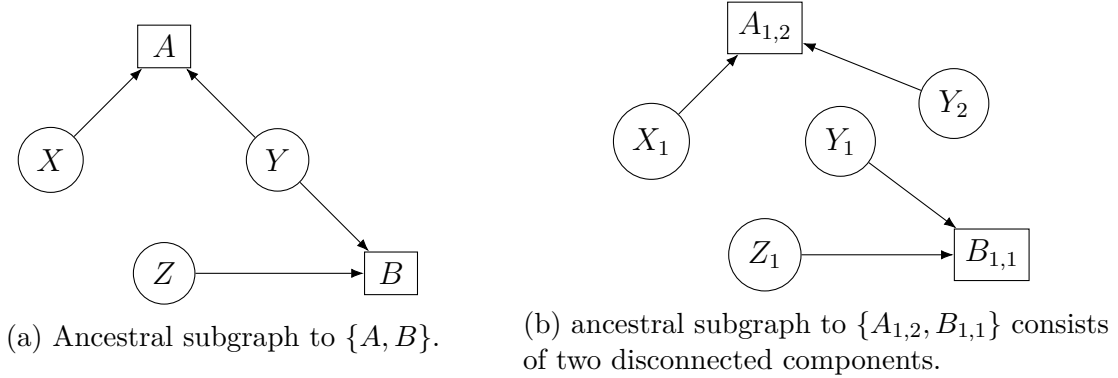


Figure 4.4: Ancestral subgraphs in the cut inflation of the classical triangle scenario. Here $\{A_{1,2}, B_{1,1}\} \sim \{A, B\}$ but $\mathcal{G}_{\text{An}_{\mathcal{G}}(\{A, B\})} \not\sim \mathcal{G}_{\text{An}_{\mathcal{G}'}(\{A_{1,2}, B_{1,1}\})}$. Hence, $\{A_{1,2}, B_{1,1}\}$ is not an injectable set. However, it is an ai-expressible set and allows for factorisation into disjunct injectable sets $\{A_{1,2}\} \sim \{A\}$ and $\{B_{1,1}\} \sim \{B\}$. Hence, we can equate $P(A_{1,2}, B_{1,1}) = P(A_{1,2})P(B_{1,1}) = P(A)P(B)$

with injectable sets $\mathcal{V}_{\text{in}}^j$ such that $\text{An}_{\mathcal{G}'}(\mathcal{V}_{\text{in}}^k) \cap \text{An}_{\mathcal{G}'}(\mathcal{V}_{\text{in}}^l) = \emptyset \forall k, l \neq k$. For $\mathcal{S}'$ an ai-expressible set, $\mathcal{G}_{\text{An}_{\mathcal{G}'}(\mathcal{S}')}$ will consist of disconnected components each representing one injectable set. As an example consider [Figure 4.4](#) and [Figure 4.5](#). There we can exploit the ancestral independence of copies of A, B and C in the inflated models of the classical triangle scenario. This was not possible in the original model.

For any probability distribution compatible with $\mathcal{G}$, its marginals over subsets of the observed nodes in $\mathcal{G}$ are compatible with $\mathcal{G}$. By the above, we can identify the corresponding injectable and ai-expressible sets in $\mathcal{G}'$ and assign them a marginal probability distribution or a product thereof. We then require that these marginals over injectable sets and the product of marginals over ai-expressible sets are compatible with $\mathcal{G}'$. That is, whether they can be obtained via marginalisation on a joint probability distribution on $\mathcal{G}'$. This marginal problem can be cast as a linear programme. Any probability distribution that fails to be compatible with the inflated model is incompatible with

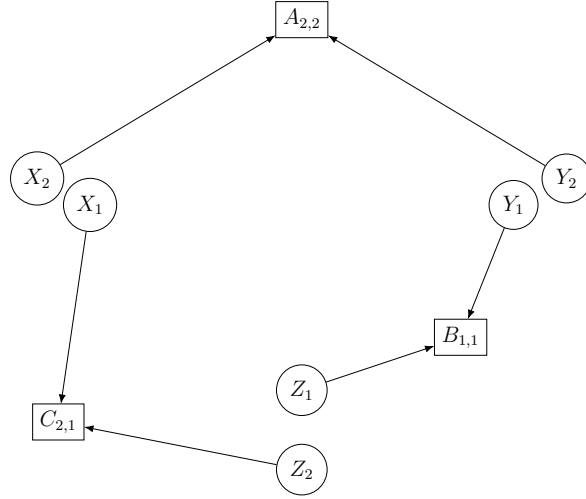


Figure 4.5: Ancestral subgraph to $\{A_{2,2}, B_{1,1}, C_{2,1}\}$ in the webb inflation to the classical triangle scenario. It consists of three disconnected components capturing the ancestral independence between $A_{2,2}$, $B_{1,1}$ and $C_{2,1}$. We can reduce the ai-expressible set $\{A_{2,2}, B_{1,1}, C_{2,1}\}$ to three disjoint injectable sets $\{A_{2,2}\}$, $\{B_{1,1}\}$ and $\{C_{2,1}\}$ with their image in the original structure $\{A\}$, $\{B\}$ and $\{C\}$. Hence, $P(A_{2,2}, B_{1,1}, C_{2,1}) = P(A_{2,2})P(B_{1,1})P(C_{2,1}) = P(A)P(B)P(C)$.

the original model. In practice, we obtain a more robust set of necessary conditions on any compatible distribution in the original model via the ancestral independencies in the inflated model.

Remark. We have limited ourselves to obtaining these necessary constraints from ancestral independencies in the inflated causal models. Analogously, we can extract necessary conditions from more general d -separations appearing in the inflated graphs.

Given a causal model M , we can construct a hierarchy of inflations $M'_1, \dots, M'_n$. Each M_i contains M_j with $j = 1, \dots, i - 1$, such that the causal structures $\mathcal{G}_j$ to M_j are subgraphs of the causal structure $\mathcal{G}_i$ to M_i . Iteratively increasing the number of copies of nodes in the inflated structure allows us to identify more and more injectable and ai-expressible sets. At each step, the necessary constraints on any compatible distribution become stronger and stronger and thus result in a hierarchy of necessary conditions. In a seminal contribution, [56] proved this hierarchy to be complete. The error of approximating the CCP via linear programmes at each step of the hierarchy asymptotically tends to zero as one ascends the hierarchy. Any incompatible distribution will be characterised at some step in the inflation hierarchy. At each step k in the inflation hierarchy, we obtain an *outer approximation* of the set of compatible distributions to a given classical causal model. It is an outer approximation in the sense that for a given k , a probability distribution $\tilde{P}$ might adhere to the constraints imposed on any compatible distribution by the inflated causal model M_k . Still, it might

fail to adhere to the compatibility constraints imposed by M_l with $l > k$. The completeness ensures that this outer approximation converges to the true set of compatible probability distributions relative to a causal model M .

Remark. While, in theory, any incompatible distribution will eventually be detected, it is not at all ensured that obtaining an infeasible certificate is computationally viable.

In summary, the classical inflation technique constitutes a linear programming hierarchy which completely solves the CCP.

4.2 Quantum inflation

Again, we start by assuming that there is a quantum causal model compatible with a given joint distribution P over the observed nodes. As in the classical case, we construct inflations of the causal model. We then translate the compatibility with the original quantum causal structure to necessary requirements on distributions to be compatible with this inflated model. As in the classical variant, non-linear and non-convex independence relations are relaxed to linear symmetry relations such that they can be imposed in convex optimisation procedures. However, the no-cloning theorem requires us to slightly modify the notion of inflation from the classical to the quantum variant. Informally, we only copy the latent nodes representing quantum systems in an inflation of a quantum causal structure. Correspondingly, we introduce copies of operators mapping the copied quantum states to classical random variables. We do not (explicitly) introduce copies of the observed classical random variables.

Consider a quantum causal structure¹¹ $\mathcal{G}$. Let us denote the proper subset of quantum nodes by $\mathcal{Q} \subset V(\mathcal{G})$ and the observed classical nodes by $\mathcal{O} = V(\mathcal{G}) \setminus \mathcal{Q}$. A *quantum inflation* of $\mathcal{G}$ is a graph homomorphism to a DAG $\mathcal{G}'$, such that $\forall v \in \mathcal{O}$

$$v \in V(\mathcal{G}') \quad \wedge \quad \text{Ch}_{\mathcal{G}}(v) = \text{Ch}_{\mathcal{G}'}(v) \quad (4.6)$$

and $\forall \varrho \in \mathcal{Q}$ there exist $\varrho^i \in V(\mathcal{G}')$ with $i \in \mathbb{N}_+$ copy-index equivalent quantum nodes with

$$\text{Ch}_{\mathcal{G}}(\varrho) = \text{Ch}_{\mathcal{G}'}(\varrho^i) \quad \wedge \quad \text{Pa}_{\mathcal{G}'}(\varrho^i) = \emptyset. \quad (4.7)$$

We denote this copy-index equivalence as $\varrho \sim \varrho^i$ and choose $\varrho =: \varrho^1$. Thus, $\mathcal{G}$ is a subgraph¹² of $\mathcal{G}'$. Copy-equivalent quantum states are mutually independent. The characteristics of the quantum nodes are preserved under the inflation. Let n be the maximal copy-index arising in $\mathcal{G}'$. We refer to $\mathcal{G}'$ as a *level n inflation* of $\mathcal{G}$. If $\forall \varrho \in \mathcal{Q}$ the maximal copy-index is n , we call it a *(full) level n inflation*, else a *partial level n inflation*. As an example consider [Figure 4.6](#) and [Figure 4.7](#). For easier notation, let us restrict to full level n inflations and denote the corresponding inflated DAG by $\mathcal{G}^n$.

Remark. We denote the quantum triangle scenario's full level n inflation by Δ_n .

¹¹Recall that we restrict to correlation scenarios.

¹²This was not a necessary condition in the classical inflation.

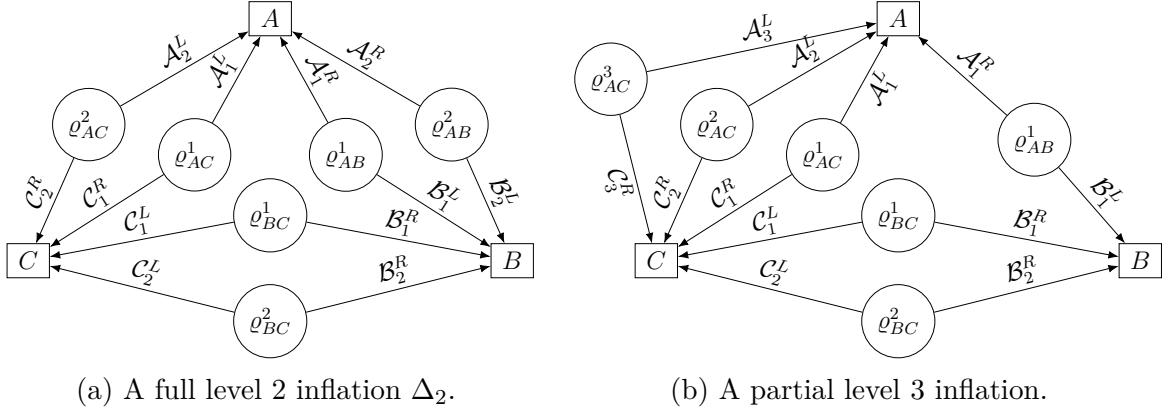


Figure 4.6: Full and partial inflations of the quantum triangle scenario.

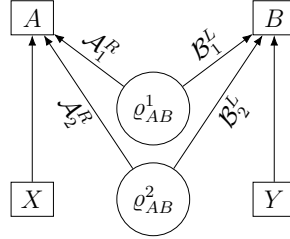


Figure 4.7: Level 2 inflation of the Bell scenario.

4.2.1 Automorphisms of inflated quantum structures

Let us now formalise the symmetries in $\mathcal{G}^n$, which result from the independencies dictated by the structure of $\mathcal{G}$. We are interested in the automorphisms of $\mathcal{G}^n$, which arise from permutations of copy-index equivalent sets. To each copy-equivalent set

$$I_\varrho = \{\varrho^i \in V(\mathcal{G}^n) : \varrho^i \sim \varrho\} \quad (4.8)$$

we can define a symmetric group S_{I_ϱ} . The elements in S_{I_ϱ} are all the permutations on I_ϱ and the group operation is given by the composition of these permutations¹³

Remark. If we consider the set of indices to any I_ϱ , then we can analogously to S_{I_ϱ} define $S_{|I_\varrho|}$ acting on that index set. To every $\sigma \in S_{I_\varrho}$ there exists a $\pi \in S_{|I_\varrho|}$ such that for any $\varrho^i \in I_\varrho$ with $i = 1, \dots, n$

$$\sigma(\varrho^i) = \varrho^{\pi(i)}. \quad (4.9)$$

Any copy-equivalent set I_ϱ can be identified as the orbit under the action of S_{I_ϱ} ,

¹³We will often abstain from explicitly indicating the group operation.

i.e. for every $\varrho \in V(\mathcal{G}^n)$

$$I_\varrho = S_{I_\varrho} \cdot \varrho = \{\sigma(\varrho) : \sigma \in S_{I_\varrho}\}. \quad (4.10)$$

We can then define graph automorphisms of $\mathcal{G}^n$ from these symmetric groups. Denote the automorphism group to $\mathcal{G}^n$ by $\text{Aut}_{\mathcal{G}^n}$. We want to select only those automorphisms corresponding to the above state permutations and which act trivially on classical random variables in $\mathcal{G}^n$. Let $\psi \in \text{Aut}_{\mathcal{G}^n}$ be such that $\forall \varrho \in I_\varrho$ there exists $\sigma \in S_{I_\varrho}$ so that

$$\psi(\varrho) = \sigma(\varrho). \quad (4.11)$$

In other words, ψ is a graph automorphism that can be expressed in state permutations from S_{I_ϱ} . We can equivalently consider such automorphism, which acts non-trivially on another copy equivalent state set in $\mathcal{G}^n$. Furthermore, any combination of those automorphisms again defines an automorphism of interest to us. These then permute the elements of multiple copy-equivalent state sets preserving the corresponding sets of descendants. Let us collect all $\psi \in \text{Aut}_{\mathcal{G}^n}$ fulfilling the above condition in a set

$$\Psi = \{\psi \in \text{Aut}_{\mathcal{G}^n} : \forall \varrho \in V(\mathcal{G}), \forall \tilde{\varrho} \in I_\varrho \exists \sigma \in S_{I_\varrho} \text{ with } \psi(\tilde{\varrho}) = \sigma(\tilde{\varrho})\}. \quad (4.12)$$

Now consider the group generated by Ψ , the *transformation group* $\text{Trans}_{\mathcal{G}^n} = \langle \Psi \rangle$. It is a proper subgroup of $\text{Aut}_{\mathcal{G}^n}$. As an example consider [Figure 4.8](#).

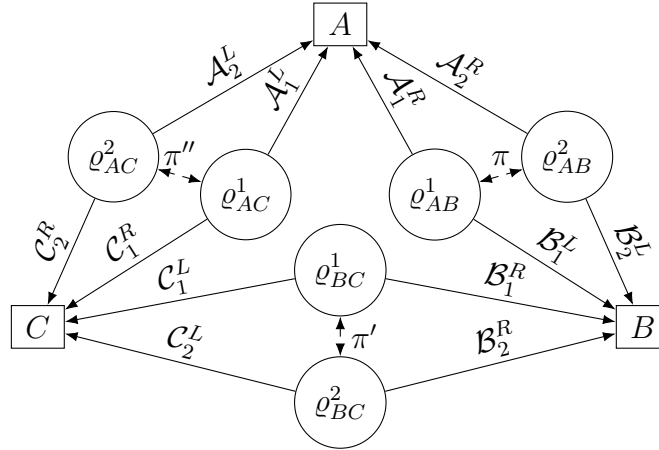


Figure 4.8: The Δ_2 contains the copy-index equivalent sets $I_{\varrho_{AC}} = \{\varrho_{AC}^1, \varrho_{AC}^2\}$, $I_{\varrho_{AB}} = \{\varrho_{AB}^1, \varrho_{AB}^2\}$ and $I_{\varrho_{BC}} = \{\varrho_{BC}^1, \varrho_{BC}^2\}$. To each, we can define a symmetric group. These are the cyclic groups $S_{\varrho_{AB}} = \langle \pi \rangle$, $S_{\varrho_{BC}} = \langle \pi' \rangle$ and $S_{\varrho_{AC}} = \langle \pi'' \rangle$. $\text{Trans}_{\mathcal{G}^n}$ is then any $\psi \in \text{Aut}_{\mathcal{G}^n}$ which acts on any ϱ_{AB}^i , $i = 1, 2$ as a permutation from $S_{\varrho_{AB}}$ and analogously with $\varrho_{AC}^i, \varrho_{BC}^i$. For any $\psi \in \text{Trans}_{\mathcal{G}^n}$ $\psi(A) = A, \psi(B) = B, \psi(C) = C$.

Remark. The more independent states the original causal structure contains, the more symmetry the inflated structure will exhibit. To each set of copy equivalent states, we assign a symmetry operation. The more such sets exist, the more symmetry operations we can formulate. Additionally, the higher the inflation level n , the larger the cardinality of those sets of copy equivalent states; thus, the more symmetric the inflated structure.

4.2.2 Quantum realisations of inflated quantum structures

Since we ultimately want to cast the QCCP as an NPO problem, we need to define the generators Σ^n and relations $\mathcal{R}^n$ yielding a universal C^* -algebra $\mathcal{D}^n$ in a quantum realisation to the level n inflation $\mathcal{G}^n$. The generators¹⁴ correspond to POVM elements indexed in the local commuting subalgebras which generate them. We collect these subalgebras in $\mathbf{s}$. Again we index the POVM elements in their outcome value i . Under the action of a state $\varrho \in S(\mathcal{D}^n)$ we assign i to a classical random variable k . We allow for an index in the value m a variable assumes. Additionally, we require generators in a quantum realisation of $\mathcal{G}^n$ to possess the same characteristics as the corresponding generators in a quantum realisation of $\mathcal{G}$. Any constraints we impose on POVM elements to $\mathcal{D}$ in $\mathcal{G}$ must also be satisfied by their counterparts in $\mathcal{G}^n$. We adopt the notation introduced in [87] and denote such a generator to $\mathcal{D}^n$ by

$$\hat{O}_{i|m}^{s|k} := \hat{O}(\text{outcome variable} = k, \text{subalgebras} = \mathbf{s}, \text{setting} = m, \text{outcome} = i). \quad (4.13)$$

A priori, we do not restrict the dimensions of the universal C^* -algebra involved. Rather than dealing with POVMs, by the *Naimark's dilation theorem* [61] we can consider a complete set of projectors. Hence, any projector needs to fulfill

$$\begin{aligned} \text{Hermiticity:} \quad & \hat{O}_{i|m}^{s|k} = \left(\hat{O}_{i|m}^{s|k} \right)^\dagger \quad \forall \mathbf{s}, k, i, m \\ \text{Projectiveness:} \quad & \hat{O}_{i|m}^{s|k} \hat{O}_{i'|m}^{s|k} = \delta_{ii'} \hat{O}_{i|m}^{s|k} \quad \forall \mathbf{s}, k, i, i', m \\ \text{Completeness:} \quad & \sum_i \hat{O}_{i|m}^{s|k} = \mathbb{1} \quad \forall \mathbf{s}, k, m. \end{aligned} \quad (4.14)$$

As for the original structure $\mathcal{G}$, the generators in a quantum realisation to $\mathcal{G}^n$ adhere to several commutation relations. Generators as elements of distinct commuting subalgebras commute. Hence, any two generators $\hat{O}_{i_1|m_1}^{s_1|k_1}, \hat{O}_{i_2|m_2}^{s_2|k_2}$ in a quantum realisation

¹⁴Without explicitly mentioning, we always assume the identity operator to be an element of the generators.

to $\mathcal{G}^n$ fulfill

$$[\hat{O}_{i_1|m_1}^{s_1|k_1}, \hat{O}_{i_2|m_2}^{s_2|k_2}] = 0 \quad \text{if } s_1 \cap s_2 = \emptyset. \quad (4.15)$$

Example 4.1. In Δ_2 a universal C^* -algebra $\mathcal{D}^2$ will be generated by projectors

$$E_a^{i,j} := \hat{O}_{a|\emptyset}^{\{A_i^L, A_j^R\}|A}, \quad F_b^{i,j} := \hat{O}_{b|\emptyset}^{\{B_i^L, B_j^R\}|B}, \quad G_c^{i,j} := \hat{O}_{c|\emptyset}^{\{C_i^L, C_j^R\}|C} \quad (4.16)$$

with $i, j = 1, 2$. They adhere to the commutation relations¹⁵

$$[H_-^{i,j}, H_-^{k,l}] = 0 \quad \text{for } i, j, k, l = 1, 2, \quad i \neq k, j \neq l \quad (4.17)$$

where $H = E, F, G$ and $[E, F] = [E, G] = [F, G] = 0$ for any indices. We neglected the outcome variables as they do not restrict commutation. We also add the unit operator to the generators and constrain it to commute with any other generator.

4.2.3 Symmetry invariance

We need to define the automorphic symmetry of $\mathcal{G}^n$ in terms of elements from $\mathcal{D}^n$. Analogously to moving between the Schrödinger picture and the Heisenberg picture in elementary quantum theory, we can translate the symmetry over states to symmetry operations on operators. It suffices to define these permutations on the generators of $\mathcal{D}^n$. These are indexed in the subalgebras and thus in the states they correspond to. We can therefore represent the action of $\text{Trans}_{\mathcal{G}^n}$ on operators by defining corresponding permutations acting on the algebra indices of the generators. Let $\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n}$ be the group of permutations corresponding to the automorphisms in $\text{Trans}_{\mathcal{G}^n}$ with an action defined on the subalgebra indices of generators. Under the action of $\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n}$, Σ^n will be partitioned into equivalence classes corresponding to the orbits

$$\text{Trans}_{\mathcal{G}^n} \cdot g, \quad \forall g \in \Sigma^n. \quad (4.18)$$

The action of $\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n}$ naturally extents to $\mathcal{D}^n$, thus partitioning $\mathcal{D}^n$ into orbits

$$\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n} \cdot x, \quad \forall x \in \mathcal{D}^n. \quad (4.19)$$

Remark. Fundamentally, $\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n}$ describes the same symmetries on generators as in [87, 46].

Example 4.2. In Δ_2 , a permutation of the state ϱ_{AB}^1 to ϱ_{AB}^2 under π corresponds to

¹⁵The bar indicates the independence of commutation relations on the outcome values.

$\mathcal{A}_1^R \leftrightarrow \mathcal{A}_2^R$ and $\mathcal{B}_1^L \leftrightarrow \mathcal{B}_2^L$. In terms of operators, this means

$$E_a^{i,j} \leftrightarrow E_a^{i,\pi(j)}, \quad F_b^{i,j} \leftrightarrow F_b^{\pi(i),j}. \quad (4.20)$$

The orbit to $E_a^{1,1}$ under $\text{Trans}_{\mathcal{D}^2, \Delta_2} = \langle \pi, \pi', \pi'' \rangle$ is

$$\text{Trans}_{\mathcal{D}^2, \Delta_2} \cdot E_a^{1,1} = \{E_a^{1,1}, E_a^{1,2}, E_a^{2,1}, E_a^{2,2}\}. \quad (4.21)$$

Fundamentally, isomorphic graphical structures to $\mathcal{G}^n$ correspond to an equivalent causal hypothesis. Thus the graphs in the orbit to $\mathcal{G}^n$ under the action of $\text{Trans}_{\mathcal{G}^n}$ constitute the same causal hypothesis. In the NPO formulation of the QCCP, we encode this symmetry into equality constraints on monomials in an orbit to $\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n}$. Concretely, any compatible state ϱ will evaluate the elements of said orbits to the same value. For any $x \in \mathcal{D}^n$ consider any $u, v \in \text{Trans}_{\mathcal{D}^n, \mathcal{G}^n} \cdot x$ then,

$$\varrho(u - v) = 0 \quad (4.22)$$

with $\varrho \in S(\mathcal{D}^n)$.

Example 4.3. In Δ_2 consider as an example

$$\begin{aligned} \varrho(E_a^{1,1} F_b^{1,2} F_{b'}^{2,1} G_c^{2,1}) &\stackrel{\varrho_{AB}^1 \leftrightarrow \varrho_{AB}^2}{=} \varrho(E_a^{1,2} F_b^{2,2} F_{b'}^{1,1} G_c^{2,1}) \stackrel{\varrho_{BC}^1 \leftrightarrow \varrho_{BC}^2}{=} \varrho(E_a^{1,2} F_b^{2,1} F_{b'}^{1,2} G_c^{1,1}) \\ &\stackrel{\varrho_{AC}^1 \leftrightarrow \varrho_{AC}^2}{=} \varrho(E_a^{2,2} F_b^{2,1} F_{b'}^{1,2} G_c^{1,2}). \end{aligned} \quad (4.23)$$

4.2.4 State factorisation

Let us now address the state factorisation in the inflated model. In a quantum realisation to $\mathcal{G}^n$, we can identify more C^* -independent subalgebras to $\mathcal{D}^n$ for which we require any compatible state $\varrho \in S(\mathcal{D}^n)$ to be a product state. Consider C^* -independent subalgebras $\mathcal{A}_i^L$ and $\mathcal{A}_j^R$ to $\mathcal{D}^n$. A $\varrho \in S(\mathcal{D}^n)$ is a product state over these algebras for indices $i, j = 1, \dots, n$.

Example 4.4. In $\mathcal{D}^1$, we do not require any state $\varrho \in S(\mathcal{D}^1)$ to be a product state with respect to any monomial in the generators Σ^1 and thus to no element in $\mathcal{D}^1$. Recall, the factorisation is with respect to local algebras $\mathcal{O}_L, \mathcal{O}_R$ with $\mathcal{O} \in \{\mathcal{A}, \mathcal{B}, \mathcal{C}\}$. Having introduced copies of these subalgebras in Δ_2 , we can formulate state factorisation over elements in $\mathcal{D}^n$. We can reformulate this factorisation property by introducing

subalgebra restrictions

$$\begin{aligned} \varrho|_{\langle \mathcal{A}_1^L, \mathcal{C}_1^R \rangle} &=: \varrho_{AC}^1 & \varrho|_{\langle \mathcal{A}_1^R, \mathcal{B}_1^L \rangle} &=: \varrho_{AB}^1 & \varrho|_{\langle \mathcal{B}_1^R, \mathcal{C}_1^L \rangle} &=: \varrho_{BC}^1 \\ \varrho|_{\langle \mathcal{A}_2^L, \mathcal{C}_2^R \rangle} &=: \varrho_{AC}^2 & \varrho|_{\langle \mathcal{A}_2^R, \mathcal{B}_2^L \rangle} &=: \varrho_{AB}^2 & \varrho|_{\langle \mathcal{B}_2^R, \mathcal{C}_2^L \rangle} &=: \varrho_{BC}^2 \end{aligned} \quad (4.24)$$

and thus

$$\varrho = \bigotimes_{i=1}^2 \left(\varrho_{AB}^i \otimes \varrho_{AC}^i \otimes \varrho_{BC}^i \right). \quad (4.25)$$

Consider the factorisation in Δ_2 depicted in [Figure 4.9](#).

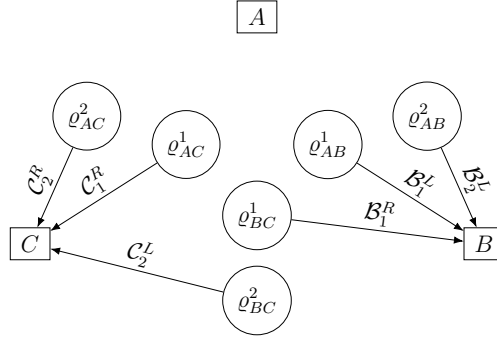


Figure 4.9: State factorisation $\varrho(F_b^{1,1} F_b^{2,1} G_c^{2,2} G_c^{1,1}) = \varrho(F_b^{1,1} F_b^{2,1}) \varrho(G_c^{1,1} G_c^{1,2})$ in Δ_n in its graphical depiction.

Remark. It can be helpful to depict monomials in $\mathcal{D}^n$ graphically. As an example consider [Figure 4.9](#). Unfortunately, we cannot simply identify all algebra elements factorising under the state action with disconnected graphical components in $\mathcal{G}^n$. This did not pose a problem in classical inflation due to our ability to introduce copies of observed nodes. In quantum inflation, states also factorise over certain algebra elements to an observed node. As an example consider [Figure 4.10](#).

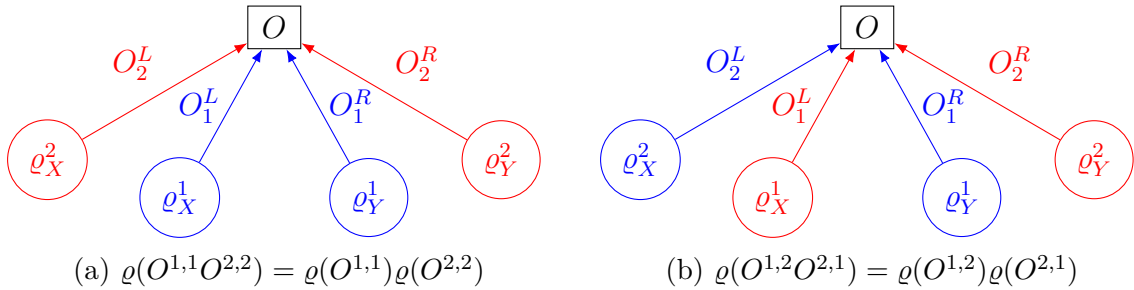


Figure 4.10: State factorisation over algebra elements to an observed node.

4.2.5 Quantum variants of injectable and ai-expressible sets

Again, requiring any compatible $\varrho \in S(\mathcal{D}^n)$ to be a product set over certain elements in $\mathcal{D}^n$ constitutes a non-convex and non-linear constraint. Thus, we relax them to symmetry relations. However, there are situations where we explicitly exploit state factorisation. We will only treat one such situation here and cover others in subsequent chapters. In classical inflation, we were able to decompose ai-expressible sets into ancestrally independent injectable sets. Recall that we could assign a probability to elements of the injectable set as they represented components locally isomorphic to components of the original causal structure, i.e. the image of the injectable set. Rather than considering ancestral independence, in quantum inflation, factorisation of the state over algebra elements corresponds to products of probabilities. We will now construct a quantum analogue of the ai-expressible set to relate a subset of elements from $\mathcal{D}^n$ to a probability distribution being tested. Let $p = |\mathcal{O}|$ be the number of observed nodes in $\mathcal{G}$. We are interested in all monomials up to degree p in generators from $\Sigma^1 \subseteq \Sigma^n$, such that there is at most one generator to each party. We refer to this as the injectable set $\mathcal{I}_{\Sigma^1}$.

Remark. In analogy with classical inflation, $\mathcal{I}_{\Sigma^1}$ corresponds to the “image” of the set

$$\bigcup_{g \in \mathcal{I}_{\Sigma^1}} \text{Trans}_{\mathcal{D}^n, \mathcal{G}^n} \cdot g \quad (4.26)$$

in a quantum realisation to the original quantum causal structure $\mathcal{G}$. Since we explicitly enforce symmetry constraints on all $x \in \mathcal{D}^n$, we can neglect this distinction.

To speak of factorisation, we need to consider a state. Concretely, consider the set $\mathcal{I}_{\Sigma^1}$ under the action of a state $\varrho \in S(\mathcal{D}^n)$, i.e.

$$\mathcal{I}_\varrho = \{\varrho(x) : x \in \mathcal{I}_{\Sigma^1}\}. \quad (4.27)$$

Now consider all products $w(\mathcal{I}_\varrho)$ of elements from $\mathcal{I}_\varrho$ up to some finite order t , which depends on the quantum causal structure $\mathcal{G}^n$ under consideration

$$\tilde{\mathcal{P}}_n(\mathcal{I}_\varrho) = \{w(\mathcal{I}_\varrho) : |w(\mathcal{I}_\varrho)| \leq t\}. \quad (4.28)$$

The set $\tilde{\mathcal{P}}_n(\mathcal{I}_\varrho)$ is our quantum variant to the ai-expressible set in classical inflation. According to the probability distribution being tested, we can assign a probability to each element in this set. Let us denote the preimage of $\tilde{\mathcal{P}}_n(\mathcal{I}_\varrho)$ via

$$\mathcal{P}_n(\mathcal{I}_\varrho) = \{x : \varrho(x) \in \tilde{\mathcal{P}}_n(\mathcal{I}_\varrho)\}. \quad (4.29)$$

Informally, these are all the monomials which factor into components under the action of the state ϱ , such that the monomial w to each component can be mapped via $\text{Trans}_{\mathcal{D}^n, \mathcal{G}^n}$ to $\mathcal{I}_{\Sigma^1}$, i.e. $w \in \bigcup_{g \in \mathcal{I}_{\Sigma^1}} \text{Trans}_{\mathcal{D}^n, \mathcal{G}^n} \cdot g$.

Example 4.5. In Δ_n , we have

$$\mathcal{I}_{\Sigma^1} = \{X_x^{1,1}, X_x^{1,1}Y_y^{1,1}, E_a^{1,1}F_b^{1,1}G_c^{1,1}\}_{X,Y,x,y,a,b,c} \quad (4.30)$$

with $X, Y \in \{E, F, G\}$ such that $X \neq Y$ and according outcomes $x, y, a, b, c \in \{1, \dots, m\}$.

Remark. We often abuse our terminology and refer to $\mathcal{P}_n(\mathcal{I}_\varrho)$ as the ai-expressible set.

Remark. While the injectable set depends on the quantum realisation $\mathcal{D}$ of the original quantum causal structure $\mathcal{G}$, the ai-expressible set depends on the quantum realisation $\mathcal{D}^n$ of the inflation $\mathcal{G}^n$.

Remark. Graphically, we try to identify those algebra elements where we can map all their components to components in $\mathcal{G}$ via graph automorphisms $\text{Trans}_{\mathcal{G}^n}$.

Remark. Whenever a factorisation component hinges on more than one copy-equivalent state, i.e. it is given in terms of projectors indexed in copy equivalent subalgebras, it cannot be mapped to an element of the injectable set via symmetry operations. As an example consider [Figure 4.9](#). Neither component can be mapped to a component solely given in terms of generators from Σ^1 .

Example 4.6. Consider once more Δ_2 . An example of assigning probabilities to state actions over algebra elements is

$$\varrho(E_a^{1,1}F_b^{1,1}G_c^{1,1}) = P(a, b, c), \quad \varrho(E_a^{1,1})\varrho(F_b^{1,1})\varrho(G_c^{1,1}) = P(a)P(b)P(c). \quad (4.31)$$

where the second expression resulted from the state factorisation

$$\varrho(E_a^{1,1}F_b^{2,1}G_c^{2,2}) = \varrho(E_a^{1,1})\varrho(F_b^{2,1})\varrho(G_c^{2,2}) \stackrel{\text{Trans}_{\mathcal{D}^2, \Delta_2}}{=} \varrho(E_a^{1,1})\varrho(F_b^{1,1})\varrho(G_c^{1,1}). \quad (4.32)$$

Remark. We formulate a stronger constraint than the one given in [\[87\]](#) as *identifiable moments*. There, the connection to a joint probability distribution P to be tested is formulated as

$$\varrho \left(\prod_{j=1}^n \prod_k \hat{O}_{i_{j,k}|m_{j,k}}^{\vec{j}_k|k} \right) = \prod_{j=1}^n P \left(\bigcap_k (i_{j,k}|m_{j,k}, k) \right) \quad (4.33)$$

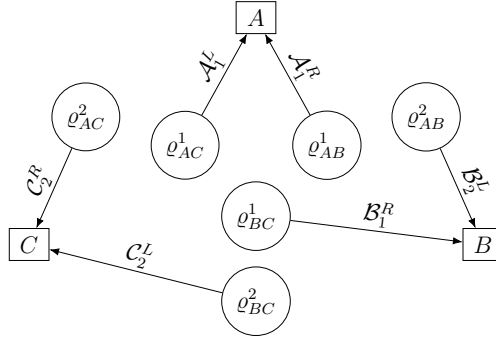


Figure 4.11: State factorisation $\varrho(E_a^{1,1}F_b^{2,1}G_c^{2,2}) = \varrho(E_a^{1,1})\varrho(F_b^{2,1})\varrho(G_c^{2,2}) \stackrel{\text{Trans}_{\mathcal{D}^2, \mathcal{G}^2}}{=} \varrho(E_a^{1,1})\varrho(F_b^{1,1})\varrho(G_c^{1,1}) = P(a)P(b)P(c)$ in Δ_2 in its graphical depiction.

or more concretely in Δ_n

$$\varrho \left(\prod_{i=1}^n E_a^{i,i} F_b^{i,i} G_c^{i,i} \right) = \prod_{i=1}^n P(a^i, b^i, c^i). \quad (4.34)$$

This corresponds to the insight that in any level n inflation $\mathcal{G}^n$ there exist $g \leq n$ -copies of the original quantum structure $\mathcal{G}$. We started this construction by assuming compatibility between a joint probability P and $\mathcal{G}$ and thus need to enforce this by a corresponding constraint. A graphical depiction of [Eq. \(4.33\)](#) for the level $n = 2$ inflation of the quantum triangle scenario is given in [Figure 4.12](#). Here, however, we

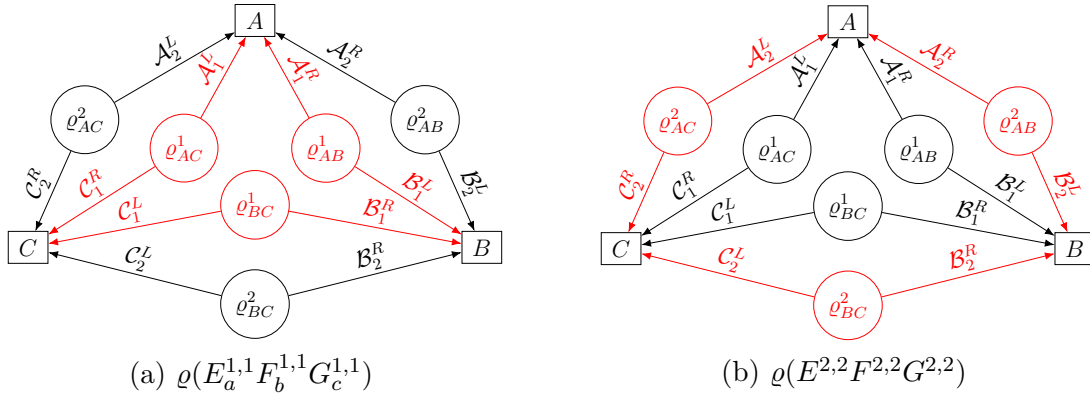


Figure 4.12: Graphical depiction of the identifiable monomials from [\[87\]](#) in Δ_2 . The original quantum triangle scenario as subgraphs of Δ_2 .

consider the factorisation property of any compatible state together with the symmetry. To see that our formulation is more restrictive, consider the three-fold factorisation depicted in [Figure 4.11](#). There is no way to formulate this via [Eq. \(4.33\)](#)

4.2.6 The QCCP to inflated quantum causal models

The QCCP in problem [\[3.4\]](#) has been relaxed to the following problem.

Problem 4.1 (Quantum inflation relaxation of QCCP). *Given an inflation $\mathcal{G}^n$ of a quantum causal structure $\mathcal{G}$. Does a quantum realisation of $\mathcal{G}^n$ that yields a joint probability distribution P over observed nodes exist?*

A quantum realisation to $\mathcal{G}^n$ necessarily fulfills all constraints on operators and states given above. If a probability distribution fails to be compatible with any inflation $\mathcal{G}^n$, then this distribution is also incompatible with the original structure $\mathcal{G}$. However, determining a probability distribution to be compatible with a given inflation does not ensure that this distribution is compatible with the original structure. Hence, as in its classical variant, the quantum inflation procedure provides an outer approximation of the set of compatible distributions to a given quantum causal structure. We can also formulate a hierarchy of such compatibility tests by increasing the level of inflation n . We require,

$$\mathcal{D}^n \subseteq \mathcal{D}^{n'}, \quad \mathcal{G}^n \subset \mathcal{G}^{n'} \quad (4.35)$$

for $n, n' \in \mathbb{N}_+$ such that $n < n'$. Any compatible distribution must necessarily satisfy each step in the hierarchy. Each step in the inflation hierarchy can be cast as an NPO problem. Infeasibility of the corresponding NPO problem implies incompatibility of the distribution being tested with the quantum causal model. Each NPO problem can be recast as a hierarchy of SDP relaxations.

Remark. The completeness of the SDP hierarchy should not be confused with the completeness of the inflation hierarchy. We ensure the completeness of the SDP hierarchy by constructing $\mathcal{D}^n$ from projective generators. In contrast to the classical inflation procedure, the quantum variant in [87] is not proven to be a complete hierarchy. However, the recently introduced LGG hierarchy constitutes a complete hierarchy to tackle the QCCP.

Remark. Rather than considering feasibility tests corresponding to the QCCP, we can also optimise over Bell inequalities or non-linear expressions over elements from the injectable set. In practice, these optimisations are computationally very expensive.

Remark. The quantum inflation procedure can also be exploited to answer the classical CCP by imposing that all operators be commutative with respect to all other operators

$$[\hat{O}_{i_1|_{m_1}}^{s_1|_{k_1}}, \hat{O}_{i_2|_{m_2}}^{s_2|_{k_2}}] = 0 \quad \forall s_j, k_j, i_j, m_j \text{ with } j = 1, 2. \quad (4.36)$$

Remark. The quantum inflations considered here fulfil a property called *fanout*¹⁶ [86, p.28]. This states that latent nodes have two or more copy-index equivalent children.

¹⁶The terminology originates from the fan-out property of logical gates.

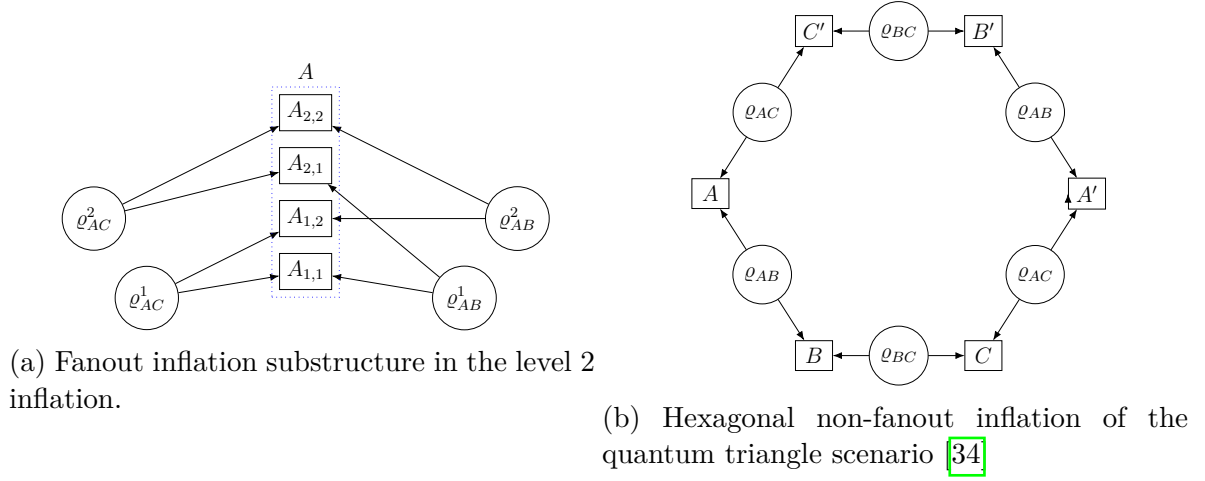


Figure 4.13: Comparison between fanout and non-fanout quantum inflations.

While we do not explicitly denote this substructure depicted in [Figure 4.13a](#), we acknowledge its existence. In comparison, consider the hexagonal non-fanout inflation of the quantum triangle scenario depicted in [Figure 4.13b](#). Constraints arising from non-fanout inflations are much easier to treat and hold for *generalised probabilistic theory* sources [86].

In [section 6](#) we will explicitly construct the NPO problem and its SDP relaxation to Δ_n .

4.3 Ligthart-Gachechiladze-Gross hierarchy

We want to briefly mention the LGG hierarchy as it poses the only provably convergent inflation hierarchy to date. The authors of [46] argue that a quantum realisation with universal C^* -algebra $\mathcal{D}$ to the composite system, even if compatible with a given distribution at all inflation levels, generally does not contain commuting subalgebras $\mathcal{A}_i^L, \mathcal{A}_i^R$ to local subsystems, such that

1. $\mathcal{A}_i^L, \mathcal{A}_i^R$ generate an algebra $\mathcal{A}_{i,i}$ containing the algebra of observed POVMs $\langle \{E_a^{i,i}\}_a \rangle$ as a subalgebra.
2. $\varrho \in S(\mathcal{D})$ is a product state over $\mathcal{A}_i^L, \mathcal{A}_i^R$.

To salvage the quantum inflation idea despite these shortcomings, introduce local operators $e_L^i(a, \alpha)e_R^j(a, \alpha)$ such that [46, II.E.]

$$\begin{aligned} \mathcal{A}_i^L &= \langle \{e_L^i(a, \alpha)\}_{a, \alpha} \rangle, & \mathcal{A}_i^R &= \langle \{e_R^i(a, \alpha)\}_{a, \alpha} \rangle, & \mathcal{A}_{i,j} &= \langle \mathcal{A}_i^L \cdot \mathcal{A}_j^R \rangle, \\ E_a^{i,j} &:= \sum_{\alpha=1}^r e_L^i(a, \alpha)e_R^j(a, \alpha). \end{aligned} \quad (4.37)$$

The LGG inflation hierarchy then builds on these local operators. For a complete description of the LGG hierarchy, see [46, IV.A.]. While we require $\{E_a^{i,j}\}_{i,j,a}$ to be POVM's and thus be bounded, we can not generally assume that the local operators are bounded. For a discussion on their unboundedness, see [46, A.II.2].

Remark. The LGG hierarchy is computationally more expensive than the original quantum inflation hierarchy. There are more operators. Additionally, we require localising matrices to enforce norm bounds on the local operators to allow for NPO theory application.

5 Scalar Extension

The scalar extension procedure was first introduced in [69]. It provided an early framework for studying complex causal networks within the NPA hierarchy. In contrast to inflation procedures where independence is relaxed to symmetries, scalar extension relaxes independence, more specifically, conditional independence relations very differently. Both approaches are compatible with semidefinite programming and thus of relevance to this thesis. Whereas quantum inflation allows for the treatment of arbitrary causal structures, the scalar extension procedure is limited to the subset of causal structures with causally independent parties¹⁷. In the implementation, the generality of quantum inflation corresponds to a higher computational cost than the scalar extension procedure. Nevertheless, for specific scenarios, scalar extension yields tighter bounds.

So far, the two frameworks are considered alternatives. In this section, we generalise the scalar extension idea from [69] to apply it to quantum inflation. We argue that scalar extension can be applied to more general quantum causal structures when considering inflations. In other words, we combine the two procedures. Via inflations, we can formalise state factorisation in quantum realisations of said inflations, even when we could not do so in the original structure. To some of these factorisations, we can assign a product over probabilities and thus have the means to incorporate them in our SDP. However, this only works if all factorisation components can be mapped to elements from the injectable sets via symmetry operations. Practice shows that at finite steps in the inflation hierarchy, there are algebra elements where this fails. The scalar extension allows us to relax these non-convex and non-linear constraints such that they can also be formulated as additional constraints in the resulting SDP. We hope to be able to formulate more restricted SDP relaxations at any given level in the quantum inflation hierarchy.

This section introduces the fundamental notion behind scalar extension and generalises it to the quantum inflation formalism. In the subsequent chapter, we will apply this generalisation to Δ_n .

¹⁷A proper subset of parties involved are causally independent. All parties being causally independent constitutes a trivial causal scenario.

5.1 Scalar extension for quantum inflation

Scalar extension happens at the level of the SDP hierarchy. The fundamental idea behind the scalar extension procedure is the following. Consider an SDP $\mathbf{R}^{(k)}$ of the form [Eq. \(2.90\)](#) with moment matrix $\Gamma^{(k)}$. Assume that the corresponding NPO problem $\mathbf{P}$ arising from quantum inflation dictates non-linear and non-convex factorisation constraints on any compatible state $\varrho \in S(\mathcal{D}^n)$ with $\mathcal{D}^n$ the corresponding universal C^* -algebra. In the corresponding hierarchy of SDP relaxations indexed in k , we formulate a requirement on a positive normalised functional $L_y \in (\mathcal{V}^{(2k)})^*$ from the dual space of the subspace of the free $*$ -algebra with monomial basis $\mathcal{B}_{2k}$. Equivalently, in terms of entries of the moment matrix $\Gamma^{(k)}$

$$\Gamma_{i,j}^{(k)} - \Gamma_{a,b}^{(k)} \Gamma_{c,d}^{(k)} = 0, \quad i, j, a, b, c, d \in \{1, \dots, n = |\mathcal{B}_k|\}, \quad (5.1)$$

with the indices such that they describe three distinct entries of $\Gamma^{(k)}$. As this constitutes a non-convex constraint, we necessarily need to relax this requirement on L_y by omitting such constraints. Scalar extension allows us to formulate a convex relaxation of [Eq. \(5.1\)](#), which we then can impose as an additional constraint in the resulting SDP. We construct from $\mathbf{R}^{(k)}$ a new SDP $\mathbf{R}_{\text{SE}}^{(k)}$ by extending the set of generating monomials $\mathcal{B}_k$ by $l \geq 1$ monomials and thus obtaining a larger moment matrix $\tilde{\Gamma}^{(k)}$ with $\Gamma^{(k)}$ as a principal submatrix. $\mathbf{R}_{\text{SE}}^{(k)}$ inherits all constraints from $\mathbf{R}^{(k)}$. The indexing in $\tilde{\Gamma}^{(k)}$ is

$$\Gamma_{i,j}^{(k)} = \tilde{\Gamma}_{i,j}^{(k)} \quad i, j = 1, \dots, n. \quad (5.2)$$

We then equate some of the entries $\tilde{\Gamma}_{n+i,n+j}^{(k)}$ for $i, j = 1, \dots, l$ to entries of Γ . Due to the hermiticity (symmetry) of $\tilde{\Gamma}^{(k)}$, it suffices to formulate these equalities in terms of elements of the upper triangular form

$$\Gamma_{i,j}^{(k)} - \tilde{\Gamma}_{n+1,e}^{(k)} = 0 \quad e \in \{1, \dots, n+1\}. \quad (5.3)$$

We then require $\tilde{\Gamma}^{(k)}$ to be psd, and the corresponding SDP contains additional linear constraints corresponding to [Eq. \(5.3\)](#). Requiring $\tilde{\Gamma}^{(k)}$ to be psd implies the principal submatrix $\Gamma^{(k)}$ to be psd.

We specify how to adapt the method when the factorisation is given in terms of components which do not correspond to elements from $\mathcal{B}_k$. We also generalise the procedure to introduce relaxations of factorisation into more than two components.

First, let us restrict to cases where the factorisation degrees are limited to $f = 2$

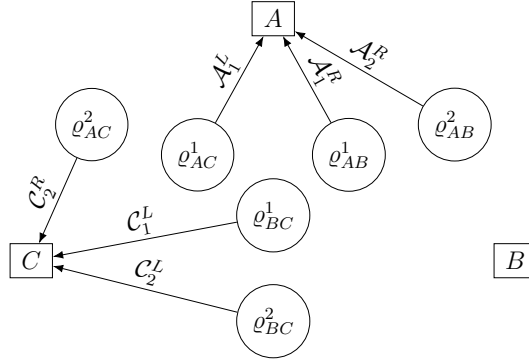


Figure 5.2: Factorisation eligible for scalar extension in Δ_2 .

Remark. In practice, we will encounter several such algebra elements at a given level of the quantum inflation hierarchy (see [Figure 4.9](#) for another example in Δ_2).

In $\mathbf{R}^{(k)}$ we require a positive normalised functional $L_y \in (\mathcal{V}^{(2k)})^*$ to also factorise over these elements, if $u^*, v \in \mathcal{V}^{(2k)}$. Assume that $u^*, v \in \mathcal{B}_k$ are generating monomials to $\Gamma^{(k)}$.

Remark. The involution $*$ preserves the degree of $x \in \mathcal{V}^{(2k)}$ it acts on. Thus, for $u \in \mathcal{B}_k$ either $u = u^*$ or $u \neq u^*$ and then $u^* \in \mathcal{B}_k$. We will show in subsequent sections that we can consider cases where $u^* = u$ for all $u \in \mathcal{B}_k$.

In the moment matrix

$$\Gamma^{(k)} = \begin{matrix} & \mathbb{1} & \dots & u^* & \dots & v & \dots \\ \begin{matrix} \mathbb{1} \\ \vdots \\ u^* \\ \vdots \\ v \\ \vdots \end{matrix} & \left(\begin{array}{cccccc} & & & & & \\ & & y_{u^*} & & y_v & \\ & & & & & \\ & & & & y_{u^*v} & \\ & & & & & \end{array} \right) \end{matrix} \quad (5.6)$$

the object of interest is

$$y_{u^*v} = L_y(u^*v) = L_y(u^*)L_y(v) = y_{u^*} \cdot y_v \quad (5.7)$$

corresponding to a non-linear and non-convex constraint incompatible with the SDP framework. We extend $\mathcal{B}_k$ with a *scalar extension variable* $g \in \mathcal{V}^{(2k)}$ mimicking a moment (scalar) multiplied by the identity. We require that $\forall w \in \mathcal{V}^{(2k)}$

$$[g, w] = 0, \quad L_y(gw) = L_y(g)L_y(w) \quad (5.8)$$

holds. Thus, we relax the requirement on g to be a scalar multiple of the identity to only commutativity and factorisation regarding any other algebra element. The identity is getting absorbed by any other operator

$$\mathbb{1}w = w\mathbb{1} = w, \quad \forall w \in \mathcal{V}^{(2k)}, \quad (5.9)$$

whereas g does not. Let $\tilde{g} = \alpha\mathbb{1} \in \mathcal{V}^{(2k)}$, with $\alpha \in \mathbb{K}$ be a scalar multiple of the identity and $w \in \mathcal{V}^{(2k)}$. Then there $\exists \beta \in \mathbb{K}$ such that

$$\beta\tilde{g}w = w, \quad \text{with } \alpha \cdot \beta = 1 \quad (5.10)$$

holds. However, we do not impose such a constraint on g .

Remark. Whenever we refer to an element from $\mathcal{V}^{(2k)}$ as a scalar variable, we require it to adhere to the above conditions.

The extended moment matrix $\tilde{\Gamma}^{(k)}$ is given by

$$\tilde{\Gamma}^{(k)} = \begin{matrix} & \mathbb{1} & \cdots & u^* & \cdots & v & \cdots & g \\ \begin{matrix} \mathbb{1} \\ \vdots \\ u^* \\ \vdots \\ v \\ \vdots \\ g^* \end{matrix} & \left(\begin{array}{cccccc} & & & & & \\ & & y_u^* & & y_v & \\ & & & & & y_g \\ & & & & y_{u^*v} & y_{u^*g} \\ & & & & & y_{vg} \\ & & & & & y_{g^*g} \end{array} \right) \end{matrix} \quad (5.11)$$

with $\Gamma^{(k)}$ as a principal submatrix. We formulate the constraint

$$y_g = L_y(g) = L_y(u) = y_{u^*} \quad (5.12)$$

resulting in

$$y_{vg} = L_y(v) \stackrel{\text{Eq. (5.8)}}{=} L_y(v)L_y(g) \stackrel{\text{Eq. (5.12)}}{=} L_y(v)L_y(u^*) = y_v y_{u^*} = y_{u^*v} \quad (5.13)$$

which constitutes a linear and convex relaxation of [Eq. \(5.7\)](#). Additionally,

$$y_{ug} = L_y(u)L_y(g) \stackrel{\text{Eq. (5.12)}}{=} L_y(g^*)L_y(g) = y_{g^*g} \geq 0. \quad (5.14)$$

Remark. We omitt enforcing the non-linear and non-convex constraint $y_{u^*g} = |y_{u^*}|^2$.

Now that we have introduced the notion behind the scalar extension procedure let us consider a generalisation where we do not assume $u, v \in \mathcal{V}^{(k)}$.

Remark. Recall that $\Gamma^{(k)}$ is indexed in $\mathcal{B}_k$ but contains the moments to $\mathcal{B}_{2k}$.

In the case of 2-factorisation, we face three scenarios.

Proposition 5.1 (Cardinality 2-factorisation extension set). *Consider $w, u, v \in \mathcal{V}^{(2k)}$ such that for any $L_y \in (\mathcal{V}^{(2k)})^*$ we have $L_y(w) = L_y(u)L_y(v)$ and no further factorisation is possible. Then differentiate between three cases¹⁸*

$$(1): u, v \in \mathcal{B}_k, \quad (2): u \in \mathcal{B}_k, v \notin \mathcal{B}_k \quad (3): u, v \notin \mathcal{B}_k. \quad (5.15)$$

Let us denote the set of scalar extension variables required to formulate a relaxation of the above factorisation condition via the scalar extension procedure by $\mathcal{S}_{L_y(w)}^k$. The indices indicate its dependence on the positive normalised functional, the monomial and the NPA level. Then,

$$|\mathcal{S}_{L_y(w)}^k| = \begin{cases} 2, & \text{for } (3) \\ 1, & \text{for } (1), (2) \end{cases} \quad (5.16)$$

Proof. We have already treated the case (1). Now consider (2). Specifically, we have $\Gamma_{1,h}^{(k)} = y_u$ for $h \in \{2, \dots, n\}$. There necessarily exist $v_1, v_2 \in \mathcal{B}_k$ such that

$$v = v_1^* v_2, \quad y_{v_1} = \Gamma_{1,r}^{(k)}, \quad y_{v_2} = \Gamma_{1,c}^{(k)} \quad (5.17)$$

for $r, c \in \{2, \dots, n\}$ and $r \neq c \neq h$. Then,

$$\Gamma_{i,j}^{(k)} = y_{u^* v} = y_{u^*} y_v = \Gamma_{1,h}^{(k)} \Gamma_{r,c}^{(k)} \quad (5.18)$$

¹⁸Equivalently, (2) can be formulated with $u \notin \mathcal{B}_k$ and $v \in \mathcal{B}_k$.

with $i, j \in \{2, \dots, n\} \setminus \{r, c, h\}$. Extend $\mathcal{B}_k$ by scalar variable $g \in \mathcal{V}^{(2k)}$. Then,

$$\tilde{\Gamma}^{(k)} = \begin{matrix} & \mathbb{1} & \cdots & u & \cdots & v_1 & \cdots & v_2 & \cdots & g \\ \begin{matrix} \mathbb{1} \\ \vdots \\ u^* \\ \vdots \\ v_1^* \\ \vdots \\ v_2^* \\ \vdots \\ g^* \end{matrix} & \left(\begin{array}{cccccccc} & & & & & & & & & y_g \\ & & & & & & & & & y_{u^*g} \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & y_{v_1^*v_2} & & & \\ & & & & & & & & y_{u^*v} & \\ & & & & & & & & & \\ & & & & & & & & & y_{g^*g} \end{array} \right) \end{matrix} \quad (5.19)$$

and formulate the constraint $y_g = y_{v_1^*v_2}$. The relaxation of the factorisation constraint corresponds to $y_{u^*g} = y_{u^*v}$.

Lastly, consider the case (3). In addition to v being a product of v_1^*, v_2 as above there now also necessarily exist $u_1, u_2 \in \mathcal{B}_k$ such that

$$u^* = u_1^*u_2, \quad y_{u_1} = \Gamma_{1,t}^{(k)} \quad y_{u_2} = \Gamma_{1,m}^{(k)} \quad (5.20)$$

with $t, m \in \{2, \dots, n\}$ and $r \neq c \neq t \neq m$. Then,

$$\Gamma_{i,j}^{(k)} = y_{u^*v} = y_{u^*}y_v = \Gamma_{t,m}^{(k)} \cdot \Gamma_{r,c}^{(k)} \quad (5.21)$$

with $i, j \in \{2, \dots, n\} \setminus \{r, c, t, m\}$. Extend $\mathcal{B}_k$ by scalar variables $g_1, g_2 \in \mathcal{V}^{(2k)}$ such that

$$[g_1, g_2] = 0, \quad L_y(g_1g_2) = L_y(g_1)L_y(g_2). \quad (5.22)$$

We have extended $\Gamma^{(k)}$ by two extra columns¹⁹ (and rows) to obtain $\tilde{\Gamma}^{(k)}$ and enforce the constraints $y_{g_1^*} = y_{u^*}$ and $y_{g_2} = y_v$. However,

$$\Gamma_{n+1,n+2}^{(k)} = y_{g_1^*g_2} = y_{u^*v} = \Gamma_{i,j}^{(k)}, \quad (5.23)$$

□

Let us now propose a generalisation to arbitrary finite factorisation degrees. First,

¹⁹For a visualisation see [Appendix B](#)

let us introduce the notion of *partial unpacking*.

Definition 5.1 (Partial unpacking). Consider $w \in \mathcal{B}_{2k}$ such that for any feasible $L_y \in (\mathcal{V}^{(2k)})^*$ we have

$$L_y(w) = \prod_{i=1}^f L_y(w_i) \quad (5.24)$$

with $w_i \in \mathcal{B}_{2k}$ and $f \geq 2$. By partial unpacking, we consider w to only factorising into $t < f$ components by ignoring any further factorisation. Concretely,

$$L_y(w) = L_y\left(\prod_{i=t}^f w_i\right) \prod_{i=1}^{t-1} L_y(w_i) \quad (5.25)$$

corresponds to a partial unpacking into k components. We refer to t as the *partial unpacking index*. Note that this unpacking is not unique. The *partial unpacking set* $\mathcal{P}_{L_y(w)}$ is then the set of sets unpacking a moment into 2-factorisations, i.e. consider moment $abcd$

$$\mathcal{P}_{L_y(abcd)} = \{\{abc, d\}, \{ab, c\}, \{a, b\}\}. \quad (5.26)$$

Example 5.2. In Δ_2 consider $L_y(E_a^{1,1}F_b^{2,1}G_c^{2,2}) = L_y(E_a^{1,1})L_y(F_b^{2,1})L_y(G_c^{2,2})$ factorising into $f = 3$. Choose a partial unpacking with partial unpacking index $t = 2$, then $L_y(E_a^{1,1}F_b^{2,1})L_y(G_c^{2,2})$ or $L_y(E_a^{1,1})L_y(F_b^{2,1}G_c^{2,2})$ represent two possible partial unpackings and we have $\mathcal{P}_{L_y(w)} = \{\{L_y(E_a^{1,1}F_b^{2,1})L_y(G_c^{2,2})\}, \{L_y(E_a^{1,1})L_y(F_b^{2,1})\}\}$.

Remark. Note that the example above does not constitute an example of a moment being eligible for scalar extension as we can identify it with $P(a)P(b)P(c)$. However, it constitutes the only example for 3-factorisation in Δ_3 .

Proposition 5.2. *Requiring any compatible $L_y \in (\mathcal{V}^{(2k)})^*$ to factorise into f components for a given $w \in \mathcal{V}^{(2k)}$ is equivalent to requiring L_y to factorise into $(f - 1)$ 2-factorisations dictated by partial unpacking.*

Proof. For $f = 3$ we have $L_y(w_1w_2w_3) = L_y(w_1)L_y(w_2)L_y(w_3)$. Enforcing the two 2-factorisation constraints $L_y(w_1w_2w_3) = L_y(w_1w_2)L_y(w_3)$ and $L_y(w_1w_2) = L_y(w_1)L_y(w_2)$ is equivalent to the 3-factorisation. For arbitrary finite f , the proof follows by induction. $\square$

Thus, we encounter an f -factorisation with $f \geq 3$ of a moment and like to relax this non-convex constraint via scalar extension. We employ partial unpacking to obtain $(f - 1)$ 2-factorisations. We can then employ the scalar extension procedure for 2-factorisations to enforce a relaxation of the f -factorisation in a given SDP.

6 Application - The Quantum Triangle Scenario

This section serves to explicitly formulate an NPO problem corresponding to a quantum inflation relaxation. Concretely, we tackle the following problem.

Problem 6.1 (The QCCP in Δ_n). *Does there exist a quantum realisation of Δ_n that yields a given joint probability distribution P over observed nodes A, B and C ?*

Subsequently, we introduce its SDP relaxation hierarchy. We refer by $\Delta_{n,k}$ to the n -th inflation level and the corresponding k -th step in the SDP relaxation hierarchy^[20]. We do not need to worry about setting indices cluttering the problem formulation. The concepts in this and the subsequent sections readily generalise to other quantum models.

Let us start by formulating the NPO problem corresponding to $\Delta_{n,k}$. The set of generators is given by

$$\Sigma^n = \{E_a^{i,j}, F_b^{i,j}, G_c^{i,j}\}_{i,j,a,b,c} \quad (6.1)$$

where $i, j = 1, \dots, n$ and $a, b, c \in \{1, \dots, m\}$. We formulate constraints such that for all i, j each $\{E_a^{i,j}\}_a$, $\{F_b^{i,j}\}_b$ and $\{G_c^{i,j}\}_c$ is a complete family of projectors. Additionally, we constrain the elements of Σ^n to commute accordingly to the commuting subalgebra structure in $\Delta_{n,k}$. We denote the universal C^* -algebra to $\Delta_{n,k}$ by $\mathcal{D}^n$. Additionally, we formulate constraints on the state action of any compatible state $\varrho \in S(\mathcal{D}^n)$ over elements from $\mathcal{D}^n$ corresponding to the symmetry and ai-expressible sets in $\Delta_{n,k}$. Recall, the transformation group with an action defined on generators in $\Delta_{n,k}$ was given by $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}$. We denote the corresponding symmetry constraints via

$$\varrho(w - u) = 0, \quad \forall w, u \in \text{Trans}_{\mathcal{D}^n, \Delta_{n,k}} \cdot v, \quad \forall v \in \mathcal{D}^n \quad (6.2)$$

The ai-expressible set is given by $\mathcal{P}_n(\mathcal{I}_\varrho)$ with $\mathcal{I}_{\Sigma^1}$ the injectable set. Let us introduce a function

$$\begin{aligned} f : \mathcal{P}_n(\mathcal{I}_\varrho) \times P &\rightarrow \mathbb{R} \\ (x, P) &\mapsto f(x, P) \end{aligned} \quad (6.3)$$

which assigns to each element in the ai-expressible set a probability corresponding to (the marginals of) P . We then formulate the corresponding constraints via the

²⁰A property, which holds for $n = 2$ and all k will be denoted by $\Delta_{2,k}$, accordingly for $\Delta_{n,1}$ or $\Delta_{n,k}$

preimage of the ai-expressible set

$$\varrho(w) - f(w, P) = 0, \quad \forall w \in \mathcal{P}_n(\mathcal{I}_\varrho). \quad (6.4)$$

Putting all of it together yields problem [6.2](#)

Problem 6.2 (NPO problem to $\Delta_{n,k}$ and P). *Let $X, Y \in \{E, F, G\}$ and $x, y, a, b, c \in \{1, \dots, m\}$, together with $i, j, t, l \in \{1, \dots, n\}$, then*

$$\begin{aligned} & p^* = \min_{\varrho \in S(\mathcal{D}^n)} 1 \\ \text{s.t.} \quad & (X_x^{i,j})^* - X_x^{i,j} = 0, \quad \forall i, j, x \\ & X_x^{i,j} X_y^{i,j} - \delta_{x,y} X_x^{i,j} = 0, \quad \forall i, j, x, y \\ & \sum_x X_x^{i,j} - \mathbb{1} = 0, \quad \forall i, j \\ \mathbf{P}: \quad & [X_x^{i,j}, X_y^{t,l}] = 0, \quad \forall i \neq t, j \neq l, x, y \\ & [X_x^{i,j}, Y_y^{t,l}] = 0, \quad \forall X \neq Y, i, t, j, l, x, y \\ & \varrho(w - u) = 0, \quad \forall w, u \in \text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}, \forall v \in \mathcal{D}^n \cdot v \\ & \varrho(w) - f(w, P) = 0, \quad \forall w \in \mathcal{P}_n(\mathcal{I}_\varrho). \end{aligned} \quad (6.5)$$

Remark. The trivial objective function corresponds to a feasibility problem. For notational purposes, we did not reformulate equalities into two inequalities.

Let us address an obvious redundancy in the above definition of Σ^n . The completeness constraint enforced on any family $\{E_a^{i,j}\}_a$, $\{F_b^{i,j}\}_b$ and $\{G_c^{i,j}\}_c$ yield $3n^2$ generators in Σ^n obsolete, since for $X \in \{E, F, G\}$ and $x, x' \in \{1, \dots, m\}$ we have

$$\sum_x X_x^{i,j} - \mathbb{1} = 0 \quad (6.6)$$

and thus can replace one operator by

$$X_{x'}^{i,j} = \mathbb{1} - \sum_{x \neq x'} X_x^{i,j} \quad (6.7)$$

resulting in a significantly reduced set of generators and problem $\mathbf{P}$. We refer to this reduced generator set Σ_{red}^n as the *reduced output* generator set. We can thus reformulate $\mathbf{P}$ to $\tilde{\mathbf{P}}$ given in terms of Σ_{red}^n .

Lemma 6.1. *$\mathbf{P}$ and $\tilde{\mathbf{P}}$ are equivalent.*

Proof. The completeness relations are a subset of the relations $\mathcal{R}$ defined on $\mathbb{K}\langle \Sigma \rangle$. For every $X \in \{E, F, G\}$ and $x, x' \in \{1, \dots, m\}$ the expression

$$X_{x'}^{i,j} - \mathbb{1} + \sum_{x \neq x'} X_x^{i,j} \quad (6.8)$$

generates a two-sided ideal in $\mathbb{K}\langle\Sigma\rangle$ such that all its elements have a vanishing maximal C^* -seminorm. The completion of $\mathbb{K}\langle\Sigma\rangle$ to $\mathcal{D}^n$ yields $X_{x'}^{i,j}$ equivalent to $1 - \sum_{x \neq x'} X_x^{i,j}$ in $\mathcal{D}^n$. Thus, constructing a universal C^* -algebra $\mathcal{D}_{\text{red}}^n$ from Σ_{red}^n and relations $\mathcal{R}'$ excluding the completeness relations yields $\mathcal{D}_{\text{red}}^n = \mathcal{D}^n$. Providing all the marginals of P fully captures the joint probability distribution P . $\square$

Equivalently, reduce Σ_{red}^n to $\tilde{\Sigma}_{\text{red}}^n$ by removing all adjoint generators via the hermiticity constraint.

We have used the equality constraints on generators to reduce the set of generators yielding $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$. Let us denote by $\mathcal{E} \subset \mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$ the union over all two-sided ideals generated by equalities over monomials in the generators in [Eq. \(6.6\)](#). Recall, we argued that it suffices to consider the quotient algebra $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle/\mathcal{E}$ in the SDP relaxation hierarchy construction. Partition $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle/\mathcal{E}$ into subspaces $\mathcal{V}^{(k)}$ with monomial basis $\mathcal{B}_k$. Via NPO theory, we formulate a complete SDP relaxation hierarchy.

Problem 6.3 (SDP relaxation $\tilde{\mathbf{R}}^{(k)}$ of $\mathbf{P}$).

$$\begin{aligned} \tilde{\mathbf{R}}^{(k)} : \quad & \text{s.t.} \quad p^k = \min_{L_y \in (\mathcal{V}^{(2k)})^*} 1 \\ & \Gamma^{(k)} \succeq 0 \\ & L_y(u - v) = 0, \quad \forall u, v \in \text{Trans}_{\mathcal{D}^n, \Delta_{n,k}} \cdot z, \forall z \in \mathcal{B}_{2k} \\ & L_y(w) - f(w, P) = 0 \quad \forall w \in \mathcal{P}_n(\mathcal{I}_\rho) \cap \mathcal{B}_{2k} \end{aligned} \tag{6.9}$$

Remark. We do not impose any inequality constraints on the level of operators. This avoids the necessity to implement a localising matrix. The entries of localising matrices are polynomial expressions over moments and hence computationally far more demanding. Implementing the LGG hierarchy would require multiple localising matrices corresponding to the C -bound and POVM constraints.

Remark. While the completeness relations and the hermiticity constraints allowed us to reduce the generating set to the free $*$ -algebra, the commutativity relations and the projective property reduce the cardinality of $\mathcal{B}_{2k}$.

Remark. Without the above reductions, the computational cost of solving [Eq. \(6.9\)](#) quickly surpasses the capabilities of an ordinary desktop computer.

In the subsequent section, we will explicitly solve problem [6.1](#) for several probability distributions.

7 Implementation

Due to the size of the SDPs of interest in quantum inflation, an efficient implementation is vital. This becomes especially relevant when restricted to a desktop computer. This section is dedicated to the construction, reduction and reformulation of the SDP $\tilde{\mathbf{R}}^{(k)}$ given in [Eq. \(6.9\)](#), such that we can effectively cast it to an SDP solver. Furthermore, we show how to quantify the incompatibility of a given probability distribution P and reformulate necessary conditions on any compatible distribution via SDP duality theory. We introduce several procedures to improve such quantifications. In essence, this section is concerned with translating the abstract mathematical concepts of the previous sections into a language a computer can understand.

This section consists of three parts. In the description of the core programme, we show how to actually construct, reduce and reformulate $\tilde{\mathbf{R}}^{(k)}$. The problem-specific programme elaborates on the implementation of *visibility tests*, *certificates*, *witnesses* and scalar extension. We conclude this section with a commentary on the necessary software enabling our implementation, specifically, SDP solvers.

For the corresponding source code, see [Appendix H](#)

7.1 Core programme

We start with the description of the core programme.

7.1.1 Alternative partitioning of $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$

We consider a relaxation $\mathbf{R}^{(k)}$ of $\tilde{\mathbf{R}}^{(k)}$ in [Eq. \(6.9\)](#) by partitioning $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$ into subspaces $\mathcal{F}^{(k)}$ with basis $\mathcal{B}_k$ and only then considering the quotient

$$\mathcal{U}^{(k)} := \mathcal{F}^{(k)} / (\mathcal{E} \cap \mathcal{F}^{(k)}) \quad (7.1)$$

with basis $\mathcal{W}_k$.

Remark. This is inevitable since, in practice, we cannot first construct the entire free $*$ -algebra and separate out an ideal. Any implementation is ultimately restricted to finite subspaces.

Let us address the freedom we have in this partitioning. The construction of a complete hierarchy of SDP relaxations to our NPO problem is non-unique, and there are partitionings more suited to our problem.

Historically motivated, the set of all monomials in the generators of length no larger than k is referred to as the *NPA levels* $\mathcal{S}_k$ spanning subspace $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle_k$. We denote the corresponding SDP by $\mathbf{R}_S^{(k)}$.

Example 7.1. For $\Delta_{n,k}$ consider

$$\mathcal{S}_2 = \{1\} \cup \left\{X_x^{i,j}\right\}_{x,i,j} \cup \left\{X_x^{i,j} (X')_{x'}^{k,l}\right\}_{x,x',i,j,k,l} \quad (7.2)$$

with $X, X' = E, F, G$, outcomes $x, x' \in \{1, \dots, m\}$ and inflation indices $i, j, k, l \in \{1, \dots, n\}$.

Clearly,

$$\mathcal{S}_1 \subseteq \mathcal{S}_2 \subseteq \mathcal{S}_3 \subseteq \dots \subseteq \mathcal{S}_\infty \quad (7.3)$$

holds. The completeness of the NPA hierarchy was proven for a partitioning of $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^2\rangle$ corresponding to these NPA levels [54].

Since we enforce different constraints on different elements in $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$ we can choose a partitioning, which respects this insight.

Example 7.2. In $\Delta_{2,k}$ we equate the action of the functional L_y on elements from the ai-expressible set to a probability. This corresponds to a constraint on a specific subset of elements, while the symmetry equivalence applies to all elements. The elements in $\mathcal{I}_\varrho$ have at most one generator for each observed random variable. Thus, in the corresponding ai-expressible set, we obtain at most two generators to each observed random variable.

The *local levels* $\mathcal{L}_k$ [52] correspond to the set of monomials with at most k generators to each observed random variable. It was shown there that such a partitioning of $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^2\rangle$ has the potential to yield tighter bounds at a comparable step to the NPA hierarchy.

Problem 7.1 (The local hierarchy $\mathbf{R}_\mathcal{L}^{(k)}$). *Let Σ be a set of generators and $\mathbb{K}\langle\Sigma\rangle$ the corresponding free $*$ -algebra. The relations are given by $\mathcal{R}$ and $\mathcal{S}$ is a countable set. Let us denote the subspace of $\mathbb{K}\langle\Sigma\rangle$ spanned by $\mathcal{L}_k$ with coefficients from $\mathbb{K}$ by $\mathbb{K}\langle\Sigma\rangle_{\mathcal{L}_k}$. The SDP relaxation to problem [2.1] at relaxation level k of the local hierarchy is then*

given by

$$\begin{aligned}
p^k &= \min_{L_y \in (\mathbb{K}\langle \Sigma \rangle_{\mathcal{L}_k})^*} L_y(p) \\
\mathbf{R}_{\mathcal{L}}^{(k)} : \quad \text{s.t.} \quad & \Gamma^{(k)} \succeq 0 \\
& \Lambda_q^{(k)} \succeq 0 \quad \forall q \in \mathcal{R} \cap \mathbb{K}\langle \Sigma \rangle_{\mathcal{L}_{2k}} \\
& L_y(s) \geq 0 \quad \forall s \in \mathcal{S} \cap \mathbb{K}\langle \Sigma \rangle_{\mathcal{L}_{2k}}.
\end{aligned} \tag{7.4}$$

Note, that

$$\mathcal{S}_k \subset \mathcal{L}_k \not\subset \mathcal{S}_{k+1} \tag{7.5}$$

and $\mathcal{S}_{pk}$ is the smallest NPA level containing $\mathcal{L}_k$, where p is the number of observed random variables in a quantum causal structure under consideration.

Theorem 7.1. $\mathbf{R}_{\mathcal{L}}^{(k)}$ is asymptotically complete.

Proof. Since $\mathcal{L}_k \subseteq \mathcal{S}_{pk}$, the $\mathbf{R}_{\mathcal{L}}^{(k)}$ constitute a subsequence of $\mathbf{R}_{\mathcal{S}}^{(k)}$. The completeness of $\mathbf{R}_{\mathcal{S}}^{(k)}$ implies the completeness of $\mathbf{R}_{\mathcal{L}}^{(k)}$. $\square$

Example 7.3. In $\Delta_{n,k}$ consider

$$\mathcal{L}_1 = \{1\} \cup \left\{ X_p^{i,j} \right\}_{p,i,j} \cup \left\{ X_p^{i,j} (X'_q)^{k,l} \right\}_{p,q,i,j,k,l} \cup \left\{ E_a^{i,j} F_b^{k,l} G_c^{m,n} \right\}_{a,b,c,i,j,k,l,m,n} \tag{7.6}$$

where $X_p \neq X'_q$ and $X, X' \in \{E, F, G\}$, outcomes $p, q, a, b, c \in \{1, \dots, m\}$ and inflation indices $i, j, k, l, mn, n \in \{1, \dots, n\}$. And thus $\Delta_{2,3}$ corresponds to $\mathcal{L}_2$.

Remark. In the feasibility tests corresponding to the QCCP, we enforce a lower bound on k from the injectable sets via the requirement

$$\mathcal{I}_{\tilde{\Sigma}_{\text{red}}^1} \subseteq \mathcal{W}_{2k}, \tag{7.7}$$

for a monomial basis $\mathcal{W}_{2k}$ to our subspace $\mathcal{U}^{(2k)}$. Thus, for the feasibility tests in $\Delta_{2,k}$ we have either $k = 3$ and consider $\mathbf{R}_{\mathcal{S}}^{(3)}$ for an NPA partitioning or $k = 2$ with $\mathbf{R}_{\mathcal{L}}^{(2)}$ for a local level partitioning.

7.1.2 Intermediate hierarchy levels

In practice, both $\mathbf{R}_{\mathcal{S}}^{(3)}$ and $\mathbf{R}_{\mathcal{L}}^{(2)}$ are computationally expensive to construct and solve. We consider intermediate steps in these hierarchies by introducing a *cutoff relaxation* on the monomial degree arising in the generating monomials. Given a set of generating

monomials $\mathcal{W}_k$ and a cutoff $c \in \mathbb{N}_+$, then the *cutoff set* is

$$\mathcal{W}_{k,c} = \{x \in \mathcal{W}_k : |x| \leq c\}. \quad (7.8)$$

Remark. We denote the generating monomials with cutoff c via $\mathcal{S}_{k,c}$ and $\mathcal{L}_{k,c}$. Analogously, the corresponding SDP relaxations are $\mathbf{R}_S^{(\mathbf{k},c)}$ and $\mathbf{R}_L^{(\mathbf{k},c)}$.

Alternatively, we can construct intermediate steps by considering certain subsets of $\mathcal{W}_k$, the *positive sets*,

$$\mathcal{P}_k = \{x \in \mathcal{W}_k : x^* = x \wedge x^2 = x\} \subseteq \mathcal{W}_k. \quad (7.9)$$

The motivation for positive sets is the following. A positive functional maps positive elements to non-negative real numbers. We relaxed this requirement to $\Gamma^{(k)} \succeq 0$ at a finite step of the SDP hierarchy. By construction, any element from $\mathcal{P}_k$ is positive. Recall that for any psd matrix, its main diagonal entries are non-negative. For $w_i \in \mathcal{P}_k = \{w_i\}_{i=1}^n$ requiring that

$$\Gamma^{(k)} = \begin{matrix} & \mathbb{1} & w_1 & w_2 & \cdots & w_n \\ \begin{matrix} \mathbb{1} \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{matrix} & \begin{pmatrix} 1 & & & & \\ & y_{w_1} & & & \\ & & y_{w_2} & & \\ & & & \ddots & \\ & & & & y_{w_n} \end{pmatrix} \end{matrix} \quad (7.10)$$

be psd implies $y_{w_i} \geq 0 \ \forall w_i \in \mathcal{P}_k$.

Remark. We require our generators to be hermitian and projectors. We identify those monomials in the generators, which are also hermitian and projectors. This is equivalent to requiring that a monomial consists of mutually commutative generators.

Remark. The elements in the injectable set are positive.

Remark. We denote the corresponding SDP by $\mathbf{R}_S^{(\mathbf{k},+)}$ or $\mathbf{R}_L^{(\mathbf{k},+)}$.

7.1.3 The monomial basis to the quotient algebra

Independent of the partitioning, we aim to determine a basis $\mathcal{W}_k$ to the quotient

$$\mathcal{U}^{(k)} := \mathcal{F}^{(k)} / (\mathcal{E} \cap \mathcal{F}^{(k)}) \quad (7.11)$$

from a basis $\mathcal{B}_k$ to $\mathcal{F}^{(k)}$ obtained from partitioning $\mathbb{K}\langle\tilde{\Sigma}_{\text{red}}^n\rangle$. To reduce the problem to this quotient algebra $\mathcal{U}^{(k)}$, we need to fully “separate out” the ideal $\mathcal{E} \cap \mathcal{F}^{(k)}$ from $\mathcal{F}^{(k)}$. To do so, we need to determine a basis of this ideal. This way, we can reduce the basis $\mathcal{B}_k$ of the algebra $\mathcal{F}^{(k)}$ to a *reduced basis* $\mathcal{W}_k$ to $\mathcal{U}^{(k)}$. Having constructed such a reduced basis to the quotient algebra ensures that we truly work only within this quotient algebra. In general, i.e. for arbitrary equality constraints, determining such a basis of the ideal is hard. Assuming that such a basis exists, the *Buchberger algorithm* yields a *commutative Gröbner basis* when considering commutative algebras [51]. However, the non-commutative variant is much harder. We refer to [51] for an elaborate discussion on *non-commutative Gröbner bases*. We restrict to the projection and commutativity constraints. We collect these binary relations in $\mathcal{R} \subset (\tilde{\Sigma}_{\text{red}}^n)^* \times (\tilde{\Sigma}_{\text{red}}^n)^*$, where $(\tilde{\Sigma}_{\text{red}}^n)^*$ denotes the Kleene star to $\tilde{\Sigma}_{\text{red}}^n$.

The problem $\mathbf{R}^{(k)}$ is fully determined in terms of elements from the basis $\mathcal{B}_{2k} \subseteq (\tilde{\Sigma}_{\text{red}}^n)^*$. Since we do not consider localising matrices and L_y is a linear functional, we need not worry about polynomial expressions²¹. The only operation we encounter is the concatenation of monomials. Thus we work solely in the free $*$ -monoid $\mathcal{M} := (\tilde{\Sigma}_{\text{red}}^n, \cdot, *)$ or, more precisely, a finite subspace thereof. We reduce the problem to a quotient monoid by modding out the set of equivalence classes with equivalence relations $\sim$ induced by projection and commutation relations defined on $\mathcal{M}$. This corresponds to a partitioning of $(\tilde{\Sigma}_{\text{red}}^n)^*$ into equivalence classes. We obtain $\mathcal{U}^{(k)}$ from $(\tilde{\Sigma}_{\text{red}}^n)^* / \sim$ by adding the required structure as described in subsection 2.4.2. Concretely, we partition $\mathcal{M}$ into *congruence classes* with *congruence relation* $\sim$, i.e. equivalence classes preserving the algebraic structure of $\mathcal{M}$. Concretely, for any $w_1, w_2, u_1, u_2 \in \mathcal{M}$ such that $w_1 \sim w_2$ and $u_1 \sim u_2$ it follows

$$w_1^* \sim w_2^*, \quad w_1 u_1 \sim w_2 u_2. \quad (7.12)$$

A congruence class is then

$$[w]_{\sim} = \{u \in \mathcal{M} : u \sim w\}. \quad (7.13)$$

We collect all such congruence classes in a set $\mathcal{M} / \sim$ and define a binary operation $[w]_{\sim} \cdot [u]_{\sim} = [w \cdot u]_{\sim}$ for any $w, u \in \mathcal{M}$. Thus, $(\mathcal{M} / \sim, \cdot, *)$ is the quotient monoid [71] with respect to the congruence relation $\sim$.

Let us now focus on a numerical implementation of the above. A computer has

²¹Therefore, we do not need to worry about a non-commutative Gröbner basis.

no notion of an operator or an abstract algebra element. However, $(\tilde{\Sigma}_{\text{red}}^n, \cdot, *)$ has a natural string representation. In the following, we will abstain from making this string representation clear. We encode the congruence relations via an *abstract rewriting system* as a natural extension of a *string rewriting system* [13] $(\tilde{\Sigma}_{\text{red}}^n, \mathcal{R})$. Each tuple $(w, u) \in \mathcal{R}$ constitutes a *rewriting rule* denoted by $w \rightarrow u$. We want to be able to apply these rewriting rules to any element of $(\tilde{\Sigma}_{\text{red}}^n)^*$ and not just those in $\mathcal{R}$. Let $\xrightarrow{\mathcal{R}}$ be the *one-step rewriting relation* [13] induced by $\mathcal{R}$ on $(\tilde{\Sigma}_{\text{red}}^n)^*$ such that for any $s, t \in (\tilde{\Sigma}_{\text{red}}^n)^*$

$$s \xrightarrow{\mathcal{R}} t \Leftrightarrow \exists x, y \in (\tilde{\Sigma}_{\text{red}}^n)^* : s = xwy, t = xuy, (w, u) \in \mathcal{R}. \quad (7.14)$$

Thus, we obtain an abstract rewriting system $((\tilde{\Sigma}_{\text{red}}^n)^*, \xrightarrow{\mathcal{R}})$ on all monomials from $(\tilde{\Sigma}_{\text{red}}^n)^*$ from our projection and commutativity constraints. Since our binary relations stem from equalities between elements of $(\tilde{\Sigma}_{\text{red}}^n)^*$, they are symmetric, reflexive and transitive and additionally preserve the algebraic structure of the corresponding free $*$ -monoid $\mathcal{M}$. We thus define the *reflexive transitive symmetric closure* of $\xrightarrow{\mathcal{R}}$ [13] as a congruence relation on $\mathcal{M}$ and denote it by $\xleftrightarrow{*}_{\mathcal{R}}$. The resulting quotient monoid $\mathcal{M}/\xleftrightarrow{*}_{\mathcal{R}}$ corresponds to $\mathcal{M}/\sim$. In other words, we have reformulated the congruence relation $\sim$ in terms of a rewriting system.

Example 7.4. Consider $E_a^{1,1}, E_a^{1,2}, F_b^{2,1} \in \tilde{\Sigma}_{\text{red}}^n$. The projection constraint gives rise to a binary relation $((E_a^{1,1})^2, E_a^{1,1}) \in (\tilde{\Sigma}_{\text{red}}^n)^* \times (\tilde{\Sigma}_{\text{red}}^n)^*$. Then an example of rewriting rules is given by

$$(E_a^{1,1})^2 \xrightarrow{\mathcal{R}} E_a^{1,1}, \quad E_a^{1,1} \xrightarrow{\mathcal{R}} (E_a^{1,1})^2 \quad (7.15)$$

and an extension is

$$E_a^{1,2}(E_a^{1,1})^2 F_b^{2,1} \xleftrightarrow{*}_{\mathcal{R}} E_a^{1,2} E_a^{1,1} F_b^{2,1}. \quad (7.16)$$

We have argued that we can use rewriting rules to traverse from element to element through each congruence class. Choosing a representative for each congruence class corresponds to modding out these congruence classes to obtain the quotient monoid. In theory, any element of a congruence class can be chosen to be a representative. To avoid unnecessary computation, we must provide a systematic framework to coherently determine the same representative to a congruence class at any step in the programme. The symmetry of our binary relations requires us to provide additional structure defined on congruence classes to choose such a representative. We need to supply a preference in the direction a rewriting rule is applied. To do so, we provide a strict well-order

$<_{\mathcal{M}}$ on $\mathcal{M}$ and require that the rewriting rules be applied in such a manner as to yield the smallest element with regard to this order. Concretely, for any congruence class $[w]_{\sim} \subset \mathcal{M}$ we require the application of the rewriting rules to yield the *normal form* $\mathcal{N}_{\mathcal{R}}(w)$ [71] which satisfies

$$\forall u \in [w]_{\sim} \setminus \mathcal{N}_{\mathcal{R}}(w) : \mathcal{N}_{\mathcal{R}}(w) <_{\mathcal{M}} u. \quad (7.17)$$

We have for any $u, v \in \mathcal{M}$

$$u \sim v \Leftrightarrow \mathcal{N}_{\mathcal{R}}(u) = \mathcal{N}_{\mathcal{R}}(v). \quad (7.18)$$

We order by degree first, i.e.

$$x <_{\mathcal{M}} y \Leftrightarrow |x| < |y| \quad (7.19)$$

In other words, apply the rewriting rules corresponding to the projection constraints in the direction, which reduces the degree of the resulting monomial. Comparing two monomials with the same degree requires us to first define a strict well-order $<_{\tilde{\Sigma}_{\text{red}}^n}$ on $\tilde{\Sigma}_{\text{red}}^n$. Our generators are indexed in their party and subalgebra correspondences and their outcomes. On party correspondence, we order as

$$E_a^{i,j} <_{\tilde{\Sigma}_{\text{red}}^n} F_b^{l,t} <_{\tilde{\Sigma}_{\text{red}}^n} G_c^{m,n} \quad (7.20)$$

for all choices of indices. For two generators to one party, we then order in terms of subalgebra correspondence by first comparing the first indices and then the second, i.e.

$$X_x^{i,j} <_{\tilde{\Sigma}_{\text{red}}^n} X_x^{m,n}, \quad (7.21)$$

if $i < m$ or for $i = m$ we have $j < n$. Lastly, we compare the outcome indices

$$X_x^{i,j} <_{\tilde{\Sigma}_{\text{red}}^n} X_{x'}^{i,j} \Leftrightarrow x < x'. \quad (7.22)$$

This order naturally extends to an order on monomials in these generators. We have thus obtained a “sense of direction” applying the rewriting rules resulting from commutation relations. In summary, we have algorithm [1]

For $\Delta_{2,k}$, we employ an optimised *bubble sort algorithm* [43] to enforce such a rewriting system, i.e. effectively reduce any monomial to its normal form. Its worst and average case performance ordering a list of length n is $\mathcal{O}(n^2)$. While there exist faster sorting algorithms, for sorting monomials of degree of $\mathcal{O}(1)$ this optimised bubble

Algorithm 1 $\mathcal{W}_k$ construction

Require: $\mathcal{B}_k, \mathcal{W}_k = \emptyset$
 for $b \in \mathcal{B}_k$ **do**
 if $\mathcal{N}_{\mathcal{R}}(b) \notin \mathcal{W}_k$ **then**
 add $\mathcal{N}_{\mathcal{R}}(b)$ to $\mathcal{W}_k$
 end if
 end for
 return $\mathcal{W}_k$

sort suffices.

Remark. Whenever we concatenate two monomials $w_1, w_2 \in \mathcal{W}_{2k}$ in the implementation to a new monomial w , we apply the rewriting rules to represent it in its normal form. Note that in general $\mathcal{N}_{\mathcal{R}}(w_1)\mathcal{N}_{\mathcal{R}}(w_2) \neq \mathcal{N}_{\mathcal{R}}(w)$.

Remark. We can exploit the fact that generators to different parties commute. To any $w \in (\tilde{\Sigma}_{\text{red}}^n)^*$ there exist $w_1 \in (\{E_a^{i,j}\}_{i,j,a})^*$, $w_2 \in (\{F_b^{i,j}\}_{i,j,b})^*$ and $w_3 \in (\{G_c^{i,j}\}_{i,j,c})^*$ such that $w = w_1 w_2 w_3$. Then, $\mathcal{N}_{\mathcal{R}}(w) = \mathcal{N}_{\mathcal{R}}(w_1)\mathcal{N}_{\mathcal{R}}(w_2)\mathcal{N}_{\mathcal{R}}(w_3)$.

We have thus “taken care” of all constraints on generators and monomials thereof from $\mathbf{R}^{(\mathbf{k})}$. The essential insight in the reformulation of $\mathbf{R}^{(\mathbf{k})}$ to an SDP compatible with SDP solvers is to do something similar with the constraints defined on moments. This notion will become more clear in the subsequent chapters.

7.1.4 Real symmetric $\Gamma^{(k)}$

In $\mathbf{R}^{(\mathbf{k})}$ the objective function is real. Construct real and symmetric matrix

$$\tilde{\Gamma}^{(k)} = \frac{(\Gamma^{(k)})^* + \Gamma^{(k)}}{2} \in \mathcal{S}_n. \quad (7.23)$$

from $\Gamma^{(k)}$ in $\mathbf{R}^{(\mathbf{k})}$. By convexity,

$$\tilde{\Gamma}^{(k)} \succeq 0 \Rightarrow \Gamma^{(k)} \succeq 0. \quad (7.24)$$

Hence, we need not worry about reformulating a complex SDP as a larger real SDP and replace $\Gamma^{(k)}$ in $\mathbf{R}^{(\mathbf{k})}$ by $\tilde{\Gamma}^{(k)}$.

Remark. SDP solvers are far more efficient at solving problems in terms of real symmetric matrices than complex, and hermitian ones [\[33\]](#).

Remark. From now on, we will drop the $\sim$ on $\tilde{\Gamma}^{(k)}$ and consider $\Gamma^{(k)}$ to be real and symmetric.

7.1.5 $\Gamma^{(k)}$ construction

Since $\Gamma^{(k)}$ is symmetric, it suffices to restrict the construction of the moment matrix to its triangular form. $\Gamma^{(k)}$ contains all moments to monomials in $\mathcal{W}_{2k}$. Thus, we first need to construct $\mathcal{W}_{2k}$. To do so, we write $\mathcal{W}_k$ as a row and a column vector and compute their matrix product. We rewrite every entry in the resulting matrix to its normal form. Collecting all these normal forms in a set corresponds to $\mathcal{W}_{2k}$. Clearly,

$$|\mathcal{W}_{2k}| \leq |\mathcal{W}_k| |\mathcal{W}_k|, \quad (7.25)$$

where the equality holds only in the trivial case. Applying L_y to each entry yields $\Gamma^{(k)}$. Thus, $\Gamma^{(k)}$ contains symmetry corresponding to the congruence classes stemming from the projection and commutation relations.

Remark. In the implementation, this mapping is hidden, and we do not notationally differentiate between monomials and moments.

In the implementation, we introduce the notion of a *moment representative*. The matrix product is computed iteratively. A moment representative is the first instance a moment appears in this computation. We collect all these moment representatives and their position in the matrix as key-value pairs in the *representative dictionary*²² $D_R(\Gamma^{(k)})$. A second *structure dictionary* $D_S(\Gamma^{(k)})$ registers all positions as keys at which a moment already in $D_R(\Gamma^{(k)})$ appears in the computation and assigns them as value the position saved in $D_R(\Gamma^{(k)})$. In other words, $D_S(\Gamma^{(k)})$ encodes the symmetry of $\Gamma^{(k)}$ from the congruence classes. A moment requires more memory than a simple tuple (x, y) with integers x, y . Thus, separating $\Gamma^{(k)}$ allows us to reduce memory requirements significantly. Additionally, faster lookups in the smaller dictionary $D_R(\Gamma^{(k)})$ reduce computing times.

Example 7.5. In the construction of $\mathcal{W}_k$ we have explicitly excluded $E_a^{1,1} E_a^{1,1}$ by exploiting the projection constraint. We include in $\mathcal{W}_k$ only the representative $[E_a^{1,1}]_{\sim}$,

²²Mathematically, a dictionary is an ordered set where each element is a tuple (a, b) denoted $(a : b)$ (key-value pair). No two elements have the same key. We denote the extraction of a value b to a key a via $D[a]$ and we add a key-value pair to D via $D[x] = y$ for some appropriate x, y . Dictionaries correspond to significantly faster lookup times. A memory-time trade-off compared to lists.

thus working in the quotient. In $\Gamma^{(k)}$, we have

$$\Gamma^{(k)} = \begin{matrix} & \mathbf{1} & [E_a^{1,1}]_{\sim} & \cdots & [F_b^{1,2}]_{\sim} & \cdots \\ \begin{matrix} \mathbf{1} \\ [E_a^{1,1}]_{\sim} \\ \vdots \\ [F_b^{1,2}]_{\sim} \\ \vdots \end{matrix} & \left(\begin{matrix} y_{[1]_{\sim}} & y_{[E_a^{1,1}]_{\sim}} & & y_{[F_b^{1,2}]_{\sim}} & \\ & y_{[E_a^{1,1}]_{\sim}} & & y_{[E_a^{1,1} F_b^{1,2}]_{\sim}} & \\ & & & & y_{[F_b^{1,2}]_{\sim}} \end{matrix} \right) \end{matrix} \quad (7.26)$$

yielding²³

$$D_R(\Gamma^{(k)}) = \{(y_{[1]_{\sim}} : (0, 0)), (y_{[E_a^{1,1}]_{\sim}} : (0, 1)), (y_{[F_b^{1,2}]_{\sim}} : (0, t)), (y_{[E_a^{1,1} F_b^{1,2}]_{\sim}} : (1, t))\} \quad (7.27)$$

for some $t \in \mathbb{N}_+$ and

$$D_S(\Gamma^{(k)}) = \{((1, 1) : (0, 1)), ((s, t) : (0, t))\} \quad (7.28)$$

for some integer $s \in \mathbb{N}_+$ in our implementation.

For a more detailed description of the construction of $D_R(\Gamma^{(k)})$ and $\mathcal{D}_S(\Gamma^{(k)})$ see [Appendix C](#). In the subsequent sections, we will exploit the equality constraints defined on moments to reduce the cardinality of $D_R(\Gamma^{(k)})$. In other words, we define equivalence classes on moments and reduce the problem to representatives of these classes.

7.1.6 Involution equivalence

Since we have argued to consider $\Gamma^{(k)}$ to be real and symmetric, there exists an *involution equivalence* on moments

$$y_{w^*}^* = y_{w^*} = y_w \quad (7.29)$$

$\forall y_w \in \Gamma^{(k)}$. While w and w^* correspond to the same moment, they are generally not equal. We again faced the difficulty of systematically and coherently choosing a representative of this equivalence relation. We lift the order $<_{\mathcal{M}}$ on $\mathcal{M}$ to an order on

²³We have used the computer science indexing.

moments²⁴ via

$$y_w <_{\mathcal{M}} y_u \iff w <_{\mathcal{M}} u \quad (7.30)$$

for any $w, u \in \mathcal{M}$. This allows us to choose between y_{w^*} and y_w for $w \neq w^*$.

Example 7.6. Even though $E_a^{1,1}, E_a^{1,2}$ do not commute and thus $E_a^{1,1}E_a^{1,2} \neq E_a^{1,2}E_a^{1,1}$ we have

$$y_{E_a^{1,1}E_a^{1,2}} = y_{E_a^{1,2}E_a^{1,1}} \quad (7.31)$$

by involution equivalence. Since $E_a^{1,1}E_a^{1,2} <_{\mathcal{M}} E_a^{1,2}E_a^{1,1}$ we choose $y_{E_a^{1,1}E_a^{1,2}}$ to be a representative of this equivalence.

7.1.7 Orbits of $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}$

We continue our endeavour to reduce the cardinality of $D_R(\Gamma^{(k)})$. We equate all moments corresponding to monomials in an orbit to $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}$. We then choose a representative of this equivalence relation defined on moments and eliminate the rest of the equivalence class from $D_R(\Gamma^{(k)})$. To $D_S(\Gamma^{(k)})$ we add the positions of these eliminated moments as keys and assign them the position of the representative as a value. For computational purposes, the choice of a representative is not determined by the induced order $<_{\mathcal{M}}$ on moments but rather by their order of appearance in $D_R(\Gamma^{(k)})$. Concretely,

Algorithm 2 Symmetry reduction of $D_R(\Gamma^{(k)})$

Require: $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}, D_R(\Gamma^{(k)}), D_S(\Gamma^{(k)})$
for $y_w \in D_R(\Gamma^{(k)})$ **do**
 $\mathcal{O}_w = \text{Trans}_{\mathcal{D}^n, \Delta_{n,k}} \cdot w$
 Add $\{(D_R(\Gamma^{(k)})[y_v] : D_R(\Gamma^{(k)})[y_w]) : v \in \mathcal{O}_w/w\}$ to $D_S(\Gamma^{(k)})$
 Erase $\{y_v : v \in \mathcal{O}_w/w\}$ from $D_R(\Gamma^{(k)})$
end for
return $D_R(\Gamma^{(k)}), D_S(\Gamma^{(k)})$

Example 7.7. For $\Delta_{2,k}$ the generating set of $\text{Trans}_{\mathcal{D}^2, \Delta_{2,k}}$ is

$$\Pi = \{\pi, \pi', \pi''\}. \quad (7.32)$$

²⁴Not to be confused with the natural order on the field $\mathbb{R}$.

Explicitly, we obtain

$$\text{Trans}_{\mathcal{D}^2, \Delta_{2,k}} = \{1, \pi, \pi', \pi'', \pi\pi', \pi\pi'', \pi'\pi'', \pi\pi'\pi''\}, \quad (7.33)$$

due to the cyclicity and that any π, π', π'' permutes disjoint sets of copy-index equivalent subalgebras. Hence, the orbits in $\Delta_{2,k}$ are at most of cardinality eight.

7.1.8 Factorisation

We require a procedure to factorise moments effectively. To do so, we introduce the *tensor representation* of any monomial in $\mathcal{W}_{2k}$.

Definition 7.1 (Tensor representation). At inflation level n , a monomial $\mathcal{N}_{\mathcal{R}}(w) \in \mathcal{W}_{2k}$ can be represented by a tensor $T_{\mathcal{N}_{\mathcal{R}}(w)}$ of shape $(3, n, n)$. Let $\mathcal{N}_{\mathcal{R}}(w)$ l -th generator²⁵ be given by $\mathcal{N}_{\mathcal{R}}(w)[l] = X_{x'}^{i,j}$. Then,

$$(T_{\mathcal{N}_{\mathcal{R}}(w)})_{s,i,j} = l \quad \text{where } s = \begin{cases} 1 & \text{for } X = E \\ 2 & \text{for } X = F \\ 3 & \text{for } X = G \end{cases} \quad (7.34)$$

All entries in $T_{\mathcal{N}_{\mathcal{R}}(w)}$ not assigned a value are set to zero.

See [Figure 7.1](#) for a graphical depiction of the generator-tensor position correspondance and for an explicit example [Figure 7.2](#).

Remark. When multiple instances of one generator appear in a monomial, we collect their positions in a list at the corresponding tensor entry.

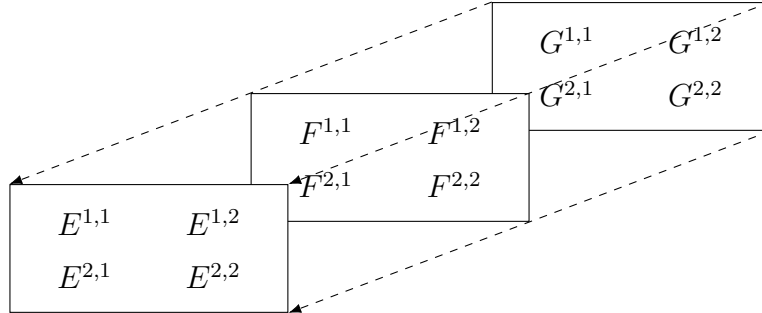


Figure 7.1: Schematic display of the generator-tensor position correspondance in $\Delta_{2,k}$.

This tensor representation allows us to encode the graphical structure of $\Delta_{n,k}$ as depicted in [Figure 7.3](#) and [Eq. \(7.35\)](#). We create lists corresponding to all state instances by summing the corresponding rows and columns. For $j = 1, \dots, n$ we append

²⁵ $\mathcal{N}_{\mathcal{R}}(w)$ is nothing more but an ordered list of generators.

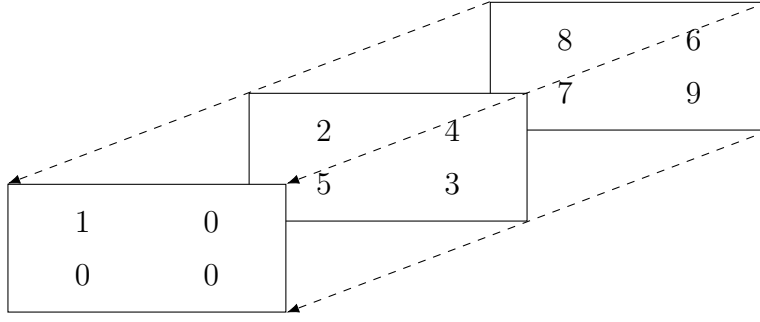


Figure 7.2: Tensor representation of $\varrho(E^{1,1}F^{1,1}F^{2,2}F^{1,2}F^{2,1}G^{1,2}G^{2,1}G^{1,1}G^{2,2})$ in $\Delta_{2,k}$.

$\tilde{A}$'s j -th column with $\tilde{B}$'s j -row, $\tilde{A}$'s j -row with $\tilde{C}$'s j -th column and $\tilde{B}$'s j -th column with $\tilde{C}$'s j -th row. We collect these $3n$ sets $\mathcal{T}_i$ in a set $\mathcal{T}$. The process of factorisation is then to consider the intersection of all combinations of subsets $\mathcal{T}_i, \mathcal{T}_j \in \mathcal{T}$ and create their union, if and only if $\mathcal{T}_i \cap \mathcal{T}_j \neq \emptyset$. We do this until all sets are disjoint. We end up with several disjoint sets corresponding to distinct factorisation components under the action of L_y . We obtain only one list if a monomial does not factorise under the state's action. To conclude the procedure, we order the integers in each sublist in ascending order to preserve the commutation relations and replace each l by $\mathcal{N}_{\mathcal{R}}(w)[l]$. The procedure is captured in algorithm [3](#).

Remark. Note, that even though $\mathcal{N}_{\mathcal{R}}(w)$ is given in normal form, the components y_{w_i} resulting from factorisation need not be in normal form, i.e. $w_i \neq \mathcal{N}_{\mathcal{R}}(w_i)$.

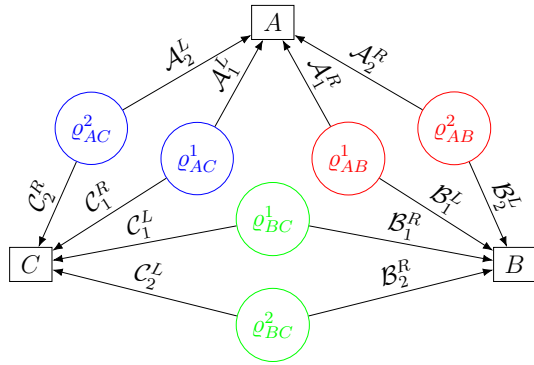


Figure 7.3: Visual aid in tensor representation factorisation.

$$\begin{aligned}
 \tilde{A} &= \begin{matrix} \varrho_{AB}^1 & \varrho_{AB}^2 \\ \varrho_{AC}^1 & \varrho_{AC}^2 \end{matrix} \begin{pmatrix} E_a^{1,1} & E_a^{1,2} \\ E_a^{2,1} & E_a^{2,2} \end{pmatrix}, \\
 \tilde{B} &= \begin{matrix} \varrho_{BC}^1 & \varrho_{BC}^2 \\ \varrho_{AB}^1 & \varrho_{AB}^2 \end{matrix} \begin{pmatrix} F_b^{1,1} & F_b^{1,2} \\ F_b^{2,1} & F_b^{2,2} \end{pmatrix}, \\
 \tilde{C} &= \begin{matrix} \varrho_{AC}^1 & \varrho_{AC}^2 \\ \varrho_{BC}^1 & \varrho_{BC}^2 \end{matrix} \begin{pmatrix} G_c^{1,1} & G_c^{1,2} \\ G_c^{2,1} & G_c^{2,2} \end{pmatrix}
 \end{aligned} \tag{7.35}$$

Algorithm 3 Moment factorisation

Require: y_w

Construct T_w

for $j \rightarrow n$ **do**

$(AB)_j := \{(T_w)_{1,i,j}, (T_w)_{2,j,i}\}_{i=1}^n$

$(AC)_j := \{(T_w)_{1,j,i}, (T_w)_{3,i,j}\}_{i=1}^n$

$(BC)_j := \{(T_w)_{2,i,j}, (T_w)_{3,j,i}\}_{i=1}^n$

end for

Factorisation by fusing sublists

$L := \{(AB)_j, (AC)_j, (BC)_j\}_{j=1}^n$

for $x \in L$ **do**

for $y \in L \setminus x$ **do**

if $x \cap y \neq \emptyset$ **then**

Replace x, y by $x \cup y$ in L

Start over

▷ Fuse sublists until no two sublists intersect

end if

end for

end for

for $x \in L$ **do**

Order x

▷ Integers in ascending order

for $y \in x$ **do**

Replace y by $w[y]$

▷ Revert position index to generator

end for

end for

return L

▷ List of factorised components

7.1.9 The ai-expressible set

To determine $\mathcal{P}_n(\mathcal{I}_\varrho) \cap \mathcal{W}_{2k}$, we factor every $y_w \in D_R(\Gamma^{(k)})$ into its components. Recall, we are able to link a component to a probability if we can map its corresponding monomial to an element of the injectable set $\mathcal{I}_{\tilde{\Sigma}_{\text{red}}^1}$ via $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}$. We exploit the $\Delta_{n,k}$ structure to reformulate this condition. We collect all subalgebra indices to generators in a component in a list. We check for distinct elements, i.e. if the corresponding set is of cardinality two or higher. If not, the monomial to the component is an element of $\mathcal{I}_{\tilde{\Sigma}_{\text{red}}^1}$. The procedure is captured in algorithm 4. A moment is then an element of the image of $\mathcal{P}_n(\mathcal{I}_\varrho) \cap \mathcal{W}_{2k}$ under L_y , if and only if this holds for all its components²⁶.

We provide the programme with the probabilities to the elements of the injectable

²⁶We will see later how to exploit moments where only a subset of its components meet this condition.

set $\mathcal{I}_{\tilde{\Sigma}_{\text{red}}^1}$. Recall, for $\Delta_{n,k}$ we have

$$\mathcal{I}_{\tilde{\Sigma}_{\text{red}}^1} = \{X_x^{1,1}, X_x^{1,1}Y_y^{1,1}, E_a^{1,1}F_b^{1,1}G_c^{1,1}\}_{X,Y,x,y,a,b,c}, \quad (7.36)$$

with $X, Y \in \{E, F, G\}$ and according outcomes $x, y, a, b, c \in \{1, \dots, m-1\}$. The distributions we consider are given in [Appendix E](#). They are symmetric under permuting the observed nodes. Linking $w \in \mathcal{I}_{\tilde{\Sigma}_{\text{red}}^1}$ to a probability is then simply a question of w 's degree. Thus,

$$y_w = \begin{cases} P(a) & \text{for } |w| = 1 \\ P(a, b) & \text{for } |w| = 2 \\ P(a, b, c) & \text{for } |w| = 3. \end{cases} \quad (7.37)$$

Since the degree of a monomial is preserved under the action of $\text{Trans}_{\mathcal{D}^n, \Delta_{n,k}}$, we link component y_{w_i} of a moment y_w in the ai-expressible set to a probability via [Eq. \(7.37\)](#). The probability to y_w is then their product. Lastly, we replace in $D_R(\Gamma^{(k)})$ the ai-expressible moments by their probability value. While this does not decrease $|D_R(\Gamma^{(k)})|$, it decreases the number of free SDP variables.

Example 7.8. The $\Gamma^{(k)}$ from [example 7.5](#) has been reduced to

$$\Gamma^{(k)} = \begin{matrix} & \mathbb{1} & [E_a^{1,1}]_{\sim} & \cdots & [F_b^{1,2}]_{\sim} & \cdots \\ \mathbb{1} & \begin{pmatrix} 1 & P(a) & & P(b) \\ & P(a) & & y_{[E_a^{1,1}F_b^{1,2}]_{\sim}} \\ & & & P(b) \end{pmatrix} \\ [E_a^{1,1}]_{\sim} & & & & & \\ \vdots & & & & & \\ [F_b^{1,2}]_{\sim} & & & & & \\ \vdots & & & & & \end{matrix}. \quad (7.38)$$

The moment $y_{[E_a^{1,1}F_b^{1,2}]_{\sim}}$ is an unknown variable in the SDP formulation. In $\Delta_{2,k}$, we have $|\mathcal{P}_2(\mathcal{I}_{\mathcal{Q}}) \cap \mathcal{W}_{2k}| = 37$.

Algorithm 4 Ai-expressible moment

Require: y_w

Factorise $y_w = \prod_{i=1}^f y_{w_i}$

for $i \rightarrow f$ **do**

 Collect subalgebra indices to generators in y_{w_i} in list L

 Construct set S to L

if $|S| \geq 2$ **then return** y_w not ai-expressible

end if

end for

return y_w is ai-expressible

7.1.10 SDP reformulation

The QCPP in problem [6.1](#) has been relaxed via quantum inflation to the NPO problem **P** in [6.2](#). This in turn has been relaxed via NPO theory to the SDP $\mathbf{R}^{(k)}$. In this section we reduced $\mathbf{R}^{(k)}$ to ultimately $D_R(\Gamma^{(k)})$ and $D_S(\Gamma^{(k)})$. Let us collect the remaining l unknown moments from $D_R(\Gamma^{(k)})$ in a vector $\mathbf{y} \in \mathbb{R}^l$. Concretely, we have recast the feasibility test $\mathbf{R}^{(k)}$ to the following **PSDP** in inequality form

$$\begin{aligned} &\text{find } \mathbf{y} \\ &\text{s.t. } \Gamma^{(k)} \succeq 0. \end{aligned} \tag{7.39}$$

A solver is assigned the task of finding numerical values to the scalar variables in $\mathbf{y}$ in the SDP such that the resulting $\Gamma^{(k)}$ is psd. Such a feasibility test is not preferred by numerical solvers and will often fail to differentiate between compatible and incompatible distributions. Instead, we will transform the **PSDP** to an eigenvalue optimisation problem with $\lambda \in \mathbb{R}$

$$\begin{aligned} &\min_{\lambda \in \mathbb{R}, \mathbf{y} \in \mathbb{R}^l} \quad -\lambda \\ &\text{s.t. } \Gamma^{(k)} - \lambda \mathbf{1} \succeq 0 \end{aligned} \tag{7.40}$$

exploiting the fact that a psd matrix has non-negative eigenvalues. For $|\mathcal{W}_k| = t$, let $C(P), \{A_i\}_{i=1}^l$ with $A_i \in S_n$ be such that

$$\Gamma^{(k)} = C(P) + \sum_{i=1}^l y_i A_i. \tag{7.41}$$

In our case, the A_i are sparse matrices (non-zero entries are ones) corresponding to the positions of the moment $y_i \in \mathbf{y}$ in $\Gamma^{(k)}$. Thus, we refer to the A 's as *position matrices*. In other words, every A_i corresponds to the equivalence class to each y_i . The constant matrix $C(P)$ is referred to as the *cost matrix*. Exploiting sparse matrix formulations al-

lows for significant time and memory savings [90]. One can then reformulate Eq. (7.40) as

$$\begin{aligned} \min_{\lambda \in \mathbb{R}, \mathbf{y} \in \mathbb{R}^l} \quad & -\lambda \\ \text{s.t.} \quad & C(P) + \sum_{i=1}^l y_i A_i - \lambda \mathbb{1} \succeq 0 \end{aligned} \quad (7.42)$$

the corresponding Lagrangian with primal variable $Z \in S_n$ and domain D_Z is

$$\begin{aligned} \mathcal{L}(\lambda, \mathbf{y}, Z) &= -\lambda - \left\langle Z, \left(C(P) + \sum_{i=1}^l y_i A_i - \lambda \mathbb{1} \right) \right\rangle \\ &= -\lambda (1 + \langle Z, \mathbb{1} \rangle) - \langle Z, C(P) \rangle - \sum_{i=1}^l \langle Z, y_i A_i \rangle \end{aligned} \quad (7.43)$$

with Lagrange dual function

$$\begin{aligned} g(Z) &= \inf_{\lambda \in \mathbb{R}, \mathbf{y} \in \mathbb{R}^l} \{ \mathcal{L}(\lambda, \mathbf{y}, Z) \} \\ &= \begin{cases} -\langle Z, C(P) \rangle & \text{for } 1 - \langle Z, \mathbb{1} \rangle = 0, \langle Z, A_i \rangle = 0 \\ -\infty & \text{otherwise} \end{cases} \end{aligned} \quad (7.44)$$

Hence, the resulting **DSDP** of the above **PSDP** is then

$$\begin{aligned} \max_{Z \in \mathbb{R}^{t \times t}} \quad & -\langle Z, C(P) \rangle \\ \text{s.t.} \quad & \langle Z, \mathbb{1} \rangle = 1 \\ & \langle Z, A_i \rangle = 0 \quad \forall i = 1, \dots, l \\ & Z \succeq 0 \end{aligned} \quad (7.45)$$

Alternatively, one can formalise the above using vector quantities. Choose $\mathbf{c} = (-1, 0, \dots, 0) \in \mathbb{R}^{l+1}$, $\mathbf{y} = (\lambda, y_1, \dots, y_l) \in \mathbb{R}^{l+1}$ and $\mathbf{A} = (\mathbb{1}, A_1, \dots, A_l)$ then

$$\begin{aligned} \min_{\mathbf{y} \in \mathbb{R}^{l+1}} \quad & \langle \mathbf{c}, \mathbf{y} \rangle \\ \text{s.t.} \quad & C(P) + \langle \mathbf{y}, \mathbf{A} \rangle \succeq 0 \end{aligned} \quad (7.46)$$

so that

$$\begin{aligned} \max_{Z \in \mathbb{R}^{t \times t}} \quad & -\langle Z, C(P) \rangle \\ \text{s.t.} \quad & \langle Z, A_i \rangle = b_i \quad \forall i = 1, \dots, l+1 \\ & Z \succeq 0 \end{aligned} \quad (7.47)$$

with $b = (1, 0, \dots) \in \mathbb{R}^{l+1}$.

Remark. See [Appendix D](#) for a reformulation involving localising matrices.

Remark. While we do not attempt to prove that strong duality holds, we point out that the solver returns a vanishing duality gap in practical applications considered in this thesis.

7.2 Problem-specific programme

Let us now elaborate on the implementation of specific problems we like to consider.

7.2.1 Visibility tests

A given probability distribution P is incompatible with a causal structure under consideration. A visibility test is a way to quantify this incompatibility. Conversely, visibility tests certify the robustness of the quantum inflation procedure [\[87\]](#). Informally, given an incompatible distribution to a causal structure, how much can it be disturbed with *white noise* so that the quantum inflation procedure still certifies it as incompatible. The threshold at which an incompatible distribution mixed with white noise becomes compatible is the *visibility threshold*. Visibility tests allow for the verification of the functionality of independently developed implementations^{[27](#)}. In addition, comparing visibility bounds for probability distributions in classical and quantum variants of a given scenario offers a way to quantify the classical-quantum gap.

Formally, white noise is represented by the *discrete uniform distribution* (see [Appendix E](#)). The *noisy* probability distribution to be tested given a causal structure is then

$$P_v(a, b, c) = v \cdot P(a, b, c) + \frac{1 - v}{t}. \quad (7.48)$$

for $v \in [0, 1]$ and $t = |\{a, b, c\}_{a,b,c}|$. The noisy distribution reduces to the discrete uniform distribution for visibility $v = 0$ and to the original distribution for $v = 1$. In visibility tests, the cost matrix is a function of P and v , i.e. we replace $C(P)$ with $C(P, v)$ in the **(PSDP)** and **(DSDP)**. The dependency on v translates to the objective function, i.e. $\lambda(v)$. We are concerned with finding a root v^* to the function $\lambda(v)$. This requires us to solve the **(PSDP)** or **(DSDP)** for different values of v . Let us assume that the **(PSDP)** and **(DSDP)** are feasible, and to each v , the optimal is obtained.

²⁷While in principle a reasonable and standard procedure, it constitutes a Sisyphean-like task (although ultimately not indefinite) for incorrect references.

The function $\lambda(v)$ is continuous in v .

The *bisection method* in algorithm 5 is a root-finding procedure defined for such continuous functions. It constitutes a straightforward but robust optimisation procedure. The convergence tolerance is given by ϵ .

Algorithm 5 Bisection algorithm

Require: $\Gamma^{(k)}$, ϵ , $v_u = 1$, $v_l = 0$

while $v_u - v_l \geq \epsilon$ **do**

$$v_m = \frac{v_u + v_l}{2}$$

evaluate $C(P, v_m)$

solve SDP obtaining λ^*

if $\lambda^* \leq 0$ **then**

$$v_u = v_m$$

else

$$v_l = v_m$$

end if

end while

return v_m

7.2.2 Certificates and witnesses

When solving the (DSDP) from Eq. (7.46)

$$\begin{aligned} \max_{Z \in \mathbb{R}^{t \times t}} \quad & -\langle Z, C(P) \rangle \\ \text{s.t.} \quad & \langle Z, \mathbf{1} \rangle = 1 \\ & \langle Z, A_i \rangle = 0 \quad \forall i = 1, \dots, m \\ & Z \succeq 0 \end{aligned} \tag{7.49}$$

for any probability distribution P , we obtain an optimal point²⁸ Z^* and a lower bound on the optimal value p^* of the (PSDP)

$$-\langle Z^*, C(P) \rangle \leq p^*. \tag{7.50}$$

Let λ^* be the primal optimal point, so that $p^* = -\lambda^*$. For any incompatible P_{in} , $\lambda^* < 0$ and for any compatible P_{com} , $\lambda^* \geq 0$ so that

$$-\langle Z^*, C(P_{\text{com}}) \rangle \leq -\langle Z^*, C(P_{\text{in}}) \rangle. \tag{7.51}$$

²⁸Assuming that it is attained.

We obtain the strongest condition on compatible P_{com} via the inequality arising on the visibility threshold v^* . Let P_{in} be incompatible for $v \in \mathcal{V}_{\text{invisible}} = (v^*, 1]$ and Z_v^* the corresponding dual optimal point. For any $v_1, v_2 \in \mathcal{V}_{\text{invisible}}$ with $v_1 \leq v_2$

$$-\langle Z_{v_1}^*, C(P_{\text{in}}, v_1) \rangle \leq -\langle Z_{v_1}^*, C(P_{\text{in}}, v_2) \rangle. \quad (7.52)$$

By construction, at the threshold v^* , $\lambda^* = 0$ so that any compatible distribution necessarily adheres to

$$-\langle Z^*, C(P_{\text{com}}) \rangle \leq -\langle Z^*, C(P_{\text{in}}, v^*) \rangle = 0. \quad (7.53)$$

To make the inequality explicit, we need to revert $C(P_{\text{in}}, v^*)$ to a variable matrix in the ai-expressible moments y_j^{ai} with $j = 1, \dots, l$

$$C_y = C_{\text{const}} + \sum_{j=1}^l y_j^{\text{ai}} A_j \quad (7.54)$$

where the A_j are again the corresponding position matrices. The matrix C_{const} has a 1 at position $(1, 1)$ and zeros otherwise. In summary, Z^* is a matrix filled with numerical values obtained by solving the **(DSDP)**, and C_y is a variable matrix in the moments to the ai-expressible set. We thus obtain P 's *certificate*.

Definition 7.2 (P 's certificate). Any incompatible probability distribution gives rise to an inequality

$$\begin{aligned} -\langle Z^*, C_y \rangle &\leq 0 \\ \Leftrightarrow -\sum_{j=1}^l y_j^{\text{ai}} \langle Z^*, A_j \rangle &\leq \langle Z, C_{\text{const}} \rangle. \end{aligned} \quad (7.55)$$

When we replace the y_j^{ai} with the corresponding probabilities, we obtain an inequality in terms of probabilities. Any distribution compatible with $\Delta_{n,r}$ necessarily needs to satisfy this inequality called P 's certificate.

The violation of P 's certificate by an incompatible distribution $\tilde{P}$ is referred to as $\tilde{P}$ *witness* of incompatibility. Additionally, certificates provide an alternative algorithm to the bisection algorithm to obtain visibility thresholds.

Certificate optimisation: Consider $v_1 \leq v^* \leq v_2$ and their corresponding dual optimal points $Z_{v_1}^*, Z_{v_2}^*$ obtained when solving their respective **(DSDP)**. Then

$$-\langle C(P, v_1), Z_{v_1}^* \rangle \leq 0 \leq -\langle C(P, v_2), Z_{v_2}^* \rangle. \quad (7.56)$$

We start at v_2 and consider the following optimisation problem

$$\begin{aligned} \max_{v \in [0,1]} \quad & v \\ \text{s.t.} \quad & -\langle C(P, v), Z_{v_2}^* \rangle \leq 0 \\ & v \leq v_2 \end{aligned} \tag{7.57}$$

where $C(P, v)$ is a variable matrix in terms of v , not v_2 . Let us denote the optimal value by $v^\bullet$. Informally, alter v_2 until obtaining a certificate corresponding to a compatible distribution. Z_{v_2} is not the dual optimal point attained when solving the **(DSDP)** with $v^\bullet$. We need to iteratively solve the **(DSDP)** followed by the above optimisation until we obtain a $v^\bullet$ ϵ -close to the v from the previous iteration and resolving the **(DSDP)** yields a compatible distribution certificate. This leads to the algorithm [6](#)

Algorithm 6 Certificate optimisation

Require: $\Gamma^{(k)}$, ϵ , $v_u = 1.1$, $v_l = 1.0$
while $v_u - v_l \geq \epsilon$ **do**
 evaluate $C(v_l)$
 solve SDP obtaining $Z_{v_l}^*$
 $\min_{v \in [0,1]} -v$ s.t. $-\text{Tr}[C(v)Z_{v_l}^*] \leq 0$ and $v \leq v_l$
 Set $v_u = v_l$, $v_l = v^\bullet$
end while
return v_u

Remark. While this algorithm requires significantly fewer iterations to converge to a visibility threshold, it cannot be combined with all types of constraints we want to enforce. The bisection algorithm is universally applicable but far slower to converge to a visibility threshold given an equal tolerance.

7.2.3 Exploiting moment classifications

Some moments allow us to formulate additional constraints in the **(PSDP)**.

Non-negative moments: Alike our positive sets to define intermediate hierarchy levels, *non-negative moments* are those $y_w \in D_R(\Gamma^{(k)})$ corresponding to monomials $w \in \mathcal{U}^{(2k)}$ satisfying

$$w^* = w \wedge w^2 = w, \tag{7.58}$$

such that $y_w \geq 0$. Determining these moments in $D_R(\Gamma^{(k)})$ corresponds to algorithm [7](#). We collect all these non-negative moments in a set $\mathcal{Y}_{\text{nonneg}}$. The corresponding

(PSDP) is then

$$\begin{aligned}
& \min_{\lambda \in \mathbb{R}} -\lambda \\
\text{s.t. } & C(P, v) + \sum_{y \in \mathcal{Y}} y A_y \succeq 0 \\
& y \geq 0, \quad \forall y \in \mathcal{Y}_{\text{nonneg}}
\end{aligned} \tag{7.59}$$

with (DSDP)

$$\begin{aligned}
& \max_{Z \in \mathbb{R}^{t \times t}} -\langle Z, C(P) \rangle \\
\text{s.t. } & \langle Z, \mathbf{1} \rangle = 1 \\
& \langle Z, A_y \rangle \leq 0, \quad \forall y \in \mathcal{Y}_{\text{nonneg}} \\
& \langle Z, A_y \rangle = 0, \quad \forall y \in D_R(\Gamma^{(k)}) \cap \mathcal{Y}_{\text{nonneg}} \\
& Z \succeq 0.
\end{aligned} \tag{7.60}$$

Algorithm 7 Determining non-negative moments

Require: $D_R(\Gamma^{(k)}), \mathcal{Y}_{\text{nonneg}} = \emptyset$
for $y_w \in D_R(\Gamma^{(k)})$ **do**
 if $w^* = w$ **then**
 Add y_w to $\mathcal{Y}_{\text{nonneg}}$
 end if
end for
return $\mathcal{Y}_{\text{nonneg}}$

Scalar multiples: We return to these moments factoring into components, where we can link a proper subset of these components to the injectable set. Concretely, we refer to a moment $y_w = \prod_{i=1}^f y_{w_i}$ as a *scalar multiple* moment, if $w_i \in \mathcal{I}_{\Sigma^1}$ for some $w_i \in \{w_i\}_{i=1}^f$ but not all. Any scalar multiple moment y_w can then be written as

$$y_w = \alpha(P, v) \cdot y_u \tag{7.61}$$

where $\alpha(P, v) \in \mathbb{R}$ is some scalar value dependent on the probability distribution P at test and visibility v . Concretely, it is the product over the probabilities we assign the elements of the injectable set to which the factored monomials map. The monomial u corresponds to the product over the monomials in the factorisation, which do not map to $\mathcal{I}_{\Sigma^1}$ and thus remains unknown. Eq. (7.61) is a linear constraint compatible with our SDP formulation. We formulate the linear constraints corresponding to scalar multiple expressions of the form Eq. (7.61) by adapting our $\Gamma^{(k)}$ decomposition. Concretely, we

absorb position matrix A_w corresponding to scalar multiple $y_w = \alpha(C, v)y_u$ into the position matrix A_u to y_u via

$$(\alpha(P, v)A_w + A_u)y_u =: \tilde{A}_u y_u. \quad (7.62)$$

The procedure extends to any number of scalar multiples. The SDP construction and reformulation are analogous to the description above.

Remark. Identifying scalar multiple moments reduces the number of SDP variables. Not identifying y_w as a scalar multiple corresponds to an SDP, where both y_w and y_u are unknown variables. Replacing y_w with $\alpha(C, v)y_u$ reduces the number of free variables by one.

Remark. Including these scalar multiple moments, certificate optimisation in visibility tests and extraction of certificates and witnesses can no longer be done. We need to resort to the bisection algorithm. Hence, we refer to constraints of this type as *non-certificate type* constraints.

7.2.4 Scalar extension

Let us elaborate on the implementation of the scalar extension procedure. The implementation closely follows the description in [section 5](#). However, we need to respect the equivalence relations between moments. We simplify the implementation by considering positive sets. Thus, for all $w \in \mathcal{W}_k$ we have $w^* = w$.

Assume, that we constructed and reduced $\mathbf{R}^{(k)}$ to [Eq. \(7.39\)](#). To determine the set of moments eligible for the scalar extension procedure, we identify those unknown moments in $D_R(\Gamma^{(k)})$, which factorise into components. Recall that we can limit the discussion to factorisation degree two. Decide whether to implement scalar multiple moments; if so, exclude them; if not, include them in the consideration. Let us denote the set of components to the eligible moments by $\mathcal{F}$. We then partition $\mathcal{F}$ into sets $\mathcal{F}_0$, $\mathcal{F}_1$ and $\mathcal{F}_2$ corresponding to the three scalar extension cases. We have indexed according to the number of components with corresponding monomials in $\mathcal{W}_k$. To avoid confusion, let us limit the description of the implementation to $\mathcal{F}_2$. Recall, for both components with a corresponding monomial in $\mathcal{W}_k$; it suffices to introduce one scalar extension variable. Let $\mathcal{S}_{\mathcal{F}_2}$ correspond to this reduced set, i.e. $2 \cdot |\mathcal{S}_{\mathcal{F}_2}| = |\mathcal{F}_2|$. To each $y_w \in \mathcal{S}_{\mathcal{F}_2}$ we define a scalar extension variable α_w . Let Ω be the set of those scalar extension variables.

Next, we extract the first column $\mathcal{C}$ as a vector from $D_R(\Gamma^{(k)})$ and $D_S(\Gamma^{(k)})$. Note that the moments in $\mathcal{C}$ have been reduced to their representative under involution and

symmetry as described above. To each $\alpha \in \Omega$ we create an additional row by multiplying $\mathcal{C}$ with α yielding $\tilde{\Gamma}^{(k)}$. We then go through $D_R(\Gamma^{(k)})$ identifying all moments which can be equated to entries in this additional row. We conclude by identifying non-negative moments in this additional row. The resulting SDP is then

$$\begin{aligned} \text{find } y &\in \mathcal{Y}_{\text{SDP}} \cup \mathcal{S}_{\mathcal{F}_2} \\ \text{s.t. } \tilde{\Gamma}^{(k)} &\succeq 0 \end{aligned} \tag{7.63}$$

with $\dim(\tilde{\Gamma}^{(k)}) = \dim(\Gamma^{(k)}) + |\mathcal{S}_{\mathcal{F}_2}|$ and $\mathcal{Y}_{\text{SDP}}$ the unknown moments before the scalar extension.

7.3 Numerical implementation

We have opted to implement the quantum inflation hierarchy using the *Python* programming language. For one, it constitutes an open-source alternative while offering a wide range of convex optimisation packages necessary for this implementation.

To maintain autonomy over the properties of generators and thus generality in the application, we abstained from simply formulating the SDPs via Ncpol2sdpa [85] as in [87, 69] or NCSOStools [15]. Instead, we introduce a customised *ComSym* class inheriting from the *symbols class* from *Sympy* [49]. In addition, *Sympy* offers us the relevant tools to implement the scalar extension procedure and visibility tests. We use *Sympy*'s *symbols class* to represent the visibility v and scalar variables in scalar extension. We employ *Sympy*'s *lambdify* function to convert symbolic expressions to numerical values. *Scipy*'s [80] minimisation procedure is used to implement the certificate optimisation.

7.3.1 Solvers

Wrappers such as *CVXPY* [22, 1] or *Gurobi* [36] offer intuitive interfaces for formulating SDPs and passing them to a variety of solvers. This circumvents the need to formulate the SDP in the native languages of solvers. These are generally quite complex and differ greatly for each solver. We were able to obtain the *Tsirelson bound* of $2\sqrt{2}$ for the *Clauser-Horne-Shimony-Holt (CHSH) inequality* [19, 81] (and optimisation problem 46 in [65]) using *CVXPY* and the *splitting cone solver (SCS)* [58, 59]. Extending the procedure to the problem sizes encountered in quantum inflation was not viable. The former corresponded to a moment matrix of $\mathcal{O}(10^0)$ whereas the latter concerns itself with matrices of $\mathcal{O}(10^2)$ to $\mathcal{O}(10^3)$ [29]. We cannot discern whether these wrappers translate the SDPs incorrectly or rather inefficiently (very memory demanding) to a

²⁹In principle, there is no limit to the matrix sizes arising in quantum inflation. In practice, computing power limits the problems viable for numerical treatment.

chosen solver. We strongly advise against formulating SDPs arising from quantum inflation using wrappers. Given the necessity to treat large optimisation problems, we departed from the use of wrappers and set out to formulate the SDPs in the native language of the SCS solver. Such a *first-order solver* is in principle more capable of treating larger optimisation problems [57]. Unfortunately, achieving the required accuracy proved to be too time-intensive. Attempts to mend this SCS implementation by customising solver settings to the SDPs under consideration improved its performance. Although we could certify the W -distribution to be incompatible at $\mathbf{R}_C^{(2,3)}$, visibility tests did not return the required accuracy for v . We want to stress that not only did the calculations still take a tremendous amount of time ($\mathcal{O}(\text{days})$), but the sensitivity of the results on solver settings did not fill us with confidence. Ultimately, we decided to abandon this approach and try our luck with interior point solvers.

Remark. The difficulty in formulating and solving the SDPs arising from quantum inflation is common among researchers in the field. Implementations using *Mathematica* [40] or *Matlab* [48], while being very simple and concise do not offer the autonomy inherent to a native formulation in SDP solvers. This work contributes to a current undertaking in providing a general SDP solver framework for quantum inflation.

7.3.2 MOSEK Fusion and Optimizer API

We opted for the MOSEK solvers. While the *MOSEK Fusion API* (abbr. *Fusion*) [3] focuses on usability and compactness, the *MOSEK Optimizer API* (abbr. *Optimizer*) [4] far outperforms the former in terms of speed, accuracy and autonomy. We obtained satisfying results using the MOSEK Fusion API but finally settled on the MOSEK Optimizer API.

We set the Optimizer to employ an *interior point algorithm* (see [14, 4] for an explanation) as it is more predictable. We leave all solver settings at their default value. The Optimizer simultaneously solves the (**PSDP**) and (**DSDP**). The Optimizer framework offers the ability to modify different aspects of an SDP even after having constructed and solved it. This allows for an efficient implementation of visibility tests, where we are interested in modifying $C(P, v)$ in each iteration by variation of v . Additionally, Optimizer allows us to set individual bounds on variables in the (**DSDP**) allowing for the implementation of non-negative moments. For an SDP formulation in the Optimizer framework, see [Appendix F](#)

8 Numerical results

For an overview of the relevant joint probability distributions, see [Appendix E](#). The restriction to $\Delta_{2,k}$ is motivated by computational limitations arising in the implementation.

8.1 Visibility tests in quantum inflation

For our visibility test results with the W -distribution, see [Table 8.1](#). Accordingly, [Table 8.2](#) displays the results for the GHZ -distribution.

W -distribution						
	NPA		Local levels			Scalar multiples ³⁰
SDP	$\mathbf{R}_S^{(3)}$	$\mathbf{R}_S^{(3,+)}$	$\mathbf{R}_L^{(2,3)}$	$\mathbf{R}_L^{(2,4)}$	$\mathbf{R}_L^{(2,+)}$	$\mathbf{R}_L^{(2,+)}$
v^*	-	0.8038	0.8128	0.8039	0.8038	0.7734
$ \mathcal{W}_k $	2108	514	395	1175	343	343
$ \mathcal{Y}_{SDP} $	20078	5857	1627	9217	1069	1033

Table 8.1: Visibility thresholds v^* for the W -distribution. $|\mathcal{Y}_{SDP}|$ denotes the number of scalar variables in the SDP. SDP solver memory requirements for calculation of v^* to $\mathbf{R}_S^{(3)}$ exceeded our computer capabilities. Certificate optimisation with $\epsilon = 0.001$. Bisection algorithm with $\epsilon = 0.001$

GHZ -distribution						
	NPA		Local levels			Scalar multiples
SDP	$\mathbf{R}_S^{(3)}$	$\mathbf{R}_S^{(3,+)}$	$\mathbf{R}_L^{(2,3)}$	$\mathbf{R}_L^{(2,4)}$	$\mathbf{R}_L^{(2,+)}$	$\mathbf{R}_L^{(2,+)}$
v^*	-	0.4641	0.4641	0.4641	0.4641	0.4453

Table 8.2: Visibility thresholds v^* for the GHZ -distribution. We have omitted $|\mathcal{W}_k|$ and $|\mathcal{Y}_{SDP}|$, since they do not differ from [Table 8.1](#). Tolerances as in [Table 8.1](#).

8.2 Certificates and witnesses

Since the marginal probabilities of the W -distribution are of order $\mathcal{O}(10^{-1})$, we set the coefficients of order less than $\mathcal{O}(10^{-5})$ to zero. For the W -certificate, we choose to rescale the inequality by a factor of 100. Rounding down to the next integer value preserves the inequality, resulting in a more readable expression. The W -certificate from $\mathbf{R}_L^{(2,+)}$ is

$$T + \sigma(T) + \sigma'(T) \leq 0 \quad (8.1)$$

where

$$\begin{aligned}
T = & 4P(a=0) - 3P(a=0)P(a=0) + 4P(a=0, b=0) \\
& - 9P(a=0)P(b=0) + 8P(a=0, b=0)P(a=0, b=0) \\
& - 5P(a=0, b=0)P(a=0) - 5P(a=0, b=0)P(b=0) \\
& + 13P(a=0)P(b=0)P(c=0)
\end{aligned} \tag{8.2}$$

and $\sigma = [A \rightarrow B \rightarrow C \rightarrow A]$ and $\sigma' = [A \rightarrow C \rightarrow B \rightarrow A]$ such that for example $\sigma(P(a=0)) = P(b=0)$.

Remark. The lefthand side of [Eq. \(8.1\)](#) has 22 terms.

W -witness of incompatibility is 0.1687. For a conversion of [Eq. \(8.1\)](#) to expectation values, see [Appendix G](#).

8.3 Scalar extension in quantum inflation

We did not improve on any of the above numerical results by exploiting additional constraints from applying scalar extension to quantum inflation.

9 Conclusions

In this thesis, we focused on inflation procedures to answer problems arising in causality. Concretely, we introduced the quantum inflation procedure to tackle the QCCP and related it via NPO theory to SDP relaxations. In our NPO formulation, we generalised the concept of ai-expressible sets from classical to quantum inflation and elaborated on how to exploit the quotient algebra to reduce the resulting SDP formulations significantly. We showed how to construct, reduce and reformulate these SDP relaxations to numerically quantify the incompatibility of several probability distributions with the quantum triangle scenario. Our results on visibility thresholds cleared up several inconsistencies in the quantum inflation community. In [87] the visibility threshold of the W -distribution for $\mathbf{R}_{\mathcal{L}}^{(2,3)}$ was determined to be $v = 0.8039$. In correspondence with the authors of [87], we corrected its value to $v = 0.8128$. In addition, introducing the intermediate hierarchy levels corresponding to positive sets, allowed us to determine a visibility threshold $v = 0.8038$ at a much smaller SDP $\mathbf{R}_{\mathcal{L}}^{(2,+)}$ than compared to $\mathbf{R}_{\mathcal{L}}^{(2,4)}$. To our knowledge, the visibility threshold of $v = 0.7734$ resulting from $\mathbf{R}_{\mathcal{L}}^{(2,+)}$ together with scalar multiples constitutes the lowest threshold for the W -distribution obtained via quantum inflation. Therefore, our lower visibility value was obtained from a much smaller SDP. Accordingly, for the GHZ -distribution we obtained a visibility threshold of $v = 0.4453$ at $\mathbf{R}_{\mathcal{L}}^{(2,+)}$ with scalar multiples. While this value $v = 0.4318$ provides a worse upper bound on the actual visibility threshold when compared to $v = 0.4318$ [68, Table 5.2], we note that the latter was obtained from inflation level $n = 3$. Our incapability to obtain a visibility threshold to $\mathbf{R}_{\mathcal{S}}^{(3)}$ shows how essential the reductions and intermediate hierarchy levels are to solving the QCCP via quantum inflation and NPO theory. Furthermore, we exploited SDP duality theory to provide the W -certificate and W -witness from $\mathbf{R}_{\mathcal{L}}^{(2,+)}$. The rescaling of certificates and witnesses in [87] makes comparison difficult.

Proposing a generalisation of the scalar extension procedure to arbitrary factorisation degrees and extending its application to quantum inflation scenarios was of mere theoretical relevance. The procedure did not yield improvements in any of our numerical results.

9.1 Discussion and outlook

On request, we learned that the authors of [87] obtained the visibility thresholds $v = 0.8039$ for the W -distribution via the hexagonal non-fanout inflation from [34]. We aim to better understand these non-fanout inflations and their connection to our

intermediate hierarchy level $\mathbf{R}_{\mathcal{L}}^{(2,+)}$. For probability distributions symmetric under permutation of the observed variables, we can formulate corresponding constraints in the NPO problem to the QCCP [71]. We wonder whether adding these constraints yield lower visibility thresholds for the W - and GHZ -distribution.

Scalar extension failed to yield lower visibility thresholds in the quantum triangle scenario. We ask whether quantum models exist where scalar extension delivers more constraint SDPs translating to lower visibility thresholds. We added non-negativity constraints on certain moments arising in scalar extension. We believe these restrictions were not imposed in [69] and are interested in how the results obtained vary with these additional constraints.

A novel implementation of the LGG hierarchy stands at the top of our list of future projects in the study of the QCCP. Additionally, we wonder to what extent our generalisation of the scalar extension procedure extends to the LGG hierarchy and its implementation. Such an implementation necessitates an adequate understanding of SDP solvers to cope with the computational complexity in solving the corresponding SDPs. Our python implementation allowed us to retain autonomy over all aspects of the implementation. However, we note that this implementation has potential for improvements in computing times and memory requirements. We contributed to a much faster implementation using *Mathematica*. Such improvements are not a priority. The inhibiting factor in calculating lower upper bounds on visibility thresholds is the SDP solver and its memory requirements. Running our calculation on more capable computers potentially yields stronger numerical results. Instead, *alternative direction method of multipliers (ADMM) solvers*, as opposed to our interior point solvers, not only allow for *warm starts* but are generally better suited at handling large-scale SDPs [60]. We are currently working on adapting such a solver to SDPs arising from quantum inflations. Furthermore, NPO theory admits a block decomposition of $\Gamma^{(k)}$ [50, 73]. Respecting this structure potentially allows our MOSEK Optimizer implementation to efficiently dissect the SDP problem and thus lowering memory requirements.

A Dual formulation for an SDP in standard form

Recall, the SDP in standard form

$$\begin{aligned} \min_{X \in \mathcal{D}} \quad & \langle C, X \rangle \\ \text{s.t.} \quad & \langle A_i, X \rangle = b_i, \quad i = 1, \dots, m \\ & X \succeq 0 \end{aligned} \tag{A.1}$$

where $C, A_i \in \mathcal{S}^n$ for $i = 1, \dots, m$ and $\langle \cdot, \cdot \rangle$ denotes the Frobenius inner product. The Lagrangian reads

$$\begin{aligned} \mathcal{L}(X, \mathbf{y}, Z) &= \langle C, X \rangle - \langle Z, X \rangle + \sum_{i=1}^m y_i (-\langle A_i, X \rangle + b_i) \\ &= \langle b, \mathbf{y} \rangle + \langle C + \sum_{i=1}^m (-A_i) - Z, X \rangle \end{aligned} \tag{A.2}$$

with dual variables $\mathbf{y} \in \mathbb{R}^m$ and $Z \in \mathcal{S}_+^n$. With the dual function is then

$$\begin{aligned} g(\mathbf{y}, Z) &= \inf_{X \in \mathcal{S}_+^n} \mathcal{L}(X, \mathbf{y}, Z) \\ &= \begin{cases} \langle b, \mathbf{y} \rangle & \text{for } C - \sum_{i=1}^m A_i = Z \\ -\infty & \text{otherwise} \end{cases} \end{aligned} \tag{A.3}$$

and Lagrange dual problem

$$\max_{Z \in \mathcal{S}_+^n} g(Z). \tag{A.4}$$

Making the constraints explicit and eliminating the slack variable Z results in the SDP in the inequality form given in Eq. (2.41)

B Scalar extension case (3) visualisation

$$\tilde{\Gamma}^{(k)} = \begin{matrix} & \mathbb{1} & \cdots & u_2 & \cdots & v_1 & \cdots & v_2 & \cdots & g_1 & g_2 \\ \begin{matrix} \mathbb{1} \\ \vdots \\ v_1^* \\ \vdots \\ v_2^* \\ \vdots \\ u_1^* \\ \vdots \\ g_1^* \\ g_2^* \end{matrix} & \left(\begin{array}{cccccccccc} & & & & & & & & & y_{g_1} & y_{g_2} \\ & & & & & & & & & & \\ & & & & & & & y_v & & & \\ & & & & & & & & y_{u^*v} & & \\ & & & & & & & & & & \\ & & & & & & & & & & \\ & & & & & y_{u^*} & & & & & \\ & & & & & & & & & & \\ & & & & & & & & & y_{g_1^*g_2} & \end{array} \right) \end{matrix} \quad (\text{B.1})$$

C Pseudo code $D_R\Gamma^{(k)}$ and $D_S(\Gamma^{(k)})$ construction

Let us specify on the construction³¹ of $\mathcal{D}_R(\Gamma^{(k)})$ and $\mathcal{D}_S(\Gamma^{(k)})$. Consider ordered monomial basis $\mathcal{W}_k$ and let $\mathcal{W}_k[i]$ be the i -th element in $\mathcal{W}_k$. Then

Algorithm 8 $\mathcal{D}_R(\Gamma^{(k)})$ and $\mathcal{D}_S(\Gamma^{(k)})$ construction

Require: $\mathcal{W}_k$

```

for  $a = 0$  to  $|\mathcal{W}_k|$  do
  for  $b = 0$  to  $a + 1$  do
     $w := \mathcal{N}_{\mathcal{R}}(\mathcal{W}_k[a] \cdot \mathcal{W}_k[b])$ 
    if  $w \notin [w^*]_{\sim}$  then
      if  $w <_{\mathcal{M}} \mathcal{N}_{\mathcal{R}}(w^*)$  then
        if  $y_w \in D_R$  then
           $D_S[(a, b)] = D_R[y_w]$ 
        else
           $D_R[y_w] = (a, b)$ 
        end if
      else
        if  $y_{\mathcal{N}_{\mathcal{R}}(w^*)} \in D_R$  then
           $D_S[(a, b)] = D_R[y_{\mathcal{N}_{\mathcal{R}}(w^*)}]$ 
        else
           $D_R[y_{\mathcal{N}_{\mathcal{R}}(w^*)}] = (a, b)$ 
        end if
      end if
    else
      if  $y_w \in D_R$  then
         $D_S[(a, b)] = D_R[(a, b)]$ 
      else
         $D_R[y_w] = (a, b)$ 
      end if
    end if
  end for
end for

```

Remark. We have additionally included the involution equivalence when constructing $\mathcal{D}_R(\Gamma^{(k)})$ and $\mathcal{D}_S(\Gamma^{(k)})$.

Remark. The order in $\mathcal{W}_k$ is irrelevant. Simultaneously exchanging rows and columns does not alter the set of feasible solutions [78].

³¹We compute the lower triangular part of $\Gamma^{(k)}$ as required by the SDP solver we employ.

D PSDP and DSDP with localising matrix Λ

Let us consider a localising matrix Λ . The SDP formulation then readily extends to multiple localising matrices. Analogue to $\Gamma^{(k)} \in \mathbb{R}^{t \times t}$, we define $\{A_i\}_{i=1}^m$ with $\forall A_i \in S_n$ and $K(P) \in S_n$ so that

$$\Lambda = K(P) + \sum_{i=1}^m y_i B_i \quad (\text{D.1})$$

resulting in

$$\begin{aligned} \min_{\lambda \in \mathbb{R}} \quad & -\lambda \\ \text{s.t.} \quad & C(P) + \sum_{i=1}^m y_i A_i - \lambda \mathbf{1} \succeq 0 \\ & K(P) + \sum_{i=1}^m y_i B_i \succeq 0. \end{aligned} \quad (\text{D.2})$$

The Lagrangian with additional dual variable $W \in S_n$ and domain D_W is then

$$\begin{aligned} L(\lambda, \mathbf{y}, Z, W) &= -\lambda - \left\langle Z, \left(C(P) + \sum_{i=1}^m y_i A_i - \lambda \mathbf{1} \right) \right\rangle - \left\langle W, \left(K(P) + \sum_{i=1}^m y_i B_i \right) \right\rangle \\ &= -\lambda (1 + \langle Z, \mathbf{1} \rangle) - \langle Z, C(P) \rangle - \sum_{i=1}^m (\langle Z, y_i A_i \rangle - \langle W, y_i B_i \rangle) - \langle W, K(P) \rangle \end{aligned} \quad (\text{D.3})$$

resulting in the Lagrange dual function

$$\begin{aligned} g(Z, W) &= \inf_{\lambda \in \mathbb{R}, \mathbf{y} \in \mathbb{R}^m} \{ \mathcal{L}(\lambda, \mathbf{y}, Z, W) \} \\ &= \begin{cases} -\langle Z, C(P) \rangle - \langle W, K(P) \rangle & \text{for } 1 - \langle Z, \mathbf{1} \rangle = 0, \langle Z, A_i \rangle = 0, \langle W, B_i \rangle = 0 \\ -\infty & \text{otherwise} \end{cases} \end{aligned} \quad (\text{D.4})$$

and finally the resulting dual SDP

$$\begin{aligned}
& \max_{Z \in \mathbb{R}^{t \times t}, W \in \mathbb{R}^{t \times t}} - (\langle Z, C(P) \rangle + \langle W, K(P) \rangle) \\
& \text{s.t.} \quad \langle Z, \mathbb{1} \rangle = 1 \\
& \quad \langle Z, A_i \rangle = 0 \quad \forall i = 1, \dots, m \\
& \quad \langle W, B_i \rangle = 0 \quad \forall i = 1, \dots, m \\
& \quad Z \succeq 0 \\
& \quad W \succeq 0
\end{aligned} \tag{D.5}$$

For r localising matrices you obtain then analogously r dual variables. The C -bound on operators in the LGG hierarchy can be collected in a sum and thus formulated in one psd constraint on a localising matrix.

E Probability distributions

Restrict to two-output observed random variables and denote the outputs by $\{0, 1\}$. We list the relevant probability distributions and their marginals below. Let i, j, k take on distinct element from $\{a, b, c\}$.

- *Discrete uniform distribution*

$$P_{uniform}(a, b, c) = \frac{1}{k} \quad (\text{E.1})$$

with $k = 8$ cardinality of set $U = \{(a, b, c) \text{ with } a, b, c = 0, 1\}$.

$$P(i = 0) = \frac{1}{2}, \quad P(i = 0, j = 0) = \frac{1}{4}, \quad P(i = 0, j = 0, k = 0) = \frac{1}{8} \quad (\text{E.2})$$

- *W-distribution*

$$P_W(a, b, c) = \begin{cases} \frac{1}{3} & a + b + c = 1, \\ 0 & \text{otherwise} . \end{cases} \quad (\text{E.3})$$

$$P(i = 0) = \frac{2}{3}, \quad P(i = 0, j = 0) = \frac{1}{3}, \quad P(i = 0, j = 0, k = 0) = 0 \quad (\text{E.4})$$

- *Greenberger–Horne–Zeilinger-distribution (GHZ)*

$$P_{GHZ}(a, b, c) = \begin{cases} \frac{1}{2} & a = b = c, \\ 0 & \text{otherwise} \end{cases} \quad (\text{E.5})$$

$$P(i = 0) = \frac{1}{2}, \quad P(i = 0, j = 0) = \frac{1}{2}, \quad P(i = 0, j = 0, k = 0) = \frac{1}{2} \quad (\text{E.6})$$

F MOSEK Optimiser API SDP formulation [4]

Let $\mathcal{S}^r$ denote real $r \times r$ symmetric matrices and $\mathcal{S}_+$ real and symmetric psd matrices. The Optimizer solves SDP's in the general form [4]

$$\begin{aligned}
& \min && \sum_{j=0}^{n-1} c_j x_j + \sum_{j=0}^{p-1} \langle C_j, Z_j \rangle + c^f \\
& \text{subject to} && l_i^c \leq \sum_{j=0}^{n-1} a_{ij} x_j + \sum_{j=0}^{p-1} \langle A_{ij}, X_j \rangle \leq u_i^c, \quad i = 0, \dots, m-1, \\
& && l_j^x \leq x_j \leq u_j^x, \quad j = 0, \dots, n-1, \\
& && x \in \mathcal{K}, X_j \in \mathcal{S}_+^{r_j}, \quad j = 0, \dots, p-1
\end{aligned} \tag{F.1}$$

with primal variable matrix Z_j , cost matrix C and coefficient matrices $A_{i,j}$. The matrix inner-product for $A, B \in \mathbb{R}^{m \times n}$ is

$$\langle A, B \rangle := \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} A_{ij} B_{ij}. \tag{F.2}$$

We choose $p = 1$ restricting to one SDP variable and will drop the corresponding index. We also do not provide any c, x, c^f . The corresponding (**DS**DP) is then

$$\begin{aligned}
& \max && (l^c)^T s_l^c - (u^c)^T s_u^c \\
& \text{subject to} && A^T y = c \\
& && -y + s_l^c - s_u^c = 0 \\
& && C - \sum_{i=0}^{m-1} y_i A_i = S \\
& && s_l^c, s_u^c \geq 0 \\
& && S \in \mathcal{S}_+^r
\end{aligned} \tag{F.3}$$

where S, s_l^c, s_u^c are the dual slack variables introduced by the Optimizer.

G Computer science vs. physics notation

As elaborated in [66] in the “computer scientist” notation outputs take values in $\{0, 1\}$, whereas in the “physicist” notation outputs take values in $\{-1, +1\}$.

$$\begin{aligned}
 p(i = 0) &= \frac{1}{2} (1 + \langle I \rangle_{\varrho}) \\
 p(i = 0, j = 0) &= \frac{1}{4} (1 + \langle I \rangle_{\varrho} + \langle J \rangle_{\varrho} + \langle IJ \rangle_{\varrho}) \\
 p(i = 0, j = 0, k = 0) &= \frac{1}{8} (1 + \langle I \rangle_{\varrho} + \langle J \rangle_{\varrho} + \langle IJ \rangle_{\varrho} + \langle IK \rangle_{\varrho} + \langle JK \rangle_{\varrho} + \langle IJK \rangle_{\varrho}) .
 \end{aligned}
 \tag{G.1}$$

Remark. The “physicist” notation is called “expectation-value picture” in [87]. We abstain from using this terminology, as we find it confusing considering that both notations have expectation values.

H Source code python implementation

```
1 # -*- coding: utf-8 -*-
2 """
3 Quantum Inflation procedure for the quantum triangle scenario. We
4 specifically consider inflation level 2. We consider
5 different SDP hierarchies and intermediate steps thereof. We answer
6 the QCCP for the W- and GHZ-distributions.
7 Visibility test quantifying incompatibility of these distributions.
8 Certificates and witnesses yielding necessary
9 conditions for any compatible distribution. Non-certificate
10 constraints for visibility tests.
11
12 Scalar extension adaptation for quantum inflation.
13
14 Generalisations to higher inflation levels are possible.
15
16 @author: Julius Alexander Zeiss
17 MOSEK OPTIMIZER API in collaboration with Elie Wolfe and Bora Ulu.
18 """
19
20 import sympy as sym
21 import itertools as it
22 import time
23 import mosek
24 import sys
25 import numpy as np
26 import sparse
27
28 sym.init_printing(wrap_line=False)
29
30 start = time.time()
31
32 class ComSym(sym.Symbol):
33     """ The fundamental object of the implementation is a custom class
34     ComSym inheriting from Sympy.Symbols modelling
35     generators. The default instance is non-commutative and a
36     projector. Overloading __mul__ and __pow__ class methods
37     to treat the multiplication and powers of generators. With a
38     modified bubble sort algorithm we reduce any monomial
39     in the generators to its equivalence class representative by
40     iteratively comparing pairs of ComSym instances."""
41
42     def __new__(cls, name):
```

```

36     """Defining a new subclass inheriting from Sympy.Symbols with
the desired modifications.
37     The default instance is non-commutative. The generator is
encoded in the string name. Concretely, we have
38      $E^{i,j}_a$  is encoded as 1_ij_0a,  $F^{i,j}_b$  as 2_ij_0b and
 $G^{i,j}_c$  as 3_ij_0c."""
39
40     obj = super().__new__(cls, name, commutative=False)
41     return obj
42
43     def __mul__(self, other):
44         """Overloading __mul__ class method to reduce any monomial to
its equivalence class representative."""
45
46         if self == 0 or other == 0:
47             return 0
48
49         """Multiplication with sympy.Mul objects"""
50         if isinstance(other, sym.Mul) or isinstance(self, sym.Mul):
51             if self == 1:
52                 prod = other
53             elif other == 1:
54                 prod = self
55             else:
56                 prod = sym.Symbol.__mul__(self, other)
57
58         """Modified Bubble Sort Algorithm. In each step of the
iteration two ComSym instances are chosen and passed
59         to ComSym.__mul__ where commutativity and projectiveness
are being enforced."""
60         arglist_sort = list(prod.args)
61         while True:
62             i = 0
63             while i < len(arglist_sort) - 1:
64                 slice_prod = ComSym.__mul__(arglist_sort[i],
arglist_sort[i + 1])
65                 if slice_prod == 0:
66                     return 0
67                 is_mul = isinstance(slice_prod, sym.Mul)
68                 arglist_sort[i:i + 2] = slice_prod.args if is_mul
else [slice_prod]
69                 i += 1
70
71         """Modification"""
72         new_prod = sym.Mul(*arglist_sort)

```

```

73         if prod == new_prod:
74             return new_prod
75         prod = new_prod
76
77         """ComSym instance multiplied with 1"""
78         if self == 1:
79             return other
80         elif other == 1:
81             return self
82
83         """Projector rule for ComSym instances"""
84         if other == self:
85             return self
86
87         elif isinstance(other, ComSym):
88             """Multiplication of two ComSym instances with sorting for
89             commuting instances"""
90             # Orthogonality condition
91             # Check for same party operator and same subalgebra spaces
92             if self.name[0] == other.name[0]:
93                 if self.name[2] != other.name[2] and self.name[3] !=
94                 other.name[3]:
95                     # No overlapping subalgebra indices
96                     if self.name > other.name:
97                         return sym.Symbol.__mul__(other, self)
98
99                     return sym.Symbol.__mul__(self, other)
100
101             """Well order on generators"""
102             if self.name > other.name:
103                 return sym.Symbol.__mul__(other, self)
104
105             return sym.Symbol.__mul__(self, other)
106
107     def __pow__(self, exponent, mod=None):
108         """Modifying the __pow__ method. Each operator is a projector"
109         ""
110         return self
111
112 def var_list_creation(num_party, inflation_lvl, num_outcomes):
113     """Creating list of variables in X_ij_0k with X=1,2,3 for E,F,G
114     notation according to the number
115     of parties involved, inflation level and the number of outcomes.
116     The dictionary collects the generators by party

```

```

113     correspondance."""
114
115     creation_list = []
116     creation_dic = {}
117
118     for i in range(1, num_party + 1):
119         help_lst = []
120         for j in range(1, inflation_lvl + 1):
121             for k in range(1, inflation_lvl + 1):
122                 for r in range(1, num_outcomes + 1):
123                     help_lst.append(ComSym(str(i) + '_' + str(j) + str
124 (k) + '_0' + str(r)))
125                 creation_list.extend(help_lst)
126                 creation_dic[i] = help_lst
127
128     return creation_list, creation_dic
129
130 def erase_duplicates(self):
131     """Method, which eliminates duplicates from lists."""
132
133     return list(dict.fromkeys(self))
134
135
136 def union(lst1, lst2):
137     """Method, unifying lists, erasing duplicates and returning a
138 sorted list."""
139
140     final_list = list(set(lst1) | set(lst2))
141     return sorted(final_list)
142
143 def flatten(t):
144     """Method to flatten lists"""
145     return [item for sublist in t for item in sublist]
146
147
148 def comsym_tpl(self):
149     """Transforming ComSym instances into their tuple representatives,
150 e.g. 1_12_01 = (1,1,2,1). Not for monomials."""
151
152     help_lst = []
153
154     for i in range(0, len(self.name)):
155         if self.name[i] != '_' and self.name[i] != '0':

```

```

155         help_lst.append(int(self.name[i]))
156     return tuple(help_lst)
157
158
159 def inv(self):
160     """Involution on a monomial instance, i.e. reversing the order and
161     then sorting."""
162
163     if isinstance(self, sym.Mul):
164         arglist = list(self.args)
165         arglist_rev = reversed(arglist)
166         var = sym.Mul(*arglist_rev)
167         var = ComSym.__mul__(1, var)
168         return var
169     else:
170         return self
171
172 def construction_vector(var_dic, cutoff, inflation_lvl, num_party,
173     sdp_step, npa_level, positive_set):
174     """Method, constructing the generating monomials, i.e. the basis.
175     Depending on the choice of partitioning we
176     obtain a basis corresponding to NPA or local levels. Additionally,
177     intermediate steps in these hierarchies
178     can be constructed via cutoff or positive sets."""
179
180     if npa_level:
181         """NPA levels"""
182         print('NPA level', sdp_step)
183
184         var_list = []
185         for key, value in var_dic.items():
186             var_list.extend(value)
187
188         help_list = []
189         for i in range(1, sdp_step+2):
190             help_list.extend(it.product(var_list, repeat=i))
191
192         help_list_2 = []
193         for element in help_list:
194             if len(element) >= 2:
195                 prod = sym.Mul(*element[1:])
196                 help_list_2.append(ComSym.__mul__(element[0], prod))
197             else:
198                 help_list_2.append(element[0])

```

```

196
197     help_list_2 = erase_duplicates(help_list_2)
198
199     if cutoff != inflation_lvl*num_party:
200         final_list = []
201         """Cutoff"""
202         for x in help_list_2:
203             if len(x.args) <= cutoff:
204                 final_list.append(x)
205     else:
206         final_list = help_list_2
207
208 else:
209     """Local levels"""
210     print('Local level', sdp_step)
211
212     for key, value in var_dic.items():
213         var_dic[key] = value + value
214     var_perm_dic = {}
215     for key, value in var_dic.items():
216         help_lst = [(1,)]
217         for i in range(1, sdp_step+1):
218             help_lst.extend(it.permutations(value, i))
219         var_perm_dic[key] = erase_duplicates(help_lst)
220
221     mul_dic = {}
222     for key, value in var_perm_dic.items():
223         help_lst = []
224         for element in value:
225             if len(element) >= 2:
226                 prod = sym.Mul(* element[1:])
227                 help_lst.append(ComSym.__mul__(element[0], prod))
228             else:
229                 help_lst.append(element[0])
230
231         mul_dic[key] = erase_duplicates(help_lst)
232
233     help_lst = []
234     for value in mul_dic.values():
235         help_lst.append(value)
236
237     if cutoff != inflation_lvl*num_party:
238         """Cutoff"""
239         print('Cutoff: ', cutoff)
240         help_list_2 = list(it.product(*help_lst))

```



```

241         final_list = []
242         for el in help_lst_2:
243             prod = sym.Mul(*el)
244             arglist = list(prod.args)
245             if len(arglist) <= cutoff:
246                 final_list.append(sym.Mul(*el))
247         else:
248             """No cutoff"""
249             help_lst_2 = list(it.product(*help_lst))
250             final_list = []
251             for el in help_lst_2:
252                 final_list.append(sym.Mul(*el))
253
254     if positive_set:
255         """Positive set"""
256         print('Positive set')
257         positive_mon_list = []
258         for key in final_list:
259             if inv(key) == key:
260                 positive_mon_list.append(key)
261
262         final_list = positive_mon_list
263
264     print('Cardinality of set of generating monomials:', len(
265 final_list))
266     print('
-----
')
267
268     return final_list
269
270 def moment_matrix_construction(self):
271     """Constructing the lower triangular moment matrix dictionary
272 from the construction vector and its involution.
273 In the dictionary dic_pos the keys are the moment representatives
274 appearing in Gamma and the values correspond to
275 the position of their first appearance in the Gamma construction,
276 e.g. for monomial w {w:(a,b)}. A second
277 dictionary mm_dic contains the information on equivalence classes
278 on moments. In mm_dic the keys are positions in
279 Gamma, which we equate to positions which are values in dic_pos,
280 e.g. {(c, d): (a,b)} since y_w at (c,d) and
281 (a,b). We enforce involution equivalence."""

```

```

278     dic_pos = {}
279     mm_dic = {}
280
281     """Creating the involution of the generating monomials."""
282     w_inv = []
283     for a in self:
284         w_inv.append(inv(a))
285
286     for a in range(0, len(self)):
287         for b in range(0, a+1):
288             prod = ComSym.__mul__(w_inv[a], self[b])
289
290             if isinstance(prod, sym.Mul):
291                 prod_inv = inv(prod)
292                 if prod_inv != prod:
293                     """Choose a representative of involution
equivalence"""
294                     arg_prod = list(prod.args)
295                     arg_inv_prod = list(prod_inv.args)
296                     for i in range(0, len(arg_prod)):
297                         if arg_prod[i].name < arg_inv_prod[i].name:
298                             prod = prod
299                             break
300                     elif arg_prod[i].name > arg_inv_prod[i].name:
301                         prod = prod_inv
302                         break
303
304             try:
305                 dic_pos[prod]
306             except KeyError:
307                 dic_pos[prod] = (a, b)
308
309             value_pos = dic_pos.get(prod)
310             if (a, b) != value_pos:
311                 mm_dic[(a, b)] = value_pos
312
313     return mm_dic, dic_pos
314
315
316 def base_set(dic):
317     """Constructing the set of all entries to monomials over
generators indexed exclusively in subalgebra indices i,
318     with i = 1,...,n. This corresponds to the identifiable monomials
in the quantum inflation paper."""
319

```

```

320     test_list = []
321     return_list = []
322     for key in dic.keys():
323         if isinstance(key, sym.Mul):
324             i = 0
325             for letter in list(key.args):
326                 i += 1
327                 if letter.name[2] != letter.name[3]:
328                     break
329                 if i == len(key.args):
330                     test_list.append(key)
331         elif isinstance(key, ComSym):
332             if key.name[2] == key.name[3]:
333                 test_list.append(key)
334
335     test_list = erase_duplicates(test_list)
336
337     for el in test_list:
338         return_list.append(dic.get(el))
339
340     return return_list
341
342
343 def permutation_dic(self, ilvl, tpl_cs):
344     """The transformation group to inflation level 2."""
345
346     pi_dic_1, pi_dic_2, pi_dic_3, pi_dic_4, pi_dic_5, pi_dic_6,
347     pi_dic_7 = ({}, for i in range(7))
348
349     for l in self:
350         v = comsym_tpl(l)
351         if tpl_cs.get(pi_1(v, ilvl)) != 1: pi_dic_1[l] = tpl_cs.get(
352             pi_1(v, ilvl))
353         if tpl_cs.get(pi_2(v, ilvl)) != 1: pi_dic_2[l] = tpl_cs.get(
354             pi_2(v, ilvl))
355         if tpl_cs.get(pi_3(v, ilvl)) != 1: pi_dic_3[l] = tpl_cs.get(
356             pi_3(v, ilvl))
357         if tpl_cs.get(pi_2(pi_1(v, ilvl), ilvl)) != 1: pi_dic_4[l] =
358             tpl_cs.get(pi_2(pi_1(v, ilvl), ilvl))
359         if tpl_cs.get(pi_3(pi_1(v, ilvl), ilvl)) != 1: pi_dic_5[l] =
360             tpl_cs.get(pi_3(pi_1(v, ilvl), ilvl))
361         if tpl_cs.get(pi_3(pi_2(v, ilvl), ilvl)) != 1: pi_dic_6[l] =
362             tpl_cs.get(pi_3(pi_2(v, ilvl), ilvl))
363         if tpl_cs.get(pi_3(pi_2(pi_1(v, ilvl), ilvl), ilvl)) != 1:
364             pi_dic_7[l] = tpl_cs.get(pi_3(pi_2(pi_1(v, ilvl),

```

```

357         ilvl), ilvl))
358
359     pi_dic = {1: pi_dic_1, 2: pi_dic_2, 3: pi_dic_3, 4: pi_dic_4, 5:
pi_dic_5, 6: pi_dic_6, 7: pi_dic_7}
360
361     return pi_dic
362
363
364 def pi_1(self, inflation_lvl):
365     """Method, when getting passed a ComSym instance in tuple
representation returns the ComSym instance in tuple
366     representation, which is equivalent under permutation pi. This
means first subalgebra index for E variables and
367     second subalgebra index for G variables are changed."""
368
369     new_list = []
370     new_list.append(self[0])
371     if self[0] == 1:
372         if self[1] + 1 <= inflation_lvl:
373             new_list.append(self[1] + 1)
374         else:
375             new_list.append(1)
376             new_list.append(self[2])
377     elif self[0] == 3:
378         new_list.append(self[1])
379         if self[2] + 1 <= inflation_lvl:
380             new_list.append(self[2] + 1)
381         else:
382             new_list.append(1)
383     else:
384         return self
385     new_list.append(self[3])
386
387     return tuple(new_list)
388
389
390 def pi_2(self, inflation_lvl):
391     """Method, when getting passed a ComSym instance in tuple
representation returns the ComSym instance in tuple
392     representation, which is equivalent under permutation pi'. This
means second subalgebra index for E variables and
393     first subalgebra index for F variables are changed."""
394
395     new_list = []

```

```

396     new_list.append(self[0])
397     if self[0] == 2:
398         if self[1] + 1 <= inflation_lvl:
399             new_list.append(self[1] + 1)
400         else:
401             new_list.append(1)
402             new_list.append(self[2])
403     elif self[0] == 1:
404         new_list.append(self[1])
405         if self[2] + 1 <= inflation_lvl:
406             new_list.append(self[2] + 1)
407         else:
408             new_list.append(1)
409     else:
410         return self
411     new_list.append(self[3])
412
413     return tuple(new_list)
414
415
416 def pi_3(self, inflation_lvl):
417     """Method, when getting passed a ComSym instance in tuple
418     representation returns the ComSym instance in tuple
419     representation, which is equivalent under permutation pi''. This
420     means second subalgebra index for F variables and
421     first subalgebra index for G variables are changed."""
422
423     new_list = []
424     new_list.append(self[0])
425     if self[0] == 3:
426         if self[1] + 1 <= inflation_lvl:
427             new_list.append(self[1] + 1)
428         else:
429             new_list.append(1)
430             new_list.append(self[2])
431     elif self[0] == 2:
432         new_list.append(self[1])
433         if self[2] + 1 <= inflation_lvl:
434             new_list.append(self[2] + 1)
435         else:
436             new_list.append(1)
437     else:
438         return self
439     new_list.append(self[3])

```

```

439     return tuple(new_list)
440
441
442 def symmetry_dic(self, ref_pos, pi_dic, dic):
443     """Method, when getting passed a monomial creates a dictionary
444     with equivalent monomials resulting
445     from inflation symmetry. It returns the entries in Gamma to be
446     equated under this symmetry. In other words,
447     we create the orbits under the transformation group pi_dic."""
448
449     equivalent_dic = {}
450
451     """Construct the orbit to the transformation group."""
452     if isinstance(self, ComSym):
453         for perm in pi_dic.values():
454             if dic.get(perm.get(self, self)) != ref_pos:
455                 equivalent_dic[dic.get(perm.get(self, self))] = ref_pos
456         return equivalent_dic
457     else:
458         self = list(self.args)
459         for perm in pi_dic.values():
460             work_lst = []
461             for l in self:
462                 work_lst.append(perm.get(l, l))
463
464             """The elements in the orbit are not necessarily a
465             representative. Thus, reduce to representative / Sorting """
466             prod = sym.Mul(*work_lst)
467             arglist_sort = list(prod.args)
468             while True:
469                 # Bubble sort
470                 i = 0
471                 while i < len(arglist_sort) - 1:
472                     slice_prod = ComSym.__mul__(arglist_sort[i],
473                     arglist_sort[i + 1])
474                     arglist_sort[i:i + 2] = slice_prod.args
475                     i += 1
476
477                 new_prod = sym.Mul(*arglist_sort)
478                 if prod == new_prod:
479                     break
480                 prod = new_prod
481
482             """Find moment matrix position / Involution"""
483             prod_pos = dic.get(prod, dic.get(inv(prod)))

```

```

479
480     """No trivial key : value pairs, no redundancies"""
481     if prod_pos != ref_pos and prod_pos not in equivalent_dic.keys
482     ():
483         equivalent_dic[prod_pos] = ref_pos
484
485     return equivalent_dic
486
487 def factorize(dic):
488     """Factorisation procedure via monomial tensor representation."""
489
490     """Constructing the tensor representation."""
491     fac_dic = {}
492     for key, value in dic.items():
493         if isinstance(key, sym.Mul):
494             h_mat = np.zeros((num_party, inflation_lvl, inflation_lvl)
495 , dtype=object)
496
497             for a in range(0, num_party):
498                 for b in range(0, inflation_lvl):
499                     for c in range(0, inflation_lvl):
500                         h_mat[a][b][c] = []
501
502             arglist = list(key.args)
503             n = len(arglist)
504             list_of_lists = []
505             for i in range(0, n):
506                 """Account for multiple instances of an operator in a
507 monomial, no overwriting"""
508                 l = arglist[i]
509                 h_mat[int(l.name[0]) - 1][int(l.name[2]) - 1][int(l.
510 name[3]) - 1].append(i + 1)
511
512             """Summing over rows and columns of tensor slices
513 according to graph structure of quantum triangle
514 scenario"""
515             for a in range(0, inflation_lvl):
516                 """Append A's columns with B's rows """
517                 list_of_lists.append([i for i in [*flatten(list(h_mat
518 [0, :, a])), *flatten(list(h_mat[1, a, :]))] if i != 0])
519
520                 """Append A's rows with C's columns """
521                 list_of_lists.append([i for i in [*flatten(list(h_mat
522 [0, a, :])), *flatten(list(h_mat[2, :, a]))] if i != 0])

```

```

517
518         """Append B's columns with C's rows """
519         list_of_lists.append([i for i in [*flatten(list(h_mat
520 [1, :, a])), *flatten(list(h_mat[2, a, :]))] if i != 0])
521
522         list_of_lists = [i for i in list_of_lists if i != []]
523
524         list_of_lists.sort(key=len, reverse=True)
525
526         """Fusing: go through list of list, calculate intersection
527 of lists, if non-empty construct the union of the
528 two lists, erase duplicates and order. Do so for all
529 lists or until all intersections are empty"""
530
531         i = 0
532         while True:
533             m_set = set(list_of_lists[i])
534             for j in range(i + 1, len(list_of_lists)):
535                 l_set = set(list_of_lists[j])
536                 if any(x in m_set for x in l_set):
537                     """Replace list_of_lists[i] by the union,
538 erase list_of_lists[j]"""
539                     list_of_lists[i] = union(list_of_lists[i],
540 list_of_lists[j])
541                     del (list_of_lists[j])
542                     i -= 1
543                     break
544             if i == len(list_of_lists) - 1:
545                 break
546             i += 1
547
548         """Translating List_of_lists into tuple of separated
549 monomials"""
550         if len(list_of_lists) == 1:
551             fac_dic[(key, 1, 1)] = value
552         elif len(list_of_lists) == 2:
553             fac_dic[(sym.Mul(*[arglist[x - 1] for x in
554 list_of_lists[0]]),
555 sym.Mul(*[arglist[x - 1] for x in
556 list_of_lists[1]]), 1)] = value
557         else:
558             fac_dic[(sym.Mul(*[arglist[x - 1] for x in
559 list_of_lists[0]]),

```



```

552         sym.Mul(*[arglist[x - 1] for x in
list_of_lists[1]]),
553         sym.Mul(*[arglist[x - 1] for x in
list_of_lists[2]]))) = value
554
555     else:
556         """Adding ComSym instances and 1 as in tuple form"""
557         fac_dic[(key, 1, 1)] = value
558
559     return fac_dic
560
561
562 def positive_mon_fac(fac_dic):
563     """Non-negative moments, corresponding to positive operators.
Testing for  $x^*=x$ , i.e. fully commutative."""
564     positive_mon = {}
565
566     for key, value in fac_dic.items():
567         for comp in key:
568             if comp == 1:
569                 positive_mon[key] = value
570                 break
571             if isinstance(comp, sym.Mul):
572                 excep = False
573                 if inv(comp) != comp:
574                     break
575                 for i in range(0, len(comp.args) - 1):
576                     if ComSym.__mul__(comp.args[i], comp.args[i + 1])
!= ComSym.__mul__(comp.args[i + 1], comp.args[i]):
577                         excep = True
578                         break
579                 if excep:
580                     break
581             elif comp == key[2]:
582                 positive_mon[key] = value
583                 break
584
585     return positive_mon
586
587
588 def ai_expressible_set(positive_mon):
589     """The ai-expressible set, no two or more copy equivalent states
in one component"""
590
591     knowable_mon = {}

```

```

592
593     for key, value in positive_mon.items():
594         for comp in key:
595             if comp == 1:
596                 knowable_mon[key] = value
597                 break
598             if isinstance(comp, sym.Mul):
599                 source = []
600                 source.extend([i.name[a] for i in comp.args for a in
range(2, 4)])
601                 if len(erase_duplicates(source)) != 1:
602                     break
603                 knowable_mon[key] = value
604
605     return knowable_mon
606
607
608 def streamprinter(text):
609     """Mosek stream printer."""
610
611     sys.stdout.write(text)
612     sys.stdout.flush()
613
614
615 def mosek_env():
616     """Creating a Mosek solver environment."""
617
618     env = mosek.Env()
619     env.set_Stream(mosek.streamtype.log, streamprinter)
620     return env
621
622
623 def mosek_task_fixed(env, A_i, physical_set):
624     """We create a fixed Mosek task. The sparse A_i matrices
625     corresponding to the positions of the unknown moments y_i
626     in Gamma are passed to a Mosek task in the environment we
627     constructed. We set their bounds according to b of
628     the PSDP. In other words, in the DSDP we define the RHS of the
629     trace equalities  $\text{Tr}(ZA_i)=b_i$  with dual variable Z.
630     We allow for a subset of the y_i to correspond to positive
631     operators. There we set  $\text{Tr}(ZA_i)\leq 0$ . We formulate a
632     minimisation problem corresponding to the DSDP."""
633
634     numcon, matsize = A_i.shape[:2]

```

```

632     """Task creation."""
633     task = env.Task(0, 0)
634
635     """Passing A_i to task"""
636     task.appendcons(numcon)
637     task.appendbarvars([matsize])
638     task.appendvars(numcon)
639     task.putbarablocktriplet(
640         A_i.nnz,
641         A_i.coords[0],
642         A_i.nnz * [0],
643         A_i.coords[1],
644         A_i.coords[2],
645         A_i.data
646     )
647
648     """Setting up constraint bounds."""
649     task.putconboundlistconst(range(numcon), mosek.boundkey.fx, 0, 0)
650     task.putconbound(0, mosek.boundkey.fx, 1, 1)
651
652     """Nonnegative moments"""
653     task.putconboundlistconst(physical_set, mosek.boundkey.up, 0, 0)
654
655     """Minimisation"""
656     task.putobjsense(mosek.objsense.minimize)
657
658     task.putintparam(mosek.iparam.log_cut_second_opt, 0)
659     task.set_Stream(mosek.streamtype.msg, streamprinter)
660     return task
661
662
663 def var_task(task, C_mat):
664     """The variable part of the Mosek task. We pass the cost matrix C
665     to the mosek task created in mosek_task_fixed. We
666     initiate the optimize. From the solution, we reconstruct the full
667     dual variable Z needed for the certificate
668     optimisation."""
669
670     """Passing C to task."""
671     task.putbarcblocktriplet(
672         C_mat.nnz,
673         C_mat.nnz * [0],
674         *C_mat.coords,
675         C_mat.data
676     )

```

```

675
676     """Writing MOSEK data to file."""
677     # task.writedata('data_ptf.ptf')
678     # task.writedata('data_cbf.cbf')
679
680     """Running the Mosek solver."""
681     task.optimize()
682
683     """Printing solver statistics."""
684     task.solutionsummary(mosek.streamtype.msg)
685
686     """Solution extraction and Z construction."""
687     matsize = len(C_mat)
688     lenbarvar = np.floor_divide(matsize * (matsize + 1), 2)
689     barx = lenbarvar * [0.]
690     task.getbarxj(mosek.soltype.itr, 0, barx)
691     y = task.getnumcon() * [0.]
692     task.gety(mosek.soltype.itr, y)
693     primalobj = task.getprimalobj(mosek.soltype.itr)
694     dualobj = task.getprimalobj(mosek.soltype.itr)
695     duality_gap = np.abs(primalobj - dualobj)
696     barx_array = np.ones(shape=(matsize, matsize), dtype=float)
697     lower_indices = np.vstack(np.tril_indices(matsize))
698     lower_indices = lower_indices[:, np.argsort(lower_indices[-1],
699 kind='stable')]
700
701     barx_array[tuple(lower_indices)] = barx
702     barx_array[tuple(np.flipud(lower_indices))] = barx
703
704     return {'primalopt': primalobj, 'gap': duality_gap, 'barx': barx,
705 'y': y, 'barx_array': barx_array}
706
707 def kill_task(task):
708     """Killing Mosek task after use."""
709
710     task.__del__()
711
712
713 def kill_env(env):
714     """Killing Mosek environment after use."""
715
716     env.__del__()
717

```

```

718
719 """MAIN PROGRAMME"""
720
721 if __name__ == '__main__':
722     print('
723
724     -----
725
726     ')
727     print('QCCP for the quantum triangle scenario')
728     print('
729     -----
730
731     ')
732
733     """Problem specific variables"""
734     num_party = 3
735     inflation_lvl = 2
736     num_outcomes = 1
737
738     print('Specifications:')
739     print('Number of parties:', num_party)
740     print('Inflation level:', inflation_lvl)
741     print('Number of reduced outputs:', num_outcomes)
742     print('
743     -----
744
745     ')
746
747     """Hierarchy partitioning. If npa_level is False then we use local
748     levels"""
749     npa_level = False
750
751     """SDP Hierarchy step. Choose according to partitioning."""
752     sdp_step = 2
753
754     """Intermediate steps, if no cutoff is desired, then set cutoff =
755     inflation_lvl*num_party"""
756     cutoff = 6
757     positive_set = True
758
759     """Visibility"""
760     v = sym.Symbol('v', real=True)
761
762     """Probability distributions to test"""
763     W_dist = True
764     GHZ_dist = True
765     Uniform_dist = True
766

```

```

755     if W_dist:
756         prob_dic = {1: v * 2. / 3 + (1 - v) / 2.,
757                     2: v * 1. / 3 + (1 - v) / 4.,
758                     3: (1 - v) / 8.}
759     elif GHZ_dist:
760         prob_dic = {1: v * 1. / 2 + (1 - v) / 2.,
761                     2: v * 1. / 2 + (1 - v) / 4.,
762                     3: v * 1. / 2 + (1 - v) / 8.}
763     else:
764         prob_dic = {1: v * 1. / 2 + (1 - v) / 2.,
765                     2: v * 1. / 4 + (1 - v) / 4.,
766                     3: v * 1. / 8 + (1 - v) / 8.}
767
768     """Both scalar extension and scalar multiples are non certificate
769     type constraints!"""
770     """Do scalar extension"""
771     scalar_extension = False
772     scalar_multiples = False
773
774     if scalar_extension or scalar_multiples:
775         """Non-certificate type constraints"""
776         non_certificate = True
777     else:
778         non_certificate = False
779
780     """Solving method: Bisection vs. Certificate optimisation. The
781     latter is default."""
782     if non_certificate:
783         bisection = True
784     else:
785         bisection = False
786
787     """Printing?"""
788     printing_var = False
789
790     dimension = num_party * inflation_lvl
791
792     """Creating a list of generators and a dictionary collecting them
793     according to their party correspondence."""
794     generators, generator_party_dic = var_list_creation(num_party,
795                                                         inflation_lvl, num_outcomes)
796
797     print('
-----
')

```

```

794
795     """Dictionary with tuple representation of generators, i.e.
(1,1,2,1)=1_12_01=E^{1,2}_1."""
796     tuples_comsym_dic = {}
797     for a in generators:
798         tuples_comsym_dic[comsym_tpl(a)] = a
799
800     """Generating monomials"""
801     generating_mon = construction_vector(generator_party_dic, cutoff,
inflation_lvl, num_party, sdp_step, npa_level,
802                                         positive_set)
803
804     if printing_var:
805         print('Generating monomials:')
806         print(generating_mon)
807
808     """Constructing the moment matrix dictionaries. The
representative_dic has key: monomials and values: position. The
809     structure_dic has key: position of equivalence class element,
value: position representative. Separating Gamma into
810     two dictionaries allows us to mainly work with the smaller
dictionary of representatives. """
811     structure_dic, representative_dic = moment_matrix_construction(
generating_mon)
812
813     """Help dictionary we need for scalar extension keeping track of
the representatives before any symmetry
814     reductions"""
815     first_dic = {}
816     first_dic.update(representative_dic)
817
818     print('D.O.F. after involution equivalence:', len(
representative_dic))
819
820     if printing_var:
821         print('Moment Matrix representatives:')
822         print(representative_dic)
823
824     print('Size of Moment Matrix', len(generating_mon), 'x', len(
generating_mon))
825
826     base_set = base_set(representative_dic)
827
828     """Dictionary reversing key, value in representative_dic in order
to determine position of moment representative"""

```

```

829 mon_to_position = {v: k for k, v in representative_dic.items()}
830
831 """Dictionaries for reductions"""
832 reduced_structure_dic = {}
833 reduced_structure_dic.update(structure_dic)
834
835 if inflation_lvl >= 2:
836     """Transformation group creation"""
837     transformation_group = permutation_dic(generators,
inflation_lvl, tuples_comsym_dic)
838     Trans_orbits_dic = {} # key: element of orbit under
transformation group , value:representative
839
840     """We start by creating the orbits to the base set, so that we
have the elements of the injectable set as
841     representatives. Only construct orbit, if monomial is not
already element of a previous orbit. """
842     for key in base_set:
843         if key not in Trans_orbits_dic.keys():
844             orbit_dic = symmetry_dic(mon_to_position.get(key), key
, transformation_group, representative_dic)
845             # key: position of orbit element, value: position of
representative
846
847             Trans_orbits_dic.update(orbit_dic)
848
849     """Help dictionary for Scalar multiples"""
850     injec_dic = {}
851     injec_dic.update(Trans_orbits_dic)
852
853     """Add the remaining orbits to all moment representatives.
Again, only construct orbit, if monomial not already
854     in any previous orbit."""
855     for key, value in representative_dic.items():
856         if isinstance(key, sym.Mul):
857             if value not in Trans_orbits_dic.keys():
858                 orbit_dic = symmetry_dic(key, value,
transformation_group, representative_dic)
859
860                 Trans_orbits_dic.update(orbit_dic)
861
862     """Reducing moment representative_dic by removing all
representatives not representatives in the
863     Trans_orbit_dic."""
864     for key in Trans_orbits_dic.keys():

```



```

865         representative_dic.pop(mon_to_position.get(key))
866
867     print('D.O.F. after symmetry reduction: ', len(
representative_dic))
868
869     """Updating structure_dic using Trans_orbits_dic. Replace all
values in structure_dic, which are keys in
870     Trans_orbits_dic and add the replaced value as a key with
value given by the orbit representative. In other
871     words, if (a,b):(c,d) in Trans_orbits_dic and (e,f):(a,b) in
structure_dic then add to reduced_structure_dic
872     (e,f):(c,d) and (a,b):(c,d)."""
873     reduced_structure_dic = {}
874     for key, value in structure_dic.items():
875         if value in Trans_orbits_dic.keys():
876             reduced_structure_dic[key] = Trans_orbits_dic.get(
value)
877             reduced_structure_dic[value] = Trans_orbits_dic.get(
value)
878         else:
879             reduced_structure_dic[key] = value
880
881     reduced_structure_dic.update(Trans_orbits_dic)
882
883     """Add all representative_dic.value : representative_dic.value
to reduced_structure_dic"""
884     for value in representative_dic.values():
885         reduced_structure_dic[value] = value
886
887     """Factorisation of representatives"""
888     factorisation_dic = factorize(representative_dic)
889     red_factorisation_dic = factorisation_dic
890
891     """Non-negative moments, i.e. which moments correspond to
monomials which factorise into components, which then
892     in turn are fully commutative, i.e. involution invariant."""
893     positive_mon = positive_mon_fac(factorisation_dic)
894     for key in positive_mon:
895         red_factorisation_dic.pop(key)
896
897     print('Number of positive monomials with non-negative moments:
', len(positive_mon))
898
899     """Determining ai-expressible set"""
900     ai_express = ai_expressible_set(positive_mon)

```

```

901     print("Cardinality ai-expressible set:", len(ai_express))
902
903     """Positive, but not in ai_expressible"""
904     positive_unknown = {}
905     for key, value in positive_mon.items():
906         if key not in ai_express.keys():
907             positive_unknown[key] = value
908
909     print("Number of unknowable positive monomials:", len(
910 positive_unknown))
911
912     """Assigning values to ai_expressible set depending on the
913 probability distribution at test. This reduces to
914 determining the length of the monomial in a component and
915 assigning the correct marginal probability."""
916     ai_expressible_probability = {}
917     for key, value in ai_express.items():
918         prod = 1
919         for comp in key:
920             if isinstance(comp, sym.Mul):
921                 prod *= prob_dic.get(len(comp.args), 1)
922             elif isinstance(comp, ComSym):
923                 prod *= prob_dic.get(1)
924             else:
925                 prod *= 1
926         ai_expressible_probability[value] = prod
927
928     if printing_var:
929         print('The ai-expressible set with corresponding
930 probabilities:')
931         for key, value in ai_expressible_probability.items():
932             print(key, value)
933
934     """Erase ai-expressible set from representative_dic"""
935     for key in ai_expressible_probability.keys():
936         representative_dic.pop(mon_to_position.get(key))
937
938     """Non-certificate type constraints: Scalar Multiples and
939 Scalar Extension"""
940     """This is limited to inflation level 2!"""
941     scalar_multiple_moments = {}
942     scal_var_dic = {}
943     if inflation_lvl == 2:
944         if scalar_multiples:

```

```

940         """Scalar multiples: identify those moments
factorising into a component corresponding to an
941         element of the ai-expressible set and an unknown
moment. Reduce the non-convex factorisation to an
942         affine expression by assigning the ai-expressible
component a probability. We are limiting to the
943         simplest case, where just one component is known"""
944
945         print(
946             ,
947             ,
948             , '-----')
949         print('Scalar Multiples')
950         print(
951             ,
952             , '-----')
953
954         count = 0
955
956         for key, value in red_factorisation_dic.items():
957             other_comps = []
958             scalar_multiple = False
959             alpha = 1
960             for comp in key:
961                 if comp == 1:
962                     break
963                 if isinstance(comp, sym.Mul):
964                     source = []
965                     source.extend([i.name[a] for i in comp.
args for a in range(2, 4)])
966                     if len(erase_duplicates(source)) == 1:
967                         """Knownable sym.Mul"""
968                         scalar_multiple = True
969                         alpha = prob_dic.get(len(comp.args))
970                         break
971                     other_comps.append(comp)
972             else:
973                 """Knownable ComSym"""
974                 scalar_multiple = True
975                 alpha = prob_dic.get(1)
976                 break
977
978         if scalar_multiple:

```

```

977         count += 1
978         y_unknown = sym.Mul(*other_comps)
979         if representative_dic.get(y_unknown) is not
None:
980             scalar_multiple_moments[value] = (alpha,
representative_dic.get(y_unknown))
981             representative_dic.pop(y_unknown)
982         else:
983             print(y_unknown)
984             """Component is not representative, Reduce
to representative / Sorting """
985             prod = y_unknown
986             arglist_sort = list(prod.args)
987             while True:
988                 # Bubble sort
989                 i = 0
990                 while i < len(arglist_sort) - 1:
991                     slice_prod = ComSym.__mul__(
arglist_sort[i], arglist_sort[i + 1])
992                     arglist_sort[i:i + 2] = slice_prod
.args
993                     i += 1
994
995             new_prod = sym.Mul(*arglist_sort)
996             if prod == new_prod:
997                 break
998             prod = new_prod
999             """Find moment matrix position /
Involution"""
1000             prod_pos = representative_dic.get(prod,
representative_dic.get(inv(prod)))
1001             if prod_pos is not None:
1002                 scalar_multiple_moments[value] = (
alpha, prod_pos)
1003                 if prod in representative_dic.keys():
1004                     representative_dic.pop(prod)
1005                 else:
1006                     representative_dic.pop(inv(prod))
1007             else:
1008                 """Symmetry"""
1009                 for which_pi, perm in
transformation_group.items():
1010                     work_lst = []
1011                     self = list(y_unknown.args)
1012                     for l in self:

```

```

1013         work_lst.append(perm.get(1, 1)
1014     )
1015
1016     """Reduce to representative /
1017     Sorting """
1018     prod = sym.Mul(*work_lst)
1019     arglist_sort = list(prod.args)
1020     while True:
1021         # Bubble sort
1022         i = 0
1023         while i < len(arglist_sort) -
1024             1:
1025             slice_prod = ComSym.
1026             __mul__(arglist_sort[i], arglist_sort[i + 1])
1027             arglist_sort[i:i + 2] =
1028             slice_prod.args
1029             i += 1
1030
1031             new_prod = sym.Mul(*
1032             arglist_sort)
1033             if prod == new_prod:
1034                 break
1035             prod = new_prod
1036
1037     """Find moment matrix position /
1038     Involution"""
1039     prod_pos = representative_dic.get(
1040     prod, representative_dic.get(inv(prod)))
1041     if prod_pos is not None:
1042         scalar_multiple_moments[value]
1043         = (alpha, prod_pos)
1044         if prod in representative_dic.
1045         keys():
1046             representative_dic.pop(
1047             prod)
1048         else:
1049             representative_dic.pop(inv
1050             (prod))
1051         break
1052
1053     print('Number of scalar multiple moments', len(
1054     scalar_multiple_moments))
1055
1056     """Scalar Extension procedure"""
1057     if scalar_extension:

```

```

1045         print(
1046             ,
1047             , '-----')
1048         print('Scalar Extension procedure')
1049         print(
1050             ,
1051             , '-----')
1052
1053         """Find first row in moment matrix"""
1054         first_col = {}
1055         first_col_structure = {}
1056         for a in range(0, len(generating_mon)):
1057             if mon_to_position.get(reduced_structure_dic.get((
a, 0))) not in first_col.keys():
1058                 first_col[mon_to_position.get(
reduced_structure_dic.get((a, 0)))] = (a, 0)
1059             else:
1060                 first_col_structure[(a, 0)] = first_col.get(
mon_to_position.get(reduced_structure_dic.get((a, 0))))
1061
1062         """Filter out remaining factorisable moments, not
accounted for in SDP"""
1063         help_dic = factorize(representative_dic)
1064         scalar_dic = {}
1065         help_list = []
1066         red_factorisation_dic = {}
1067         for key, value in help_dic.items():
1068             if key[1] != 1:
1069                 scalar_dic[key] = value
1070                 help_list = []
1071                 for comp in key:
1072                     if comp == 1:
1073                         help_list.append(1)
1074                     elif comp in representative_dic.keys():
1075                         help_list.append(comp)
1076                 else:
1077                     """Reduce to normal form, use
first_dic without moment reductions."""
1078                     help_list.append(mon_to_position.get(
reduced_structure_dic.get(

```

```

1079         first_dic.get(comp, first_dic.get(
inv(comp))))))
1080         red_factorisation_dic[tuple(help_list)] =
value
1081
1082         """Filter out scalar multiples moments"""
1083         scalar_ext_dic = {}
1084         for key, value in scalar_dic.items():
1085             for comp in key:
1086                 if comp != 1:
1087                     val = first_dic.get(comp)
1088                     if val in injec_dic.keys() or val in
injec_dic.values():
1089                         break
1090                 else:
1091                     scalar_ext_dic[key] = value
1092
1093         if printing_var:
1094             print('Full set scalar extension entries')
1095             for key, value in scalar_ext_dic.items():
1096                 print(key, value)
1097
1098         """Consider the different scalar extension cases!"""
1099         """Filter out moments where components not in
generating moments"""
1100         reduced_scal_ext_dic = {}
1101         for key, value in scalar_ext_dic.items():
1102             for comp in key:
1103                 if comp != 1:
1104                     if comp not in first_col.keys() and
mon_to_position.get(reduced_structure_dic.get(
1105                         first_dic.get(comp, first_dic.get(
inv(comp)))))) not in first_col.keys():
1106                         break
1107                 else:
1108                     reduced_scal_ext_dic[key] = value
1109
1110         """Moments without a counterpart in the generating
monomials."""
1111         additional_scalar_ext_dic = {}
1112         for key, value in scalar_ext_dic.items():
1113             if key not in reduced_scal_ext_dic.keys():
1114                 additional_scalar_ext_dic[key] = value
1115
1116         if printing_var:

```

```

1117         print('Additional set scalar extension entries')
1118         for key, value in additional_scalar_ext_dic.items
():
1119             print(key, value)
1120
1121         if printing_var:
1122             print('Reduced set scalar extension entries')
1123             for key, value in reduced_scal_ext_dic.items():
1124                 print(key, value)
1125
1126         """Create minimal set of scalar extension. Only one
representative component from reduced set needed!"""
1127         min_set = []
1128         for key in reduced_scal_ext_dic.keys():
1129             for comp in key:
1130                 if comp != 1:
1131                     if comp not in min_set:
1132                         min_set.append(comp)
1133                         break
1134
1135         for key in additional_scalar_ext_dic.keys():
1136             for comp in key:
1137                 if comp != 1:
1138                     if comp not in min_set:
1139                         min_set.append(comp)
1140
1141         if printing_var:
1142             print('Minimal set scalar extension: ')
1143             for el in min_set:
1144                 print(el)
1145
1146         """Reducing elements in minimal set to their
representatives appearing in the gamma matrix"""
1147         rep_min_set = []
1148         for el in min_set:
1149             if el in representative_dic.keys():
1150                 rep_min_set.append(el)
1151             elif inv(el) in representative_dic.keys():
1152                 rep_min_set.append(inv(el))
1153             elif mon_to_position.get(reduced_structure_dic.get(
(
1154                 first_dic.get(el, first_dic.get(
inv(el)))))) in representative_dic.keys():
1155                 rep_min_set.append(mon_to_position.get(
reduced_structure_dic.get(

```



```

1156         first_dic.get(el, first_dic.get(
inv(el))))))
1157
1158         rep_min_set = erase_duplicates(rep_min_set)
1159
1160         if printing_var:
1161             print('Representatives minimal set scalar
extension: ')
1162             for el in rep_min_set:
1163                 print(el)
1164
1165             """Assign sympy.Symbol to each element in rep_in_set"""
1166
1167             scal_var_dic = {}
1168             i = 1
1169             for el in rep_min_set:
1170                 scal_var_dic[sym.Symbol('a' + '_' + str(i))] = el
1171                 i += 1
1172
1173             if printing_var:
1174                 print('Variable assignment scalar extension')
1175                 for key, value in scal_var_dic.items():
1176                     print(key, value)
1177
1178             squared_set = []
1179             """Construct the additional rows and make all possible
moment to moment assignments"""
1180             i = 0
1181             for key, value in scal_var_dic.items():
1182                 print('Scalar Variable', key, value)
1183                 help_dic = {}
1184                 double_dic = {}
1185                 for key_x, value_x in first_col.items():
1186                     if key_x == 1:
1187                         """<a> case"""
1188                         help_dic[(len(generating_mon) + i, value_x
[0])] = representative_dic.get(value)
1189                         continue
1190                     if key_x in scal_var_dic.values():
1191                         if key_x == value:
1192                             """a<a> case"""
1193                             help_dic[(len(generating_mon) + i,
value_x[0])] = (len(generating_mon) + i, value_x[0])
1194                             representative_dic[key*key_x] = (len(
generating_mon) + i, value_x[0])

```

```

1194         squared_set.append(key*key_x)
1195         continue
1196         for red_fac_mom, red_fac_mon_pos in
red_factorisation_dic.items():
1197             """b<a> case"""
1198             if (value, key_x, 1) == red_fac_mom or
(key_x, value, 1) == red_fac_mom:
1199                 """b<a> case with corresponding
entry in moment matirx"""
1200                 help_dic[(len(generating_mon) + i,
value_x[0])] = red_fac_mon_pos
1201                 break
1202             else:
1203                 """b<a> case no moment matrix
correspondance"""
1204                 help_dic[(len(generating_mon) + i,
value_x[0])] = (len(generating_mon) + i, value_x[0])
1205                 representative_dic[key * key_x] = (len
(generating_mon) + i, value_x[0])
1206                 continue
1207                 continue
1208                 """purely scalar extension representatives
entry, <a><b>"""
1209                 if key_x in scal_var_dic.keys():
1210                     for red_fac_mom, red_fac_mon_pos in
red_factorisation_dic.items():
1211                         if (value, scal_var_dic.get(key_x), 1)
== red_fac_mom or (scal_var_dic.get(key_x),
1212                         value, 1) == red_fac_mom:
1213                             """<a><b> case, which can be
mapped to something in moment matrix"""
1214                             help_dic[(len(generating_mon) + i,
value_x[0])] = red_fac_mon_pos
1215                             break
1216                         else:
1217                             """<a><b> case with no equivalent
entries"""
1218                             help_dic[(len(generating_mon) + i,
value_x[0])] = (len(generating_mon) + i, value_x[0])
1219                             representative_dic[key * key_x] = (len
(generating_mon) + i, value_x[0])
1220                             continue
1221                             continue
1222                     else:

```

```

1223         for red_fac_mom, red_fac_mon_pos in
red_factorisation_dic.items():
1224             if (value, key_x, 1) == red_fac_mom or
(key_x, value, 1) == red_fac_mon:
1225                 """x<a> with equivalent entry in
moment matrix"""
1226                 help_dic[(len(generating_mon) + i,
value_x[0])] = red_fac_mon_pos
1227                 break
1228             else:
1229                 """x<a> without equivalent entry in
moment matrix"""
1230                 help_dic[(len(generating_mon) + i,
value_x[0])] = (len(generating_mon) + i, value_x[0])
1231                 representative_dic[key * key_x] = (len(
generating_mon) + i, value_x[0])
1232                 continue
1233                 continue
1234
1235                 """Adding <a>**2"""
1236                 if value in first_col.keys():
1237                     """a<a> = <a>**2"""
1238                     help_dic[(len(generating_mon) + i, len(
generating_mon) + i)] = (len(generating_mon) + i,
1239
first_col.get(value)[0])
1240
1241                 else:
1242                     """For a not in generating moments / no
equating a(a) = (a**2)"""
1243                     help_dic[(len(generating_mon) + i, len(
generating_mon) + i)] = (len(generating_mon) + i,
1244
len(generating_mon) + i)
1245                     representative_dic[key * key] = (len(
generating_mon) + i, len(generating_mon) + i)
1246                     squared_set.append(key * key)
1247
1248                     reduced_structure_dic.update(help_dic)
1249
1250                     for el_x, el_y in first_col_structure.items():
1251                         reduced_structure_dic [(len(generating_mon) +
i, el_x[0])] = help_dic.get((len(generating_mon)
1252
+ i, el_y[0]))

```

```

1253         first_col[key] = (len(generating_mon) + i, 0)
1254         i += 1
1255
1256
1257         print('Number of additional rows: ', len(scal_var_dic)
1258 )
1259
1260         print('Number of variables in SDP after scalar
1261 extension: ', len(representative_dic))
1262
1263         # print('Final number of D.O.F. / Unknown moments: ', len(
1264 representative_dic) - len(scalar_multiple_moments))
1265
1266         print('
1267 -----
1268 ')
1269         print('MOSEK Semidefinite Programming')
1270         print('
1271 -----
1272 ')
1273         """MOSEK Semidefinite Programming"""
1274         n = len(generating_mon) + len(scal_var_dic) # Matrix sizes in
1275 mosek
1276
1277         """Construct constant matrix with dependence on visibility C(v
1278 ) as a numpy.dense matrix. Find positions of
1279 ai-expressible set in Gamma and assign value according to
1280 ai_expressible_probability."""
1281         C_v = sym.zeros(n, n)
1282         count = 0
1283         count_diag = 0
1284         for key, value in ai_expressible_probability.items():
1285             for key_x, value_x in reduced_structure_dic.items():
1286                 if value_x == key:
1287                     if key_x[0] == key_x[1]:
1288                         count_diag += 1
1289                     else:
1290                         count += 1
1291                         C_v[key_x[0], key_x[1]] = value
1292                         C_v[key_x[1], key_x[0]] = value
1293         print('Specified elements in C(v)', 2*count + count_diag)
1294         lambdify_Cv = sym.lambdify(v, C_v, modules='numpy', cse=True)
1295         # Lambdify C(v) so we can feed it a specific v
1296         del C_v

```



```

1324         slice_2.append(key_x[1])
1325         slice_3.append(alpha_i.subs(v, v_mid))
1326         # need to replace v with numerical value
1327         elif val_x == y_u:
1328             slice_0.append(i)
1329             slice_1.append(key_x[0])
1330             slice_2.append(key_x[1])
1331             slice_3.append(1)
1332         """Collect non-negative moments, so that we
1333         set according bounds in mosek fixed task"""
1334         if y_u in positive_unknown.values():
1335             non_negative_set.append(i)
1336
1337     else:
1338         """Non scalar multiples representatives"""
1339         for key_x, val_x in reduced_structure_dic.
1340 items():
1341             if val_x == value:
1342                 slice_0.append(i)
1343                 slice_1.append(key_x[0])
1344                 slice_2.append(key_x[1])
1345                 slice_3.append(1)
1346             if value in positive_unknown.values(): # non-
1347 negative moment
1348                 non_negative_set.append(i)
1349
1350         """Constructing a sparse coo array, i.e. a higher
1351 dimensional sparse object. """
1352         A_coords = np.array([slice_0, slice_1, slice_2])
1353         A_i_sparse = sparse.COO(A_coords, np.array(slice_3))
1354
1355         """Starting the mosek environment and defining the
1356 fixed task."""
1357         my_env = mosek_env()
1358         my_task = mosek_task_fixed(my_env, A_i_sparse,
1359 non_negative_set)
1360
1361         """Evaluate C(v) for a given v and convert it to the
1362 sparse lower triangular format"""
1363         C_evaluated = lambdify_Cv(v_mid)
1364         C_mosek = sparse.tril(sparse.COO.from_numpy(
1365 C_evaluated))
1366
1367         """find the solution to the SDP for this given C
1368 matrix"""

```

```

1359         mosek_Solution = var_task(my_task, C_mosek)
1360
1361         if mosek_Solution['primalopt'] < 0:
1362             v_upper = v_mid
1363         else:
1364             v_lower = v_mid
1365
1366         print('Visibility threshold:', v_upper)
1367         """Non witnesses for non-certificate type constraints. """
1368         exit()
1369
1370     else:
1371         """Certificate type constraints."""
1372
1373         """Construct A_i matrices. We make use of the COOrdinate
format to define sparse matrices."""
1374         # Identity at A_0
1375         slice_0 = [0] * n
1376         slice_1 = [i for i in range(0, n)]
1377         slice_2 = [i for i in range(0, n)]
1378
1379         """A_i matrices"""
1380         i = 0
1381         for key, value in representative_dic.items():
1382             i += 1
1383             for key_x, val_x in reduced_structure_dic.items():
1384                 if val_x == value:
1385                     slice_0.append(i)
1386                     slice_1.append(key_x[0])
1387                     slice_2.append(key_x[1])
1388             if value in positive_unknown.values(): # non-negative
moment
1389                 non_negative_set.append(i)
1390
1391         """COO sparse matrix array"""
1392         A_coords = np.array([slice_0, slice_1, slice_2])
1393         numcon = A_coords.shape[-1]
1394
1395         """Putting scalar multiples at the end"""
1396         A_i_sparse = sparse.COO(A_coords, np.ones(numcon, dtype=
bool))
1397
1398         """Initiating mosek environment and constructing fixed
task."""
1399         my_env = mosek_env()

```

```

1400     my_task = mosek_task_fixed(my_env, A_i_sparse,
non_negative_set)
1401
1402     """ We give the option to choose between bisection or
certificate optimisation for only certificate type
1403     constraints in a problem. Note that the bisection is much
slower than certificate optimisation. """
1404     if bisection:
1405         print('BISECTION ALGORITHM')
1406         print(
1407             '
-----
',
1408             '-----')
1409
1410     """Bisection algorithm"""
1411
1412     v_lower = 0.0
1413     v_upper = 1.
1414     abs_tol = 0.01
1415     while v_upper - v_lower > abs_tol:
1416
1417         v_mid = (v_upper + v_lower) / 2.
1418
1419         print('Testing visibility: ', v_mid)
1420
1421         """Evaluate C(v) for a given v and convert it to the
sparse lower triangular format"""
1422         C_evaluated = lambdify_Cv(v_mid)
1423         C_mosek = sparse.tril(sparse.C00.from_numpy(
C_evaluated))
1424
1425         """find the solution to the SDP for this given C
matrix"""
1426         mosek_Solution = var_task(my_task, C_mosek)
1427
1428         if mosek_Solution['primalopt'] < 0:
1429             v_upper = v_mid
1430         else:
1431             v_lower = v_mid
1432
1433         print(v_upper)
1434         exit()
1435
1436     else:

```



```

1437         """Certificate optimization. Exploits SDP duality theory
to find optimal v."""
1438         print('Certificate Optimization')
1439         print(
1440             '
-----
',
1441             '-----')
1442
1443         """We exploit the minimisation from scipy."""
1444         from scipy.optimize import minimize
1445
1446         v_up = 1.1
1447         v_down = 1.
1448         tol = 0.001
1449         while v_up - v_down > tol:
1450
1451             """Evaluate C(v) for a given C and convert it to the
sparse lower triangular format"""
1452             C_evaluated = lambdify_Cv(v_down)
1453             C_mosek = sparse.tril(sparse.C00.from_numpy(
C_evaluated))
1454
1455             """find the solution to the SDP for this given C
matrix"""
1456             mosek_Solution = var_task(my_task, C_mosek)
1457
1458             print("Mosek SDP iteration complete, now computing
implication on visibility.")
1459
1460             """Turn max v into min -v"""
1461             objective = lambda v: -v
1462
1463             """We minimise -v s.t.  $\text{Tr}(C(v)Z^*) \leq 0$  and  $v < v_{\text{previous}}$ 
, where  $Z^*$  is dual optimal variable obtained
from mosek."""
1464             constraint_from_sdp = sym.lambdify(v, np.tensordot(
lambdify_Cv(v), mosek_Solution['barx_array']),
1465                                     modules='numpy', cse=
True)
1466
1467
1468
1469             def verbose_constraint_from_sdp(v):
1470                 print("Current visibility under consideration by
SciPy:", v)

```

```

1471         return constraint_from_sdp(v)
1472
1473
1474         x0 = np.array([0.0])
1475         bounds = np.array([[0.0, 1.0]])
1476         constraints = ({'type': 'ineq', 'fun':
constraint_from_sdp})
1477
1478         solution = minimize(objective, x0, method='SLSQP',
bounds=bounds, constraints=constraints,
1479                             options={'disp': True})
1480
1481         v_up = v_down
1482         v_down = solution['x'][0]
1483         print("Testing v:", v_down)
1484
1485         if v_up - v_down <= tol:
1486             print("Visibility threshold:", v_down)
1487
1488             """Generating Certificate from critical v
threshold"""
1489             print(
1490
-----
1491             ,
1492             '-----')
1493             print('Certificate')
1494             print(
1495
-----
1496             ,
1497             '-----')
1498
1499             """Calculating the coefficients  $\text{Tr}(C, Z^*)$  to each
element in the ai-expressible set."""
1500             a_i = {}
1501             c_const = 0
1502             for key in ai_expressible_probability.keys():
1503                 C_identifiable = sym.zeros(n, n)
1504                 if key == (0,0):
1505                     C_identifiable[0, 0] = 1 # Only entry is 1
at top left
1506
1507                     c_const = np.tensordot(C_identifiable,
mosek_Solution['barx_array']) #RHS term
1508                 else:

```

```

1506         for key_x, value_x in
reduced_structure_dic.items():
1507             if value_x == key:
1508                 C_identifiable[key_x[0], key_x[1]]
= 1
1509                 C_identifiable[key_x[1], key_x[0]]
= 1
1510             a_i[mon_to_position.get(key)] = np.
tensordot(C_identifiable, mosek_Solution['barx_array'])
1511             # LHS terms
1512             del C_identifiable
1513
1514             certificate_dic = {}
1515             witness_dic = {}
1516
1517             """Erasing zero entries, rescaling and rounding
down."""
1518             rescaling_factor = 100
1519             rounding_down = True
1520             cutoff_small = 10**(-5)
1521             scal_round_certificate_dic = {}
1522
1523             """Collect in dic all terms large enough and
rescale"""
1524             for key, value in a_i.items():
1525                 if abs(value) > cutoff_small:
1526                     scal_round_certificate_dic[key] = np.floor
(rescaling_factor*value)
1527
1528             for key, value in a_i.items():
1529                 certificate_dic[key] = value
1530                 witness_dic[key] = value
1531
1532             """No rescaling"""
1533             print('Full Certificate')
1534             for key, value in certificate_dic.items():
1535                 print(key, value)
1536
1537             print('Right hand side inequality', mosek_Solution
['primalopt']-c_const)
1538             print('Number of terms LHS:', len(certificate_dic)
)
1539
1540             print('Rescaled and rounded Certificate')

```

```

1541         for key, value in scal_round_certificate_dic.items
1542     ):
1543
1544         print(key, value)
1545
1546         """RHS"""
1547         help_var = mosek_Solution['primalopt']-c_const
1548         if abs(help_var) < cutoff_small:
1549             print('Right hand side inequality', 0)
1550         else:
1551             print('Right hand side inequality', np.floor(
1552                 help_var))
1553
1554             print('Number of terms LHS:', len(
1555                 scal_round_certificate_dic))
1556
1557             factor_witness_dic = factorize(witness_dic)
1558
1559             """Witness in computer science notation {0, 1}"""
1560             print('Witness in computer science notation {0, 1}
1561 ')
1562
1563             witness_sum = 0
1564             comp_list = []
1565             for key, value in factor_witness_dic.items():
1566                 prod = 1
1567                 for comp in key:
1568                     if comp != 1:
1569                         comp_list.append(comp)
1570                         prod *= value * comp
1571
1572                 witness_sum += prod
1573
1574             witness_sum = sym.expand(witness_sum)
1575             comp_list = erase_duplicates(comp_list)
1576
1577             print('Witness Inequality:')
1578             witness_terms = list(witness_sum.args)
1579             for el in witness_terms:
1580                 print(el)
1581
1582             print('RHS inequality', mosek_Solution['primalopt'
1583 ]-c_const)
1584
1585             print('Testing compatibility of distribution via
1586 witness')

```

```

1579         """Witness value for the distribution being tested
1580
1581         """
1582         comp_prob_sub_list = []
1583         for el in comp_list:
1584             if isinstance(el, sym.Mul):
1585                 comp_prob_sub_list.append((el, prob_dic.
1586 get(len(el.args)).subs(v, 1)))
1587             else:
1588                 comp_prob_sub_list.append((el, prob_dic.
1589 get(1).subs(v, 1)))
1590
1591         print('Inequality value probability distribution
1592 at test', witness_sum.subs(comp_prob_sub_list))
1593
1594         if witness_sum.subs(comp_prob_sub_list) >
1595 mosek_Solution['primalopt']-c_const:
1596             print('Witness of incompatibility!')
1597
1598         """Witness in physical notation {-1, +1}"""
1599         print('Witness in physical notation {-1, +1}')
1600         witness_sum = 0
1601         A = sym.Symbol('A')
1602         B = sym.Symbol('B')
1603         C = sym.Symbol('C')
1604         AC = sym.Symbol('AC')
1605         AB = sym.Symbol('AB')
1606         BC = sym.Symbol('BC')
1607         ABC = sym.Symbol('ABC')
1608
1609         party_dic = {'1': A, '2': B, '3': C, '12': AB, '13
1610 ': AC, '23': BC, '123': ABC}
1611         for key, value in factor_witness_dic.items():
1612             prod = 1
1613             for comp in key:
1614                 if isinstance(comp, ComSym):
1615                     prod *= value * 0.5*(1+party_dic.get(
1616 comp.name[0]))
1617
1618                 elif isinstance(comp, sym.Mul):
1619                     arglist = list(comp.args)
1620                     help_list = []
1621                     for el in arglist:
1622                         help_list.append(el.name[0])
1623
1624                     if len(arglist) == 2:

```

```

1617         help_list.append(help_list[0] +
help_list[1])
1618         fac = 0
1619         for i in help_list:
1620             fac += party_dic.get(i)
1621             prod *= value*0.25*(1+fac)
1622         else:
1623             help_list.append(help_list[0] +
help_list[1])
1624             help_list.append(help_list[1] +
help_list[2])
1625             help_list.append(help_list[0] +
help_list[2])
1626             help_list.append(help_list[0] +
help_list[1] + help_list[2])
1627             fac = 0
1628             for i in help_list:
1629                 fac += party_dic.get(i)
1630                 prod *= value * 0.125 * (1 + fac)
1631
1632             witness_sum += prod
1633
1634             witness_sum = sym.expand(witness_sum)
1635
1636             print('Full Witness Inequality:')
1637             witness_terms = list(witness_sum.args)
1638             for el in witness_terms:
1639                 print(el)
1640
1641             con_trans = list(witness_sum.args)[0]
1642             witness_sum = witness_sum - con_trans
1643             print('RHS inequality', mosek_Solution['primalopt'
]-c_const - con_trans)
1644
1645
1646             """Closing task and environment"""
1647             kill_task(my_task)
1648             kill_env(my_env)

```

References

- [1] Akshay Agrawal et al. “A rewriting system for convex optimization problems”. In: *Journal of Control and Decision* 5.1 (2018), pp. 42–60.
- [2] John-Mark A. Allen et al. “Quantum Common Causes and Quantum Causal Models”. In: *Physical Review X* 7.3 (2017). DOI: [10.1103/physrevx.7.031021](https://doi.org/10.1103/physrevx.7.031021). URL: <https://doi.org/10.1103/physrevx.7.031021>
- [3] MOSEK ApS. *MOSEK Fusion API for Python - Release 9.3.18*. 2022. URL: <https://docs.mosek.com/9.3/pythonfusion.pdf>
- [4] MOSEK ApS. *MOSEK Optimizer API for Python - Release 9.3.18*. 2022. URL: <https://docs.mosek.com/9.3/pythonapi.pdf>
- [5] R. B. Ash. *Basic probability theory* / Robert B. Ash. eng. New York: Wiley, 1970.
- [6] Jonathan Barrett, Lucien Hardy, and Adrian Kent. “No signaling and quantum key distribution”. In: *Physical review letters* 95.1 (2005), p. 010503.
- [7] Jonathan Barrett, Robin Lorenz, and Ognian Oreshkov. *Quantum Causal Models*. 2019. DOI: [10.48550/ARXIV.1906.10726](https://arxiv.org/abs/1906.10726). URL: <https://arxiv.org/abs/1906.10726>
- [8] S. Basu, R. Pollack, and M.F. Coste-Roy. *Algorithms in Real Algebraic Geometry*. Algorithms and Computation in Mathematics. Springer Berlin Heidelberg, 2007. ISBN: 9783540330998. URL: <https://books.google.de/books?id=8Hyp1SLis4IC>
- [9] J. S. Bell. “On the Einstein Podolsky Rosen paradox”. In: *Physics Physique Fizika* 1 (3 1964), pp. 195–200. DOI: [10.1103/PhysicsPhysiqueFizika.1.195](https://link.aps.org/doi/10.1103/PhysicsPhysiqueFizika.1.195). URL: <https://link.aps.org/doi/10.1103/PhysicsPhysiqueFizika.1.195>
- [10] Norman Biggs, Norman Linstead Biggs, and Biggs Norman. *Algebraic graph theory*. 67. Cambridge university press, 1993.
- [11] Bruce Blackadar. *Operator algebras: theory of C^* -algebras and von Neumann algebras*. Vol. 122. Springer Science & Business Media, 2006.
- [12] Paolo Boldi and Sebastiano Vigna. “Fibrations of graphs”. In: *Discrete Mathematics* 243.1-3 (2002), pp. 21–66.
- [13] R.V. Book and F. Otto. *String-Rewriting Systems*. Monographs in Computer Science. Springer New York, 2012. ISBN: 9781461397717. URL: <https://books.google.de/books?id=GV7SBwAAQBAJ>
- [14] Stephen Boyd and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.
- [15] Kristijan Cafuta, Igor Klep, and Janez Povh. “NCSOSTools: a computer algebra system for symbolic and numerical computation with noncommutative polynomials”. In: *Optim. Methods Softw.* 26.3 (2011), pp. 363–380. DOI: [10.1080/10556788.2010.544312](https://doi.org/10.1080/10556788.2010.544312). URL: <https://doi.org/10.1080/10556788.2010.544312>
- [16] Tullio Ceccherini-Silberstein and Michel Coornaert. “The Banach-Alaoglu Theorem”. In: *Cellular Automata and Groups*. Springer, 2010, pp. 383–385.

- [17] Rafael Chaves, Christian Majenz, and David Gross. “Information–theoretic implications of quantum causal structures”. In: *Nature Communications* 6.1 (2015), p. 5766. DOI: [10.1038/ncomms6766](https://doi.org/10.1038/ncomms6766). URL: <https://doi.org/10.1038/ncomms6766>.
- [18] Rafael Chaves et al. “Inferring latent structures via information inequalities”. In: *arXiv preprint arXiv:1407.2256* (2014).
- [19] B. S. Cirel’son. “Quantum generalizations of Bell’s inequality”. In: *Letters in Mathematical Physics* 4.2 (1980), pp. 93–100. DOI: [10.1007/BF00417500](https://doi.org/10.1007/BF00417500). URL: <https://doi.org/10.1007/BF00417500>.
- [20] Fabio Costa and Sally Shrapnel. “Quantum causal modelling”. In: *New Journal of Physics* 18.6 (2016), p. 063032. DOI: [10.1088/1367-2630/18/6/063032](https://doi.org/10.1088/1367-2630/18/6/063032). URL: <https://doi.org/10.1088/1367-2630/18/6/063032>.
- [21] John Daintith and Edmund Wright. *A Dictionary of Computing*. Oxford University Press, 2008. ISBN: 9780191726576. DOI: [10.1093/acref/9780199234004.001.0001](https://www.oxfordreference.com/view/10.1093/acref/9780199234004.001.0001). URL: <https://www.oxfordreference.com/view/10.1093/acref/9780199234004.001.0001/acref-9780199234004>.
- [22] Steven Diamond and Stephen Boyd. “CVXPY: A Python-embedded modeling language for convex optimization”. In: *Journal of Machine Learning Research* 17.83 (2016), pp. 1–5.
- [23] R. Diestel. *Graph Theory*. 5th. Springer Publishing Company, Incorporated, 2017. ISBN: 3662536218.
- [24] Michael P. Drazin. “Natural structures on semigroups with involution”. In: *Bulletin of the American Mathematical Society* 84.1 (1978), pp. 139–141. DOI: [bams/1183540393](https://doi.org/10.1183540393). URL: <https://doi.org/10.1183540393>.
- [25] D.S. Dummit and R.M. Foote. *Abstract Algebra*. Wiley, 2003. ISBN: 9780471433347. URL: <https://books.google.de/books?id=KJDBQgAACAAJ>.
- [26] A. Einstein, B. Podolsky, and N. Rosen. “Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?” In: *Phys. Rev.* 47 (10 1935), pp. 777–780. DOI: [10.1103/PhysRev.47.777](https://link.aps.org/doi/10.1103/PhysRev.47.777). URL: <https://link.aps.org/doi/10.1103/PhysRev.47.777>.
- [27] Artur K Ekert. “Quantum Cryptography and Bell’s Theorem”. In: *Quantum Measurements in Optics*. Springer, 1992, pp. 413–418.
- [28] R. J. Evans. “Model selection and local geometry”. In: *Annals of Statistics* (6 2020), pp. 3514–3544. arXiv: [1801.08364](https://arxiv.org/abs/1801.08364).
- [29] Martin Florig and Stephen J. Summers. “On the statistical independence of algebras of observables”. In: *Journal of Mathematical Physics* 38.3 (1997), pp. 1318–1328. DOI: [10.1063/1.531812](https://doi.org/10.1063/1.531812). eprint: <https://doi.org/10.1063/1.531812>. URL: <https://doi.org/10.1063/1.531812>.
- [30] Tobias Fritz. “TSIRELSON’S PROBLEM AND KIRCHBERG’S CONJECTURE”. In: *Reviews in Mathematical Physics* 24.05 (2012), p. 1250012. DOI: [10.1142/S0129055x12500122](https://doi.org/10.1142/S0129055x12500122). URL: <https://doi.org/10.1142/S0129055x12500122>.

- [31] TOBIAS FRITZ. “TSIRELSON'S PROBLEM AND KIRCHBERG'S CONJECTURE”. In: *Reviews in Mathematical Physics* 24.05 (2012), p. 1250012. DOI: [10.1142/s0129055x12500122](https://doi.org/10.1142/s0129055x12500122) URL: <https://doi.org/10.1142%2Fs0129055x12500122>
- [32] Dan Geiger and Christopher Meek. “Quantifier Elimination for Statistical Problems”. In: *Proceedings of the Fifteenth Conference on Uncertainty in Artificial Intelligence*. UAI'99. Stockholm, Sweden: Morgan Kaufmann Publishers Inc., 1999, 226–235. ISBN: 1558606149.
- [33] Jean Charles Gilbert and Cédric Jozs. *Plea for a semidefinite optimization solver in complex numbers*. Research Report. Inria Paris, Mar. 2017, p. 35. URL: <https://hal.inria.fr/hal-01422932>
- [34] Nicolas Gisin et al. “Constraints on nonlocality in networks from no-signaling and independence”. In: *Nature Communications* 11.1 (2020). DOI: [10.1038/s41467-020-16137-4](https://doi.org/10.1038/s41467-020-16137-4) URL: <https://doi.org/10.1038%2Fs41467-020-16137-4>
- [35] Michel X. Goemans and David P. Williamson. “Approximation algorithms for Max-3-Cut and other problems via complex semidefinite programming”. In: *Journal of Computer and System Sciences* 68.2 (2004). Special Issue on STOC 2001, pp. 442–470. ISSN: 0022-0000. DOI: <https://doi.org/10.1016/j.jcss.2003.07.012> URL: <https://www.sciencedirect.com/science/article/pii/S0022000003001454>
- [36] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2022. URL: <https://www.gurobi.com>
- [37] A. Gut. *Probability: A Graduate Course*. Springer, 2005.
- [38] Joe Henson, Raymond Lal, and Matthew F Pusey. “Theory-independent limits on correlations from generalized Bayesian networks”. In: *New Journal of Physics* 16.11 (2014), p. 113043. DOI: [10.1088/1367-2630/16/11/113043](https://doi.org/10.1088/1367-2630/16/11/113043) URL: <https://doi.org/10.1088%2F1367-2630%2F16%2F11%2F113043>
- [39] Ryszard Horodecki et al. “Quantum entanglement”. In: *Reviews of Modern Physics* 81.2 (2009), pp. 865–942. DOI: [10.1103/revmodphys.81.865](https://doi.org/10.1103/revmodphys.81.865) URL: <https://doi.org/10.1103%2Frevmodphys.81.865>
- [40] Wolfram Research, Inc. *Mathematica, Version 13.0.0*. Champaign, IL, 2021. URL: <https://www.wolfram.com/mathematica>
- [41] Zhengfeng Ji et al. *MIP*=RE*. 2020. DOI: [10.48550/ARXIV.2001.04383](https://arxiv.org/abs/2001.04383) URL: <https://arxiv.org/abs/2001.04383>
- [42] M. Junge et al. “Connes' embedding problem and Tsirelson's problem”. In: *Journal of Mathematical Physics* 52.1 (2011), p. 012102. DOI: [10.1063/1.3514538](https://doi.org/10.1063/1.3514538) URL: <https://doi.org/10.1063%2F1.3514538>
- [43] Donald Knuth. *The Art Of Computer Programming, vol. 3: Sorting And Searching*. Addison-Wesley, 1973.
- [44] Klaas Landsman. *Foundations of quantum theory: From classical concepts to operator algebras*. Springer Nature, 2017.

- [45] Ciarán M. Lee and Robert W. Spekkens. “Causal Inference via Algebraic Geometry: Feasibility Tests for Functional Causal Structures with Two Binary Observed Variables”. In: *Journal of Causal Inference* 5.2 (2017). DOI: [10.1515/jci-2016-0013](https://doi.org/10.1515/jci-2016-0013). URL: <https://doi.org/10.1515%2Fjci-2016-0013>
- [46] Laurens T. Ligthart, Mariami Gachechiladze, and David Gross. *A convergent inflation hierarchy for quantum causal structures*. 2021. DOI: [10.48550/ARXIV.2110.14659](https://arxiv.org/abs/2110.14659). URL: <https://arxiv.org/abs/2110.14659>
- [47] Sara Magliacane, Tom Claassen, and Joris M. Mooij. “Ancestral Causal Inference”. In: *Proceedings of the 30th International Conference on Neural Information Processing Systems*. NIPS’16. Barcelona, Spain: Curran Associates Inc., 2016, 4473–4481. ISBN: 9781510838819.
- [48] MATLAB. *version 7.10.0 (R2010a)*. Natick, Massachusetts: The MathWorks Inc., 2010.
- [49] Aaron Meurer et al. “SymPy: symbolic computing in Python”. In: *PeerJ Computer Science* 3 (Jan. 2017), e103. ISSN: 2376-5992. DOI: [10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103). URL: <https://doi.org/10.7717/peerj-cs.103>
- [50] Felipe Montealegre-Mora et al. *Certifying Numerical Decompositions of Compact Group Representations*. 2021. DOI: [10.48550/ARXIV.2101.12244](https://arxiv.org/abs/2101.12244). URL: <https://arxiv.org/abs/2101.12244>
- [51] Teo Mora. “An introduction to commutative and noncommutative Gröbner bases”. In: *Theoretical Computer Science* 134.1 (1994), pp. 131–173. ISSN: 0304-3975. DOI: [https://doi.org/10.1016/0304-3975\(94\)90283-6](https://doi.org/10.1016/0304-3975(94)90283-6). URL: <https://www.sciencedirect.com/science/article/pii/0304397594902836>
- [52] Tobias Moroder et al. “Device-Independent Entanglement Quantification and Related Applications”. In: *Physical Review Letters* 111.3 (2013). DOI: [10.1103/physrevlett.111.030501](https://doi.org/10.1103/physrevlett.111.030501). URL: <https://doi.org/10.1103%2Fphysrevlett.111.030501>
- [53] Lawrence Narici and Edward Beckenstein. “The Hahn-Banach theorem: the life and times”. In: *Topology and its Applications* 77.2 (1997), pp. 193–211. ISSN: 0166-8641. DOI: [https://doi.org/10.1016/S0166-8641\(96\)00142-3](https://doi.org/10.1016/S0166-8641(96)00142-3). URL: <https://www.sciencedirect.com/science/article/pii/S0166864196001423>
- [54] Miguel Navascués, Stefano Pironio, and Antonio Acín. “A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations”. In: *New Journal of Physics* 10.7 (2008), p. 073013. DOI: [10.1088/1367-2630/10/7/073013](https://doi.org/10.1088/1367-2630/10/7/073013). URL: <https://doi.org/10.1088%2F1367-2630%2F10%2F7%2F073013>
- [55] Miguel Navascués, Stefano Pironio, and Antonio Acín. “Bounding the Set of Quantum Correlations”. In: *Physical Review Letters* 98.1 (2007). DOI: [10.1103/physrevlett.98.010401](https://doi.org/10.1103/physrevlett.98.010401). URL: <https://doi.org/10.1103%2Fphysrevlett.98.010401>
- [56] Miguel Navascués and Elie Wolfe. “The Inflation Technique Completely Solves the Causal Compatibility Problem”. In: *Journal of Causal Inference* 8.1 (2020), pp. 70–91. DOI: [10.1515/jci-2018-0008](https://doi.org/10.1515/jci-2018-0008). URL: <https://doi.org/10.1515%2Fjci-2018-0008>

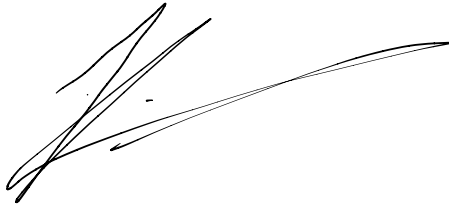
- [57] Brendan O’Donoghue et al. “Conic Optimization via Operator Splitting and Homogeneous Self-Dual Embedding”. In: (2013). DOI: [10.48550/ARXIV.1312.3039](https://arxiv.org/abs/1312.3039). URL: <https://arxiv.org/abs/1312.3039>
- [58] Brendan O’Donoghue et al. “Conic Optimization via Operator Splitting and Homogeneous Self-Dual Embedding”. In: *Journal of Optimization Theory and Applications* 169.3 (2016), pp. 1042–1068. URL: <http://stanford.edu/~boyd/papers/scs.html>
- [59] Brendan O’Donoghue et al. *SCS: Splitting Conic Solver, version 3.2.1*. <https://github.com/cvxgrp/scs> Nov. 2021.
- [60] Danilo Elias Oliveira, Henry Wolkowicz, and Yangyang Xu. *ADMM for the SDP relaxation of the QAP*. 2015. DOI: [10.48550/ARXIV.1512.05448](https://arxiv.org/abs/1512.05448) URL: <https://arxiv.org/abs/1512.05448>.
- [61] Vern Paulsen. *Completely Bounded Maps and Operator Algebras*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 2003. DOI: [10.1017/CB09780511546631](https://doi.org/10.1017/CB09780511546631).
- [62] J. Pearl. *Causality: Models, Reasoning and Inference*. 2nd. USA: Cambridge University Press, 2009. ISBN: 052189560X.
- [63] Judea Pearl and Thomas Verma. “The Logic of Representing Dependencies by Directed Graphs”. In: *Proceedings of the 6th National Conference on Artificial Intelligence. Seattle, WA, USA, July 1987*. Ed. by Kenneth D. Forbus and Howard E. Shrobe. Morgan Kaufmann, 1987, pp. 374–379. URL: <http://www.aaai.org/Library/AAAI/1987/aaai87-067.php>
- [64] Jacques Pienaar and Časlav Brukner. “A graph-separation theorem for quantum causal models”. In: *New Journal of Physics* 17.7 (2015), p. 073020. DOI: [10.1088/1367-2630/17/7/073020](https://doi.org/10.1088/1367-2630/17/7/073020) URL: <https://doi.org/10.1088/1367-2630/17/7/073020>
- [65] S. Pironio, M. Navascués, and A. Acín. “Convergent Relaxations of Polynomial Optimization Problems with Noncommuting Variables”. In: *SIAM Journal on Optimization* 20.5 (2010), pp. 2157–2180. DOI: [10.1137/090760155](https://doi.org/10.1137/090760155) URL: <https://doi.org/10.1137/090760155>
- [66] Stefano Pironio, Jean-Daniel Bancal, and Valerio Scarani. “Extremal correlations of the tripartite no-signaling polytope”. In: *Journal of Physics A: Mathematical and Theoretical* 44.6 (2011), p. 065303. DOI: [10.1088/1751-8113/44/6/065303](https://doi.org/10.1088/1751-8113/44/6/065303) URL: <https://doi.org/10.1088/1751-8113/44/6/065303>
- [67] Sandu Popescu and Daniel Rohrlich. “Quantum nonlocality as an axiom”. In: *Foundations of Physics* 24.3 (1994), pp. 379–385. DOI: [10.1007/BF02058098](https://doi.org/10.1007/BF02058098) URL: <https://doi.org/10.1007/BF02058098>.
- [68] A. Pozas-Kerstjens. “Quantum information outside quantum information”. PhD thesis. Universitat Politècnica de Catalunya, 2019.
- [69] Alejandro Pozas-Kerstjens et al. “Bounding the Sets of Classical and Quantum Correlations in Networks”. In: *Physical Review Letters* 123.14 (2019). DOI: [10.1103/physrevlett.123.140503](https://doi.org/10.1103/physrevlett.123.140503) URL: <https://doi.org/10.1103/physrevlett.123.140503>

- [70] Miklós Rédei. “Reichenbach’s Common Cause Principle and Quantum Correlations”. In: *Non-locality and Modality*. Ed. by Tomasz Placek and Jeremy Butterfield. Dordrecht: Springer Netherlands, 2002, pp. 259–270. ISBN: 978-94-010-0385-8. DOI: [10.1007/978-94-010-0385-8_17](https://doi.org/10.1007/978-94-010-0385-8_17). URL: https://doi.org/10.1007/978-94-010-0385-8_17.
- [71] Denis Rosset. *SymDPoly: symmetry-adapted moment relaxations for noncommutative polynomial optimization*. 2018. DOI: [10.48550/ARXIV.1808.09598](https://arxiv.org/abs/1808.09598). URL: <https://arxiv.org/abs/1808.09598>.
- [72] Denis Rosset, Nicolas Gisin, and Elie Wolfe. “Universal bound on the cardinality of local hidden variables in networks”. In: *arXiv preprint arXiv:1709.00707* (2017).
- [73] Denis Rosset, Felipe Montealegre-Mora, and Jean-Daniel Bancal. “RepLAB: A Computational/Numerical Approach to Representation Theory”. In: *Quantum Theory and Symmetries*. Springer International Publishing, 2021, pp. 643–653. DOI: [10.1007/978-3-030-55777-5_60](https://doi.org/10.1007/978-3-030-55777-5_60). URL: https://doi.org/10.1007/978-3-030-55777-5_60.
- [74] W. Rudin. *Functional Analysis*. International series in pure and applied mathematics. McGraw-Hill, 1991. ISBN: 9780070542365. URL: https://books.google.de/books?id=Sh_vAAAAAAAJ.
- [75] William Slofstra. “The set of quantum correlations is not closed”. In: *Forum of Mathematics, Pi*. Vol. 7. Cambridge University Press. 2019.
- [76] M. Takesaki. *Theory of Operator Algebras I*. Encyclopaedia of Mathematical Sciences. Springer Berlin Heidelberg, 2001. ISBN: 9783540422488. URL: <https://books.google.de/books?id=dTnq4hjjtGMC>.
- [77] B.S. Tsirelson. “Bell inequalities and operator algebras:” in: (2006). URL: <http://www.imaph.tu-bs.de/qi/problems/33.html>.
- [78] Frank Vallentin. “Symmetry in semidefinite programs”. In: *Linear Algebra and its Applications* 430 (June 2007), pp. 360–369. DOI: [10.1016/j.laa.2008.07.025](https://doi.org/10.1016/j.laa.2008.07.025).
- [79] Lieven Vandenbergh and Stephen Boyd. “Semidefinite Programming”. In: *SIAM Review* 38.1 (1996), pp. 49–95. DOI: [10.1137/1038003](https://doi.org/10.1137/1038003). eprint: <https://doi.org/10.1137/1038003>. URL: <https://doi.org/10.1137/1038003>.
- [80] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [81] Stephanie Wehner. “Tsirelson bounds for generalized Clauser-Horne-Shimony-Holt inequalities”. In: *Physical Review A* 73.2 (2006). DOI: [10.1103/physreva.73.022110](https://doi.org/10.1103/physreva.73.022110). URL: <https://doi.org/10.1103/physreva.73.022110>.
- [82] R. F. Werner and M. M. Wolf. “All-multipartite Bell-correlation inequalities for two dichotomic observables per site”. In: *Phys. Rev. A* 64 (3 2001), p. 032112. DOI: [10.1103/PhysRevA.64.032112](https://link.aps.org/doi/10.1103/PhysRevA.64.032112). URL: <https://link.aps.org/doi/10.1103/PhysRevA.64.032112>.

- [83] Reinhard F. Werner and Michael M. Wolf. *Bell inequalities and Entanglement*. 2001. DOI: [10.48550/ARXIV.QUANT-PH/0107093](https://arxiv.org/abs/quant-ph/0107093). URL: <https://arxiv.org/abs/quant-ph/0107093>.
- [84] Angelika Wiegele. “Nonlinear optimization techniques applied to combinatorial optimization problems / Angelika Wiegele”. In: 2006.
- [85] Peter Wittek. “Algorithm 950”. In: *ACM Transactions on Mathematical Software* 41.3 (2015), pp. 1–12. DOI: [10.1145/2699464](https://doi.org/10.1145/2699464). URL: <https://doi.org/10.1145/2699464>.
- [86] Elie Wolfe, Robert W. Spekkens, and Tobias Fritz. “The Inflation Technique for Causal Inference with Latent Variables”. In: *Journal of Causal Inference* 7.2 (2019). DOI: [10.1515/jci-2017-0020](https://doi.org/10.1515/jci-2017-0020). URL: <https://doi.org/10.1515/jci-2017-0020>.
- [87] Elie Wolfe et al. “Quantum Inflation: A General Approach to Quantum Causal Compatibility”. In: *Physical Review X* 11.2 (2021). DOI: [10.1103/physrevx.11.021043](https://doi.org/10.1103/physrevx.11.021043). URL: <https://doi.org/10.1103/physrevx.11.021043>.
- [88] Christopher J Wood and Robert W Spekkens. “The lesson of causal discovery algorithms for quantum correlations: causal explanations of Bell-inequality violations require fine-tuning”. In: *New Journal of Physics* 17.3 (2015), p. 033002. DOI: [10.1088/1367-2630/17/3/033002](https://doi.org/10.1088/1367-2630/17/3/033002). URL: <https://doi.org/10.1088/1367-2630/17/3/033002>.
- [89] William K Wootters and Wojciech H Zurek. “The no-cloning theorem”. In: *Physics Today* 62.2 (2009), pp. 76–77.
- [90] Richard Y. Zhang and Javad Lavaei. “Sparse semidefinite programs with guaranteed near-linear time complexity via dualized clique tree conversion”. In: *Mathematical Programming* 188.1 (2020), pp. 351–393. DOI: [10.1007/s10107-020-01516-y](https://doi.org/10.1007/s10107-020-01516-y). URL: <https://doi.org/10.1007/s10107-020-01516-y>.

Statement of Authorship

Hiermit versichere ich an Eides statt, dass ich die vorliegende Arbeit selbstständig und ohne die Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten und nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden. Ich versichere, dass die eingereichte elektronische Fassung der eingereichten Druckfassung vollständig entspricht.

A handwritten signature in black ink, consisting of several fluid, overlapping strokes that form a stylized representation of the name Julius Alexander Zeiss.

Julius Alexander Zeiss